

密 级： 公开

加密论文编号：

论文题目： 面向云原生应用权限提升漏洞的动态防
控技术研究

学 号： M202310652

研 究 生： 韩晓阳

专 业 名 称： 计算机科学与技术

2026 年 5 月 1 日

面向云原生应用权限提升漏洞的动态防控技术研究

**Research on Dynamic Prevention and Control
Technologies for Privilege Escalation
Vulnerabilities in Cloud-Native Applications**

研究生：韩晓阳

指导教师：孙昌爱

北京科技大学 计算机与通信工程学院

北京 100083，中国

Doctor Degree Candidate: Xiaoyang Han

Supervisor: Chang'ai Sun

School of Computer and Communication Engineering

University of Science and Technology Beijing

30 Xueyuan Road, Haidian District

Beijing 100083, P.R.CHINA

中图分类号:

学校代码:

10008

U D C:

密 级:

公开

北京科技大学硕士学位论文

论文题目: 面向云原生应用权限提升漏洞的动态防控技术研究

研究生: 韩晓阳

指 导 教 师: 孙昌爱 单位: 北京科技大学 职称: 教授

指 导 小 组 成 员: 单位: 职称:

 单位: 职称:

论文提交日期: 2026 年 5 月 1 日

学位授予单位: 北 京 科 技 大 学

摘 要

论文中文摘要字数约为 300~600 字，如遇特殊需要字数可以略多，限一页。

论文摘要是论文内容不加注释和评论的简短陈述，一般以第三人称语气写成。摘要的编写应遵循下列原则：

1) 摘要应具有独立性和自含性，即不阅读论文的全文，就能获得必要的信息。摘要是学位论文的缩影，是学位论文的主要内容、见解、结论简短明了的缩写。

2) 摘要应是一篇完整的短文，可以独立使用，可以引用。

3) 摘要的内容应包含与论文等量的主要信息，供读者确定有无必要阅读全文，也可供文摘汇编等二次文献采用。

4) 摘要一般应说明研究工作的目的意义、研究方法、研究结果、主要结论及意义、创造性成果和新见解，而重点是结论和创新点。

5) 要用文字表达，不要附图和照片，除了实在无变通办法可用以外，摘要中不用图、表、化学结构式、非公知公用的符号和术语，不要使用表格、公式、上下标以及其他特殊符号，要突出重点，阐述清楚，少用数据表。论文摘要用语力求简洁、准确。原则上 300~600 字。

关键词：摘要；论文；要求；字数；格式（关键词个数为 3~5 个，正式写作请删除此括号）

Research on Dynamic Prevention and Control Technologies for Privilege Escalation Vulnerabilities in Cloud-Native Applications

ABSTRACT

In environmental economics, environmental resources including environmental quality are categorized as amenity resources. Due to its importance to human welfare, the amenity resources theoretical study and valuation is an ongoing issue at the academic frontier in the environmental economics area.

注：论文的英文摘要应有英文题目和关键词，内容与中文摘要相同，用另页置于中文摘要之后；外文摘要实词在 300 个左右。

Key Words: Key word 1; Key word 2; Key word 3; ...

目 录

摘要	I
ABSTRACT	III
1 引言	1
1.1 背景与意义	1
1.2 论文组织结构	4
2 背景介绍	5
2.1 相关概念与技术	5
2.1.1 容器	5
2.1.2 Kubernetes 架构与工作原理	6
2.1.3 Kubernetes 权限模型	7
2.1.4 Kubernetes 权限提升漏洞	7
2.2 云原生安全工具	9
2.2.1 监控策略：容器运行时安全监控工具——Falco	9
2.2.2 防测一体：Cilium Tetragon	10
2.2.3 防护策略：准入控制与配置约束工具	11
2.2.4 防护策略：网络隔离工具——NetworkPolicy	12
2.2.5 静态扫描与配置审计工具：镜像、依赖与 IaC	13
2.3 国内外研究现状	14
2.3.1 系统权限安全问题	14
2.3.2 Kubernetes 安全问题	15
2.3.3 本课题组研究基础	18
2.4 小节	21
3 面向云原生权限提升漏洞的智能防控技术	23
3.1 形式化定义	23
3.1.1 基本对象	23
3.1.2 评价与选择	24
3.2 开源应用漏洞安全知识库构建	25
3.2.1 漏洞收集与分类框架	25
3.2.2 知识库构建	28
3.3 基于多智能体协同的策略生成与优化	28
3.3.1 安全上下文构建	29
3.3.2 基于漏洞分析思维链的策略生成	30
3.3.3 基于评审反馈的策略优化	35

3.4 策略评估与质量控制	39
3.4.1 评价算子	39
3.4.2 多智能体评估体系	41
3.4.3 多源排名聚合模型	43
3.5 小节	44
4 面向云原生权限提升智能化漏洞防控系统工具	45
4.1 需求分析	45
4.1.1 用户角色与用例分析	45
4.1.2 功能需求	45
4.2 总体设计	46
4.2.1 系统架构设计	46
4.2.2 数据流转与审计设计	47
4.2.3 交互时序设计	47
4.2.4 数据模型设计	48
4.3 详细设计	49
4.3.1 CVE 知识库管理	49
4.3.2 智能策略生成模块	49
4.3.3 策略部署管理	55
5 实验研究	57
5.1 研究问题	57
5.2 实验设置	57
5.3 RQ1: 消融实验与 workflow 模块贡献	58
5.3.1 workflow 跨模型稳健性验证	60
5.4 RQ2: 生成模型能力评估	60
5.4.1 三维质量指标的能力分化	60
5.4.2 输出稳定性评估	62
5.4.3 生成模型能力评估小结	63
5.5 RQ3: 评审方法可信度评估	64
5.5.1 不同 LLM 的评审一致性验证	64
5.5.2 跨维度评审差异分析	67
5.5.3 与人工评审的对比验证	68
5.5.4 小结	71
5.6 RQ4: 静态扫描的处置价值	71
5.7 RQ5: 资源消耗与效率评估	72

6 案例分析	75
6.1 案例分析实验设计	75
6.1.1 CVE 案例选择标准	75
6.1.2 实验环境与复现流程	75
6.2 案例一：CVE-2022-24768 内部权限问题	77
6.2.1 漏洞详解与复现	77
6.2.2 策略部署	79
6.2.3 三因素分析	80
6.3 案例二：CVE-2025-2241 (Hive ClusterProvision 敏感信息泄漏)	80
6.3.1 漏洞详解与复现	80
6.3.2 策略部署	82
6.3.3 三因素分析	82
6.4 案例三：CVE-2023-30512 (CubeFS CSI 过度权限配置)	83
6.4.1 漏洞详解与复现	83
6.4.2 策略部署	85
6.4.3 三因素分析	86
6.5 案例四：CVE-2020-4062 Conjur OSS 缺乏网络隔离式	86
6.5.1 漏洞详解与复现	86
6.5.2 策略部署	88
6.5.3 三因素分析	88
6.6 案例五：CVE-2022-24877 (Flux kustomize-controller 路径穿 越)	89
6.6.1 漏洞详解与复现	89
6.6.2 策略部署	90
6.6.3 三因素分析	90
6.7 案例六：CVE-2022-29171 (Sourcegraph gitserver 远程代码执 行)	91
6.7.1 漏洞详解与复现	91
6.7.2 策略部署	92
6.7.3 三因素分析	92
6.8 案例七：CVE-2023-22482 (Argo CD OIDC aud 未校验导致越权 访问)	93
6.8.1 漏洞详解与复现	93
6.8.2 策略部署	94
6.8.3 三因素分析	94
6.9 小节	95

7 结论与展望	97
7.1 研究工作总结	97
7.1.1 理论建模与形式化框架	97
7.1.2 方法体系与技术创新	98
7.1.3 工程实践与实验验证	99
7.1.4 研究意义与价值	99
7.2 未来研究方向	100
7.2.1 高危漏洞的自动化监控与实时更新	100
7.2.2 漏洞利用的自动化复现与验证	101
7.2.3 研究范围向云原生其他漏洞类型扩展	101
附录 A 单位	111
附录 B 云原生权限提升漏洞分类与 CWE 对应关系	113
附录 C CVE-2022-24768 复现步骤与策略清单	115
C.1 漏洞复现详细步骤	115
C.2 策略代码清单	117
C.2.1 Kubernetes RBAC: 收敛 AppProject 的写权限	117
C.2.2 Argo CD RBAC: 默认只读 + 高风险能力隔离	119
C.2.3 准入控制: Gatekeeper (推荐)	120
C.2.4 准入控制: Kyverno (替代方案)	124
附录 D CVE-2025-2241 复现步骤与策略清单	129
D.1 漏洞复现详细步骤	129
D.2 策略代码清单	130
D.2.1 1) 立刻收敛 ClusterProvision 的读取权限 (RBAC)	131
D.2.2 2) 对读取行为启用审计并联动告警	131
D.2.3 3) 缩短暴露窗口: 供应完成后清理 ClusterProvision	132
D.2.4 4) vCenter 凭据轮换与强化 (止血)	135
D.2.5 5) “敏感字段守护”检测 (Gatekeeper/Kyverno, 建议审计模式)	136
附录 E CVE-2023-30512 复现步骤与策略清单	141
E.1 漏洞复现详细步骤	141
E.2 策略代码清单	142
E.2.1 1) RBAC 最小化: 移除 ClusterRole 中的 secrets 权限	142
E.2.2 2) ServiceAccount 硬化 (按需评估)	143
E.2.3 3) 准入防回归 (OPA Gatekeeper): 禁止创建包含 secrets 读 取动词的 ClusterRole	144
E.2.4 4) 审计与检测: 对 secrets 枚举行为进行可观测化	145

E.2.5 5) 网络收敛 (可选): 限制 CSI Pod 出站访问面	146
附录 F CVE-2020-4062 复现步骤与策略清单	149
F.1 漏洞复现详细步骤	149
F.2 策略代码清单	150
F.2.1 1) 缓解措施: NetworkPolicy, 默认拒绝并仅放通 Conjur Server 访问 Postgres 5432	150
F.2.2 2) 命名空间隔离与 RBAC 收敛	151
附录 G CVE-2022-24877 复现步骤与策略清单	153
G.1 漏洞复现详细步骤	153
G.2 策略代码清单 (优先级方案)	154
G.2.1 1) 工作区缓解: 在 CI/CD 中校验 kustomization.yaml 路 径策略	154
G.2.2 2) 多租户变更治理与运行时硬化 (纵深防御)	155
附录 H CVE-2022-29171 复现步骤与策略清单	157
H.1 漏洞复现详细步骤	157
H.2 策略清单	158
H.2.1 1) 快速缓解: 禁用或移除 Gitolite code host (若业务允许) ..	158
H.2.2 2) 网络隔离: 限制 gitserver 的可达面 (默认拒绝并按需 放通)	158
H.2.3 3) 权限分离与暴露面收敛: 限制管理面与观测面的高权限 ...	160
附录 I CVE-2023-22482 复现步骤与策略清单	161
I.1 漏洞复现详细步骤	161
I.2 策略清单	162
I.2.1 1) 身份提供方治理: 独立客户端与唯一受众	162
I.2.2 2) 入口补偿控制: 前置强制校验 aud	162
I.2.3 3) 权限收敛: RBAC 最小化与项目边界约束	162
致谢	163
作者简历及在学研究成果	165
独创性说明	167
关于论文使用授权的说明	169
学位论文数据集	171

1 引言

本章节主要介绍课题的研究背景与意义、研究内容与成果，以及论文的组织结构。

1.1 背景与意义

容器技术通过内核级资源隔离与标准化的镜像分发机制，显著优化了微服务应用的生命周期管理。作为容器编排领域的核心基础设施 [1], Kubernetes 提供的分布式容器编排能力有效支撑了复杂业务的部署与扩展 [2, 3]。为满足多样化的服务治理需求，现代 Kubernetes 集群需深度集成云原生计算基金会 CNCF 开源生态中的第三方应用，各类组件通过基于角色的访问控制 RBAC 机制 [4] 获取集群资源访问权限。第三方应用在赋能业务的同时亦引入了潜在攻击面：在多租户场景下，过度权限配置 [5]、内部权限管理缺陷、敏感信息泄露、网络隔离缺失等多维因素均可能诱发权限提升风险，对云原生系统的整体安全构成系统性威胁。

围绕云原生安全治理，相关研究与工程实践已形成较为丰富的工具链与方法体系：在静态侧，面向镜像依赖与 Kubernetes 清单、IaC 模板的漏洞与误配置检测被广泛用于风险前置发现与持续治理 [6, 7]；在运行时侧，基于系统调用或 eBPF 事件的观测与审计可为异常行为检测与取证提供支撑 [8, 9]；在防护侧，准入控制与配置约束机制可在资源持久化前阻断不合规对象进入集群 [10, 11]，网络隔离与最小连通收敛则用于降低横向移动风险 [12, 13]。然而，上述工作在应对权限提升漏洞的补丁窗口期场景时仍存在明显缺口：首先，多数方法以通用静态规则或单一方向检测为主，难以针对特定 CVE 的利用前置条件与攻击链路给出精准处置建议 [6, 14]；其次，多数方案侧重发现问题而非压制利用，缺少可在生产环境中快速落地且对业务影响可控的临时防控方案 [15]；最后，策略从生成到落地往往依赖专家经验进行手工拼装，难以在复杂异构环境中形成可复用、可规模化的工程流程 [15]。

云原生第三方开源应用的权限提升漏洞从披露到修复通常面临显著的响应滞后。根据本文对 2020 年至 2025 年间 55 个云原生权限提升漏洞样本的统计分析，30.4% 的漏洞在披露后存在较长的攻击窗口期，如图 1-1 所示。开源应用官方补丁的发布、验证与分发存在固有周期，管理失职则会进一步拉

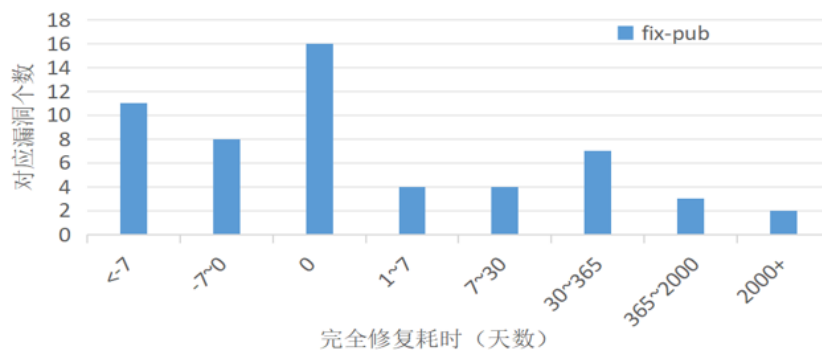


图 1-1: 云原生环境下权限提升漏洞的修复时间分布

长攻击窗口；同时，由于生产环境组件组合复杂且兼容性要求严格，用户往往难以及时完成受影响组件的升级。在补丁尚未就绪或升级受阻的阶段，企业亟需可快速下发的临时防控措施，通过压制攻击路径来降低风险暴露面。

基于上述现实需求，本文聚焦于云原生权限提升漏洞在补丁窗口期内的动态防控技术。从防控机制的作用维度考量，本文将云原生环境下的动态控制手段归纳为监控策略与防护策略两个互补层次：监控策略侧重运行时异常行为的感知与取证，防护策略侧重事前阻断与攻击面收敛。

防护策略层面向权限提升攻击链的事前阻断与攻击面收敛，通过准入控制、网络隔离与权限加固三类机制构建多层防御体系，具体包括：

- 准入控制与配置约束：借助 OPA/Gatekeeper[10] 或 Kyverno[11] 等准入控制器，在资源对象持久化前对特权容器、危险路径挂载及异常 RBAC 绑定等高风险配置实施强制阻断；
- 网络隔离与连通收敛：基于 NetworkPolicy 构建微隔离边界，将工作负载间可达性收敛至最小必要集合，从而抑制横向移动与数据外传路径；
- 权限与配置动态加固：面向漏洞利用路径，通过收紧 RBAC 授权边界、移除冗余权限及调整容器运行时安全上下文，压缩攻击成功概率。

监控策略层聚焦于运行时异常行为的持续观测与取证能力构建，为威胁检测与应急响应提供实时证据链，具体包括：

- 运行时行为监测与审计：基于 Falco[8] 等系统调用层监控工具，对异常系统调用、特权变更与敏感文件访问等事件进行持续观测与告警，为取证及应急响应提供依据；
- 防测一体化机制：Cilium Tetragon[9] 基于 eBPF 的内核层安全观测与运行时强制执行能力，可对进程执行、系统调用与 I/O 活动进行细粒度监测，并在必要时实施策略过滤或阻断，实现监控防护一体化。

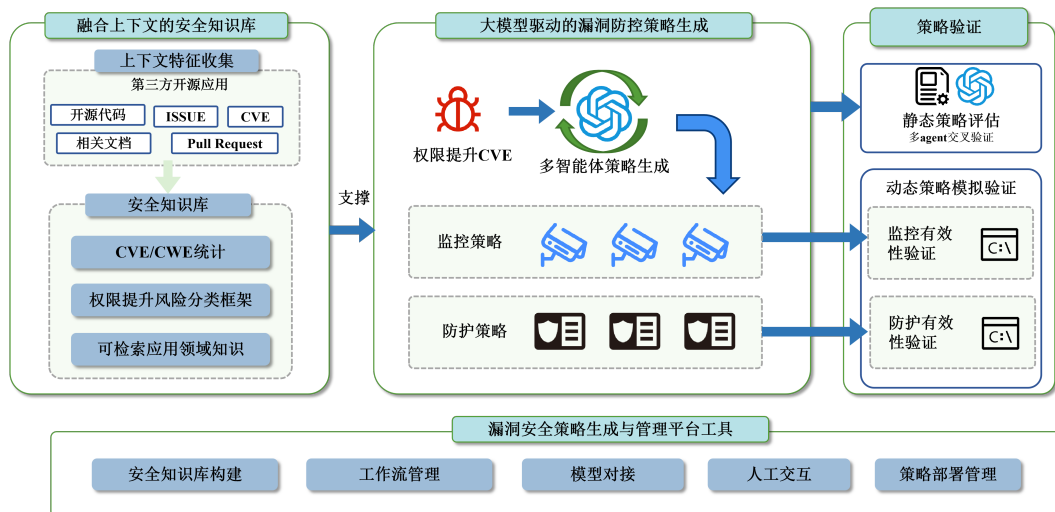


图 1-2: 基于大语言模型的多智能体漏洞策略化防控总体框架

如图 1-2所示，本文提出的智能防控框架以权限提升类 CVE 为输入，通过融合上下文的安全知识库支撑、大语言模型驱动的多智能体协同策略生成、以及静态与动态相结合的质量评估三个核心阶段，实现从漏洞披露到临时防控策略交付的全流程自动化。整体架构遵循“证据支撑—智能推理—验证交付”的闭环逻辑，旨在补丁窗口期内为运维团队提供可快速部署的监控策略与防护策略。

（1）融合上下文的安全知识库。该模块面向第三方开源应用场景，通过上下文特征收集机制汇聚多源证据：从开源代码仓库提取功能实现与权限配置逻辑，从 Issue 与 Pull Request 追溯漏洞成因与修复演变，从 CVE 公告及相关文档获取攻击面描述与影响范围标注。在此基础上，知识库以向量化索引方式组织安全知识库，包含 CVE 与 CWE 统计、权限提升风险分类框架、以及可检索的应用领域知识，为后续策略生成提供结构化证据链与领域先验约束。

（2）大模型驱动的多智能体漏洞防控策略生成。该模块以多智能体协同机制实现从漏洞语义到可执行策略的转化。流程起始于知识库对输入 CVE 的证据检索与风险分类标注；随后，多智能体策略生成环节通过安全知识智能体整理上下文、漏洞分析智能体输出结构化威胁分析、策略生成智能体分别产出监控策略与防护策略草案，并由策略评审与优化智能体基于评审反馈进行迭代修订。监控策略侧重运行时行为的异常检测与取证能力构建，防护策略侧重准入控制、网络隔离与权限加固等事前阻断机制。该阶段的核心目标在于将漏洞攻击链条映射为可在 Kubernetes 平台上快速落地的控制动作序列 actions，并通过智能体间的交叉验证降低策略生成中的幻觉与不一致性。

(3) 策略验证。该模块对候选策略集合 Π_c 实施两阶段质量控制。在静态策略评估环节，评价智能体基于有效性 E 、无干扰性 I 、可部署性 D 三维质量向量对候选进行多主体交叉评审，输出维度排名与共识排序；在动态策略模拟验证环节，针对监控策略，于沙箱环境中注入攻击行为并评估监控有效性验证其能否准确捕获权限提升行为特征；针对防护策略，通过模拟漏洞利用路径验证其阻断能力与业务兼容性，确保策略既能有效覆盖关键攻击面，又不会对正常工作负载产生过度限制。两阶段验证结果共同构成策略筛选与排序的依据，并为最终交付提供可审计的质量证明。

(4) 漏洞安全策略生成与管理平台工具。为支撑上述流程的工程化落地，本文构建了平台化工具体系，涵盖安全知识库构建、 workflow 管理、模型对接、人工交互与策略部署管理五项能力。该平台通过模块化设计实现知识库维护与策略生成流程的解耦，支持多种大语言模型的灵活对接与评价主体的动态配置，并提供可视化界面用于人工复核与策略迭代优化，从而将智能防控能力转化为可持续运营的安全服务。

1.2 论文组织结构

全文结构如下：第2章介绍相关概念、技术与国内外研究现状；第3章给出面向云原生权限提升漏洞的智能防控技术体系；第4章描述漏洞防控系统设计实现；第5章呈现实验设置与结果；第6章给出案例分析；第7章总结与展望。

2 背景介绍

本章节主要介绍相关概念、技术，以及与 Kubernetes 权限提升漏洞相关的国内外研究现状。

2.1 相关概念与技术

2.1.1 容器

服务应用的部署经历从物理机部署、到虚拟机部署、再到容器化部署的演进过程 (见图2-1)。虚拟机部署基于 KVM 等虚拟化技术, 实现了完整操作系统的模拟, 相比物理机部署实现了资源的隔离与复用, 但存在虚拟化开销较大, 启动时间较长, 镜像体积庞大等问题, 影响了应用的交付效率和资源利用率。

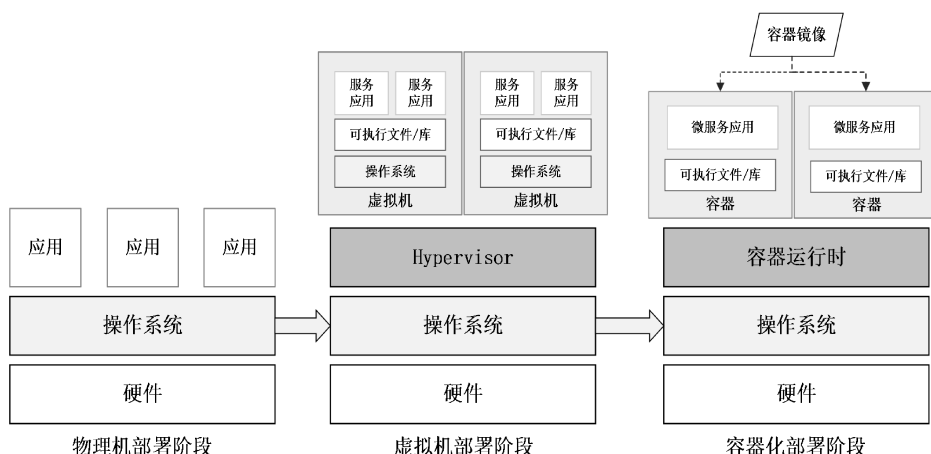
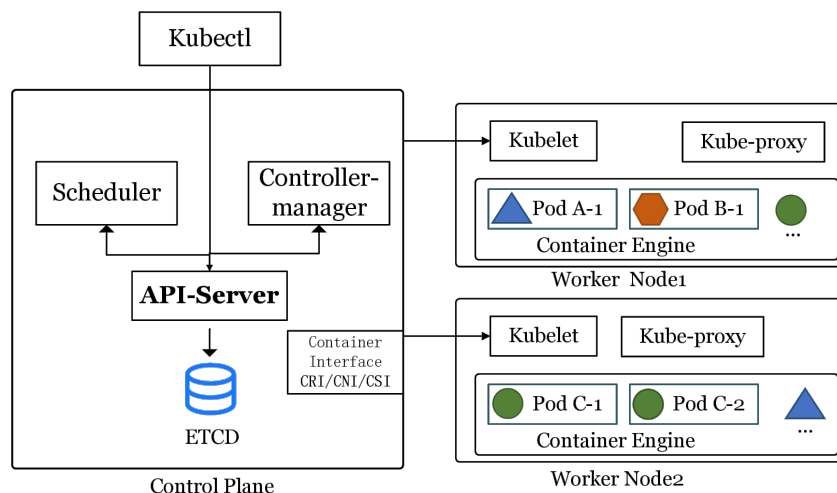


图 2-1: 服务部署形式演进

与传统的物理机和虚拟机相比, 容器通过操作系统内核级的资源隔离, 实现了更高的资源利用率和更快的启动速度。容器以**镜像**为交付单元, 讲部署应用及其依赖环境以镜像形式打包。从实现机制看, 容器的关键能力主要来自三类内核与运行时支撑: (1) **命名空间 (Namespace) 隔离**, 将进程、网络、挂载点、IPC、用户等资源视图隔离开来, 使容器内进程“看见”的系统环境与宿主机相互独立; (2) **控制组 (cgroup) 资源管控**, 对 CPU、内存、I/O 等资源进行限制与统计, 防止单个工作负载挤占宿主机资源; (3) **镜像与分层文件系统**, 将应用与依赖固化为可复用的只读层, 基于 OverlayFS 技术叠加读写层形成运行时文件系统, 同时基于切根 (chroot) 技术实现文件系统隔



Architecture of the Kubernetes

图 2-2: Kubernetes 体系结构示意图

离，使得容器内的进程只访问其专属的文件视图。

2.1.2 Kubernetes 架构与工作原理

Kubernetes 是面向容器工作负载的编排系统, 源自于谷歌内部的虚拟化编排系统 Borg[16] 并在 2014 年正式对外开源。其核心思想是以**声明式 API** 描述期望状态, 再由控制平面持续驱动实际状态向期望状态收敛 [2]。图2-2所示 Kubernetes 的体系结构示意图, 一个 Kubernetes 集群由**控制平面**与多个**工作节点**构成。控制平面负责提供控制接口、调度与集群控制逻辑, 其中: *API Server* 是所有资源访问与状态变更的统一入口, 基于键值数据库 ETCD 承担认证、授权、准入与资源对象持久化等关键职责; 调度器根据资源请求、亲和/反亲和等约束为待调度 Pod 选择工作节点; 控制器基于不同资源对象的当前状态触发协调动作以达到期望状态。工作节点侧, *kubelet* 负责与 *API Server* 通信并维护调度到本节点的容器运行, 容器运行时负责拉取镜像、创建容器并管理生命周期, *kube-proxy* 负责服务转发与网络规则的实现。

Kubernetes 通过一系列持久化**资源对象**实现对集群多种资源的编排与管理。例如, *Pod* 是最小调度单元, 包含一个或多个共享网络与存储命名空间的容器; *Deployment/StatefulSet/DaemonSet* 分别面向无状态、有状态与节点守护型工作负载提供期望副本与更新策略; *Service* 与 *Endpoint/EndpointSlice* 提供稳定的服务发现与负载均衡抽象; *Ingress* (或 *Gateway API*) 面向南北向流量提供入口控制; *ConfigMap/Secret* 用于配置与敏感信息的声明式注入; *Volume/PV/PVC* 则提供存储生命周期与绑定关系的抽象。

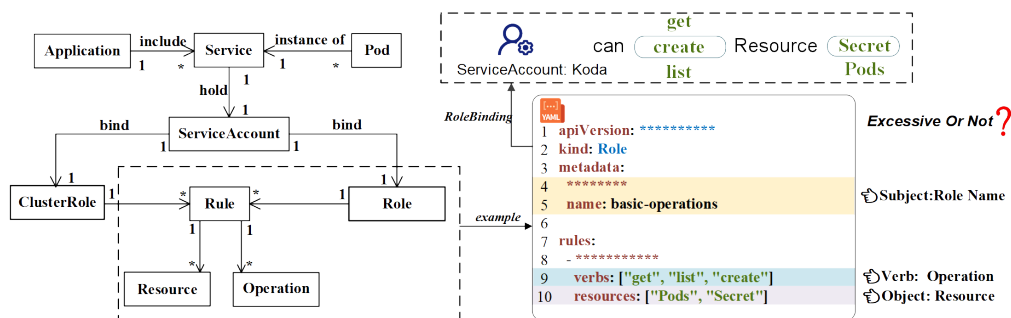


图 2-3: Kubernetes RBAC 权限配置 (YAML) 示意 [17]

2.1.3 Kubernetes 权限模型

基于角色的访问控制即 RBAC 构成了 Kubernetes 系统授权机制的核心架构 [2]。该模型以 ServiceAccount 作为工作负载的身份主体，并将权限抽象为 Role 或 ClusterRole 定义的规则集合。这些规则从 apiGroups 与 resources、操作动词 verbs 以及 namespace 或 cluster 作用域等维度，精确界定了允许的操作边界。通过 RoleBinding 或 ClusterRoleBinding 将身份主体与角色进行关联，主体方可获得针对特定资源的访问能力。在此基础上，Namespace 实现了租户与资源域的逻辑隔离，而 NetworkPolicy 机制则通过收敛网络可达性，有效抑制了集群内部的横向移动风险。在实践中，RBAC 遵循声明式配置范式，通过 YAML 清单定义规则映射，从而确保了权限管理过程的规范化与自动化。

如图 2-3 所示，该配置示例展示了典型的 RBAC 权限授予过程：首先创建名为 kada 的 ServiceAccount，随后定义名为 basic-operation 的 Role 对象并在其 rules 字段中指定资源范围与操作权限——此处授予了针对 pods 与 secrets 两类资源的 get、list、create 操作能力；最终通过 RoleBinding 将该 Role 绑定至 kada 账户，完成权限授予链路。此配置模式在第三方应用接入场景中较为常见，但若规则设计不当，过度授予的资源访问能力可能为权限提升攻击提供利用路径。

2.1.4 Kubernetes 权限提升漏洞

权限提升 (Privilege Escalation) 指攻击者在仅具备受限权限或初始执行能力的情况下，借助漏洞或错误配置，将自身权限提升到更高等级的过程 [18]。在 Kubernetes 场景下，权限提升既可能表现为授权范围的扩大或横向获取，也可能表现为节点侧系统权限提升（例如获得节点上的 root 权限），从而对更大范围的资源与工作负载产生影响。

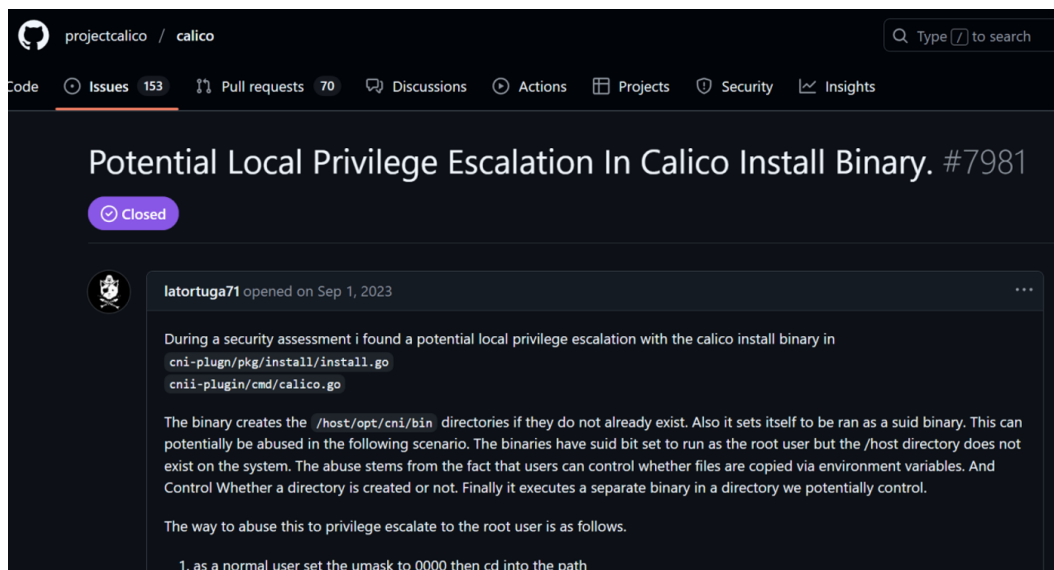


图 2-4: CVE-2024-33522（Calico）节点侧权限提升漏洞报告 ISSUE

在云原生环境中，权限提升的危害往往具有更强的放大效应。一方面，单个节点通常承载多个 Pod，节点侧提权成功后，攻击者可能对同节点上的多个工作负载实施窃取敏感信息、篡改运行环境或注入恶意组件等行为。另一方面，Kubernetes 的资源对象与运维动作高度集中且自动化；当攻击者获得更高权限后，可能进一步读取 Secret、创建高权限工作负载、修改网络防护策略或部署守护进程，进而将影响从单一应用扩展到集群层面的安全与稳定。因此，权限提升漏洞不仅是单点缺陷，还可能引发平台级安全事件与隔离失效。

为说明节点侧提权在云原生场景中的现实危害，图2-4给出了 CVE-2024-33522 的示例。该漏洞影响 Calico 的 CNI 安装程序：在部分易受影响版本中，安装二进制的 SUID 位配置不当，同时允许攻击者控制输入二进制。在攻击者已具备 Kubernetes 节点本地访问的前提下，攻击者可诱导安装过程以更高权限执行任意二进制，从而实现节点侧权限提升。虽然该漏洞的利用前提包含节点本地访问，但在真实环境中，这一前提可能由节点服务暴露、弱口令、供应链投毒或通过其他漏洞等其他入侵路径满足。一旦节点侧权限提升成功，攻击者可能读取或篡改节点容器运行时目录、配置与日志等关键数据，窃取工作负载凭据与令牌，干扰网络组件行为，并进一步扩大影响范围，从而显著加剧集群整体风险。

2.2 云原生安全工具

在补丁未就绪或难以及时升级的阶段，生产环境往往需要依赖可快速下发的控制手段来降低风险暴露面。从防控机制的作用时机与机理考量，云原生安全工具可划分为**监控策略与防护策略**两个互补层次：

- **监控策略**在不中断业务的前提下对异常行为进行告警与取证，主要涵盖运行时检测与审计、静态扫描与配置审计；
- **防护策略**在风险入口处主动阻断与收敛攻击面，包括准入与配置约束、网络隔离与最小连通；

下文分别介绍监控策略中的运行时检测工具 Falco 与静态扫描类工具、防测一体工具 Cilium Tetragon，以及防护策略中的准入控制工具 OPA/Gatekeeper 与 Kyverno、网络隔离机制 NetworkPolicy。

2.2.1 监控策略：容器运行时安全监控工具——Falco

在容器技术与云原生基础设施广泛部署的背景下，传统基于主机或网络的入侵监测系统（IDS）逐渐暴露出适应性不足、粒度较粗以及缺乏云原生上下文语义等问题。为满足容器化环境对运行时安全的实时性与上下文感知需求，开源项目 Falco 应运而生。Falco 最初由 Sysdig 公司开发，并于 2018 年捐赠给云原生计算基金会（CNCF），成为较早面向容器运行时的行为监测系统之一。其核心思想是在**系统调用层**进行透明监控，并通过规则语言定义**监控策略**，从而在不中断业务的前提下实现精细化威胁监测与审计。

Falco 的监测机制基于 **eBPF** 技术对 Linux 系统调用的动态捕获与分析。eBPF（extended Berkeley Packet Filter）允许在**内核态**运行经过验证的字节码程序，并以受控方式访问内核提供的上下文与辅助函数；eBPF 程序可动态挂载到系统调用入口/退出点、内核跟踪点（tracepoint）、kprobe 以及网络相关钩子等位置，从而以较低侵入性捕获安全相关事件。相较于传统内核模块方案，eBPF 通过验证器与隔离执行机制降低了对内核稳定性的影响，并在性能与可观测性之间提供更可控的权衡，适合用于容器运行时的持续监测。

基于高语义的行为数据流，Falco 利用面向安全运维的领域特定语言（DSL）构建声明式规则引擎。与依赖统计特征或黑盒模型的检测方法不同，Falco 允许管理员针对特定威胁场景定义确定性的逻辑约束。例如图2-5展示了监测“在容器内启动交互式 shell 进程”的 Falco 规则，一旦检测到会产生日志/告

警。这种白盒化的检测方式不仅具备极强的可解释性，便于合规审计与取证分析，还能确保告警信息直接关联至具体的云原生实体，显著降低了安全运营的定位成本。然而，该机制在工程应用中仍存在局限性：一方面，基于规则的检测高度依赖专家经验与知识库的持续更新，对未知攻击或变种攻击的泛化检测能力较弱；另一方面，Falco 的观测视野局限于系统调用层，对应用层协议滥用或纯内存型攻击的感知能力有限。

```

1  - rule: Terminal Shell in Container
2    desc: Detects the spawning of a shell process inside a container.
3    condition: >
4      evt.type = execve and
5      container.id != host and
6      proc.name in (bash, sh, zsh)
7    output: "Notice: Shell spawned (user=%user.name container_id=%container.
8      id container_name=%container.name cmd=%proc.cmdline)"
9    priority: WARNING
10

```

图 2-5: Falco 运行时监控与规则示意

2.2.2 防测一体：Cilium Tetragon

在云原生安全工具谱系中，大多数工具侧重于单一维度的防护（如准入控制）或监控（如行为检测）。Cilium Tetragon[9] 通过基于 eBPF 的内核层安全观测与运行时强制执行，实现了**监控策略与防护策略**的深度融合，构建起从感知到响应的闭环防护体系。

Tetragon 由 Cilium 项目开发，作为 CNCF 云原生计算基金会的孵化项目，其核心设计理念是将安全策略的过滤、阻断与响应直接下沉至 eBPF 内核层执行，从而实现**实时性与低开销**的统一。与传统用户态 Agent 需要通过上下文切换将事件传递至用户空间不同，Tetragon 在内核态完成事件捕获、策略匹配与执行决策，显著降低了高频事件（如 send、read、write 系统调用）的观测开销。

从**监控策略**维度考量，Tetragon 能够检测并响应安全关键事件，包括：

- **进程执行事件**：捕获进程创建、特权变更与可执行文件调用链；
- **系统调用活动**：监控系统调用参数、返回值及其关联的内核状态；
- **I/O 活动**：涵盖网络通信与文件访问的细粒度追踪。

在 Kubernetes 环境中，Tetragon 具备原生的上下文感知能力，可将安全事件关联至命名空间、Pod、ServiceAccount 等 Kubernetes 身份，从而实现面向工作负载的精准检测与审计。

从**防护策略**维度考量，Tetragon 不仅局限于被动观测，更可在内核层面执行**实时强制**（Runtime Enforcement）：当检测到违反策略的行为时，Tetragon 可直接在系统调用完成前终止进程、阻断网络连接或拒绝文件访问，从而在攻击链早期阶段实施干预。例如，当应用程序试图提升特权（如通过 `setuid` 系统调用）时，Tetragon 可在系统调用返回前终止该进程，避免其执行后续的恶意操作。

这种**防测一体**的架构具有以下优势：

- **深度可见性**：通过在内核层 Hook 任意函数并过滤其参数、返回值及关联元数据，Tetragon 可获取用户态无法伪造或隐藏的真实内核状态；
- **灵活策略表达**：用户可通过编写 Tracing Policy 定义检测与响应逻辑，无需修改 Tetragon 核心引擎；
- **闭环响应**：将 Kubernetes 感知能力、内核状态与用户策略相结合，实现从检测到阻断的自动化闭环。

然而，Tetragon 的内核层强制执行也引入了新的工程挑战：策略配置不当可能导致误杀合法进程或影响业务可用性，因此在生产环境部署前需充分验证策略的副作用边界，并建立完善的灰度发布与回滚机制。

2.2.3 防护策略：准入控制与配置约束工具

准入控制（Admission Control）作为云原生安全的事前防护策略机制，通过在 API Server 请求链路中插入校验（Validation）或变更（Mutation）逻辑，在资源对象持久化前对其进行规则检测，从源头阻断不合规配置进入集群。在这一领域，OPA (Open Policy Agent) 及其 Kubernetes 实现 Gatekeeper 提供了通用的策略引擎支持，但其依赖 Rego 语言的特性在一定程度上提高了使用门槛。

相比之下，**Kyverno** 作为 Kubernetes 原生**防护策略引擎**，通过以 YAML 定义策略降低复杂度。Kyverno 不仅支持对 Pod 安全标准（PSS）、镜像签名及供应链安全的校验，还具备对资源进行自动修正（Mutate）与基于触发器生成配置（Generate）的能力，能够将安全基线与平台治理要求固化为可复用的声明式规则。如图 2-6 所示，该**防护策略**展示了一个典型的 Kyverno 应用场景：强制要求所有 Pod 必须携带 `owner` 标签。通过定义 `validationFailureAction: enforce`，任何缺失该元数据的 Pod 创建请求都将被准入控制器直接拒绝。这种声明式的策略管理方式不仅实现了明确的治理效果，更确保了集群内所有

工作负载具备可追溯的权属标识。在漏洞补丁尚不可用时，此类准入防护策

```
demo > ky.yaml
1  apiVersion: kyverno.io/v1
2  kind: ClusterPolicy
3  metadata:
4    name: audit-env-label
5  spec:
6    validationFailureAction: audit    # <--- 关键点: 这里是 audit (审计), 而不是 enforce (强制)
7    background: true                # 对集群里已经存在的 Pod 也进行扫描
8    rules:
9      - name: check-env-label
10     match:
11       resources:
12         kinds: [Pod]
13     validate:
14       message: "建议添加 env 标签以区分环境。"
15       pattern:
16         metadata:
17           labels:
18             env: "?*"                # "?" 意思是有这个 key 且 value 不为空
```

图 2-6: Kyverno 策略示例：强制要求 Pod 包含 owner 标签

略尤其适用于补丁窗口期策略化防控（Patch-gap Strategy-based Defense）场景：通过强制最小权限、限制高危特性（如 privileged 容器）与收敛敏感对象访问边界，可显著压缩攻击面。同时，准入防护策略的发布与回滚成本较低，便于安全团队在不同业务环境中进行灰度验证与影响评估，构建起灵活且坚固的配置安全防线。

2.2.4 防护策略：网络隔离工具——NetworkPolicy

网络隔离用于降低横向移动与数据外传风险，是云原生环境中与身份权限控制互补的关键手段。Kubernetes 的 NetworkPolicy 为声明式防护策略，允许以 Pod/Namespace 选择器与端口规则的形式，定义入站（Ingress）/出站（Egress）通信的允许集合，从而将网络访问从“默认全通”收敛到“默认拒绝 + 按需放通”。在多租户或微服务体系中，合理的 NetworkPolicy 可将控制域落到命名空间与工作负载级别，减少单点被攻破后对整个集群的扩散影响。

需要注意的是，NetworkPolicy 的效果依赖底层网络插件（CNI）对策略的实现能力。常见 CNI（如 Calico、Cilium、Antrea 等）均可提供对 NetworkPolicy 的支持；其中部分实现还可在标准三/四层策略之外，扩展更细粒度能力（例如基于 eBPF 的高性能转发与可观测性、面向应用协议的七层策略等）。工程实践中通常结合业务依赖关系梳理通信图谱，先对关键命名空间与高价值服务进行最小连通收敛，再逐步扩展覆盖范围，并配合观测与告警机制验证策略变更带来的连通性影响。

如图 2-7 所示，该策略定义了一个典型的微服务网络隔离场景：针对带有 role: db 标签的数据库工作负载，仅允许来自 role: frontend 前端服


```
YAML

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-allow-frontend-only
spec:
  # 对应描述: “针对带有 role: db 标签的数据库工作负载”
  podSelector:
    matchLabels:
      role: db

  # 声明策略类型为入站控制
  policyTypes:
    - Ingress

  ingress:
    # 对应描述: “仅允许来自 role: frontend 前端服务的流量”
    - from:
        - podSelector:
            matchLabels:
              role: frontend

    # 对应描述: “通过 TCP 5432 端口访问”
    ports:
      - protocol: TCP
        port: 5432
```

图 2-7: NetworkPolicy 示例：限制数据库仅允许前端服务访问

务的流量通过 TCP 5432 端口访问。这种基于标签选择器（Label Selector）的白名单机制有效地构建了微隔离（Micro-segmentation）边界，确保即使集群内其他容器（如被入侵的非关联服务）试图横向移动访问数据库，也会在网络层被 CNI 插件直接阻断，从而显著提升了核心数据资产的安全性。

2.2.5 静态扫描与配置审计工具：镜像、依赖与 IaC

与运行时检测相对，静态扫描更偏向事前发现与持续治理：在镜像构建、交付与部署前，通过自动化检查提前识别已知漏洞、弱配置与合规风险，避免高风险对象进入集群。

第一类是**镜像与依赖漏洞扫描**。该类工具通常解析镜像分层文件系统与软件包清单，基于公开漏洞库对已知 CVE 进行匹配，并输出风险等级、受影响版本与修复建议。代表性工具包括 Trivy[19]、Grype/Anchore[20] 等；其中，Syft[21] 等工具可生成软件物料清单（SBOM），用于提升漏洞告警的可追溯性与供应链治理能力。在工程实践中，镜像扫描往往与 CI 流水线结合，在构建阶段阻断高危漏洞或不合规组件进入制品仓库。

第二类是**Kubernetes 清单与基础设施即代码（IaC）扫描**。此类工具面向 YAML/Helm/Terraform 等配置，检查特权容器、宿主机路径挂载、过宽权限、明文密钥、缺失资源限制等常见风险，并可作为代码评审与准入策略的前置门

禁。例如, kubelinter[22]、kubescape[23]、kube-score[24]、kubesecc[25]、Polaris[26]等可对工作负载清单进行规则化检查; Checkov[27]等可对 Terraform 等 IaC 模板进行安全基线与合规性验证。静态扫描的优势在于易于规模化推广与持续运行,但也可能因业务差异产生误报,因此通常需要结合组织基线与例外机制、以及准入与运行时告警进行闭环。

2.3 国内外研究现状

随着云服务与云原生基础设施的广泛采用,围绕权限系统的安全问题逐渐受到更多关注。总体而言,相关研究可分为“系统权限安全问题”“Kubernetes 安全问题”“漏洞缓解与防控”以及“本文工作”四个层次。

2.3.1 系统权限安全问题

围绕系统权限治理,既有研究形成了从设计原则、访问控制模型到数据驱动推断与风险后果分析的较为完整的谱系。Saltzer 与 Schroeder[28]归纳的信息保护原则为权限设计提供了规范性基准,其中“最小权限原则”强调主体应仅获得完成任务所需的最小权限集合,以减少可被滥用的能力边界。面向工程落地, Ferraiolo 等 [4] 提出并系统阐释 RBAC 的基本动机与结构,将权限组织为角色并通过用户—角色映射实现规模化授权,为后续策略表达、角色工程与权限管理研究提供了通用模型。

在上述框架之上,“过度/冗余授权”被普遍视为长期且结构性的风险来源。O'Shea[29]从冗余访问权角度讨论了不必要授权的普遍性与危害,指出超出业务所需的权限会显著扩大攻击面并提高误用概率。针对“应当授予哪些权限”这一核心问题, Molloy 等 [30] 提出面向日志的生成式角色/策略挖掘方法,在 RBAC 角色挖掘中显式结合真实使用证据 (usage) 与用户属性 (attributes),以学习与使用行为具有因果关联且可解释的角色模型;该类模型在配置对齐、分配错误发现与异常行为识别等任务上具有直接应用价值,并通过多组真实数据集对覆盖率、稳定性与泛化性等指标进行了评估。

进一步地,在权限集合高度异构的云服务场景中, Sanders 与 Yue[31]将最小权限构建表述为安全风险与可用性成本之间的量化折中,提出 Privilege Error Minimization Problem (PEMP) 及相应的策略评分方法,以同时度量过度授权风险与修正欠授权的代价,并在包含 520 万条 AWS 审计日志的数据集上比较了朴素、无监督与有监督三类策略生成算法。

除传统访问控制研究外,其他生态的权限规范抽取与权限滥用检测为“权限一行为一致性”提供了可迁移的方法与经验证据。Au 等 [32] 提出 PScout, 通过静态分析 Android 源码自动抽取权限规范, 以应对大规模代码分析、跨进程 IPC 权限检查建模与多样化检查机制统一抽象等挑战; 该研究指出, 即便在接口使用受到约束的条件下, 应用仍可能申请到与其实际行为不匹配的权限。面向小程序生态, Wang 等 [33] 围绕权限申请行为提出五类滥用模式, 并在基准集与真实数据上验证其可用性, 揭示机制缺陷与不当实践叠加下的权限滥用风险。

在风险后果与治理闭环层面, Papadimitriou 与 Garcia-Molina[34] 从数据泄露检测问题出发, 讨论泄露行为的识别与追溯机制, 为“授权边界被突破后, 敏感数据如何传播与外泄”的分析提供了问题定义与思路。信息流跟踪与污点分析构成了技术层面的支撑: Tripp 等 [35] 面向 Web 应用开展污点分析以追踪敏感数据传播路径; Enck 等 [36] 在智能手机上实现运行期信息流跟踪用于隐私监测; You 等 [37] 进一步在 Android 运行时 (ART) 上给出兼容的动态污点分析框架, 强调在无需修改系统与无需 root 的条件下提升对隐私数据传播的可观测性。由此可见, 权限治理不仅依赖静态的授权边界刻画, 也需要结合运行期使用证据与数据流证据, 以支撑风险识别、定位与缓解。

2.3.2 Kubernetes 安全问题

Kubernetes 场景下的权限与安全风险往往因配置项繁多、组件生态复杂以及运行时动态性而被放大, 其中误配置与最佳实践未落地是最常见的风险来源之一。Islam Shamim 等 [38] 对 Kubernetes 安全实践开展系统化梳理, 总结社区与工业界反复出现的关键关注点, 为理解配置风险与治理要点提供了结构化框架。在此基础上, Shamim[39] 进一步从攻击者视角讨论部署清单层面的最佳实践违规如何演化为可利用路径, 并以拒绝服务等场景说明持续审计与合规治理的现实必要性。从实践侧的开发者讨论数据亦可获得补充证据: Curtis 与 Eisty[40] 对 Stack Overflow 中 Kubernetes 相关帖子开展实证分析, 发现安全议题具有较高关注度且其影响力随时间上升, 反映出生产环境中安全治理的长期性与普遍性。

围绕误配置的自动化治理, Ul Haque 等 [41] 提出 KGSecConfig, 通过知识图谱对多厂商、多平台的安全配置空间进行统一建模, 并结合关键词与学习模型自动抽取安全配置选项与概念, 在 Kubernetes、Docker、Azure 与 VMWare

等环境构建配置知识图谱。该研究进一步给出了在 Kubernetes 集群中开展自动化误配置缓解的用例，说明该思路在工程落地中的可行性。

围绕 Kubernetes 部署清单的误配置检测，已有工作通过开源数据与工业部署数据开展了系统性的实证分析。Rahman 等 [7] 从开源仓库出发，挖掘 92 个仓库中的 2,039 份 Kubernetes 部署清单，识别并归纳 11 类安全误配置，共发现 1,051 处误配置实例，并构建静态分析工具 SLI-KUBE 以量化其出现频率。该研究同时报告了从业者对所反馈问题的修复意愿与采纳情况。Shamim 等 [6] 以 CIS 推荐的 53 项配置为基准，结合从业者问卷与开源及企业配置文件评估推荐项的遵守程度与静态检测工具的覆盖能力。研究指出，从业者对部分推荐项存在认知差异，企业与开源配置文件中均可观察到较为普遍的违规现象。在对 5 款 SAST 工具的评测中，工具间在配置类别覆盖与告警一致性方面存在明显差异。作为对静态评测的补充，Shamim 等 [14] 进一步在工业部署上开展 DAST 经验研究，讨论在真实运行环境中对配置基线进行动态核查的必要性。该类方法通过结合集群运行时状态与实际可达性，对仅依赖部署清单静态规则难以覆盖的违规情形进行补充，并为风险确认、处置优先级排序与修复验证提供更直接的证据。

除原生部署清单外，Helm Charts 等模板化部署工件同样会引入误配置与供应链风险，并使安全评估对象由单份配置文件扩展至参数化生成的资源集合。Zerouali 等 [42] 对 Artifact Hub 上 9,482 个 Helm Charts 开展挖掘与实证分析，刻画其演化特征，并从容器镜像过期与漏洞暴露角度揭示潜在安全风险。Blaise 与 Rebecchi[43] 以 Helm Charts 为输入抽取拓扑图并量化节点与依赖关系的安全特征，进而评估风险并提取高风险攻击路径，为部署工件与风险路径之间的系统化分析提供方法支撑。Minna 等 [44] 提出面向 Artifact Hub 的 chart 挖掘与分析流水线，比较 Checkov、KICS 等工具在误配置报告上的一致性与差异，并研究使用大语言模型生成修复建议与重写模板。研究指出，不同模型给出的修复建议质量差异较大，且仅提供局部 YAML 片段作为输入可能不足以支撑稳健修复。上述研究表明，配置治理除检测准确性外，还需面向工程可用性与回归验证设计可控的修复与防控流程。

除对配置项逐条核查外，亦有研究从安全异味角度刻画 Kubernetes 部署中更具结构性的风险模式。Dell' Imagine 等 [45] 面向 Kubernetes 部署的微服务应用，提出相应的分析思路并实现原型工具 KubeHound，并在受控环境与第三方应用样本上进行验证，为将最佳实践抽象为可复用的风险模式识别

提供补充证据。

除误配置外，过度权限及其链式利用构成 Kubernetes 权限风险研究的另一条主线。Kubernetes 的 RBAC 机制与第三方组件生态使高权限配置更易出现，并可能成为跨层攻击的关键支点。Yang 等 [5] 从控制平面第三方应用入手，系统分析其授权情况并指出过度关键权限的普遍存在。攻击者可利用相关权限从工作节点逃逸并进一步接管集群。该研究在 CNCF 的 153 个第三方应用中识别出 51 个存在潜在风险，并报告在云厂商服务中同样可观察到类似风险，相关披露获得 8 个新 CVE。Gu 等 [46] 提出以已攻陷单个 Pod 为起点的威胁模型，并据此提出 EPScan，通过面向 Pod 的程序分析提取资源访问行为，将行为所需权限与清单请求权限对比以报告可被滥用的过度权限。该方法在 108 个 CNCF 应用上发现 50 个应用的 106 个 Pod 存在此前未知的可利用过度权限，报告精确率达 94.6%，并获得 9 个 CVE。

除配置与授权外，Kubernetes 的网络抽象与 CNI 实现亦会影响攻击面刻画与防护手段的有效性。Minna 等 [12] 指出 Kubernetes 网络抽象可能带来与传统网络安全心智模型不一致的攻击面认知，因而需要从平台语义出发重新理解可达性与隔离边界。Budigiri 等 [13] 围绕 NetworkPolicy 的性能开销与安全性开展评估，比较基于 eBPF 的实现并讨论网络隔离策略在低开销场景下的适用性。Kim 等 [47] 进一步从 CNI 插件架构差异出发，对多类网络策略处理能力与性能影响进行系统比较，揭示 L3/4 与 L7 策略、策略复杂度与并发规模之间的权衡关系。该类研究为本文在补丁窗口期采用网络隔离与访问控制等防护手段提供背景依据。

在缓解与防护方面，研究工作一方面尝试提升授权约束粒度，另一方面将分析视角扩展至流水线与运维链路。Pecka 等 [15] 从 DevOps 流水线视角构建并分析典型 Kubernetes 环境威胁，提出滥用部署权限、篡改构建产物、暴露内部 IP 以及从 CRUD 权限账号提升至 root 等攻击场景，强调仅部署 DevSecOps 工具并不足以替代对流水线与集群治理的整体性安全设计。Cesarano 与 Natella[48] 指出仅依赖 RBAC 难以对 API 请求的规格属性进行足够细粒度约束。为此，该研究提出 KubeFence，通过分析可信仓库中的 Operator 及其配置，为特定客户端工作负载生成更细粒度的 API 过滤策略，以限制不必要的 API 功能并收敛攻击面。

在节点侧强制执行与运行时约束方面，Zhu 与 Gehrmann[49] 提出 KubSec，通过分布式采集工作负载行为并集中生成 AppArmor 配置文件，实现对

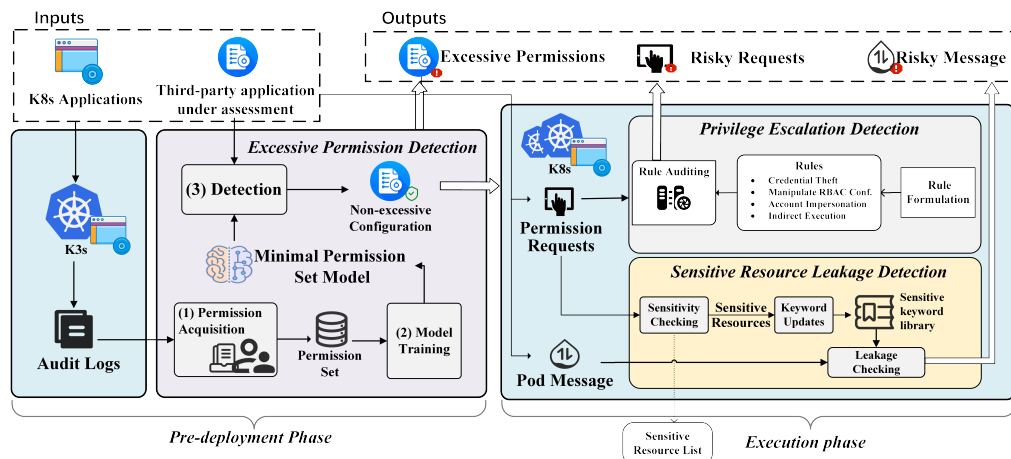


图 2-8: KubeGuard 方法框架图 [17]

应用容器的细粒度系统调用与资源访问约束，并尝试在不显著干扰服务的前提下执行策略。该方向与本文关注的漏洞利用特征驱动的可执行防控策略之间具有互补关系。前者强调从行为数据自动生成主机侧约束，后者强调在补丁窗口期基于可利用性证据生成面向工程落地的策略化防控方案。

从降低节点侧内核暴露面的角度，Le 等 [50] 提出面向系统调用暴露度量的调度方法，通过将工作负载调度至对高风险系统调用暴露更少的节点，以限制邻近工作负载攻击对集群的影响范围。该研究表明，在补丁窗口期除策略约束外，还可结合调度与资源编排层面的机制进一步收敛攻击面。

在工程工具层面，KubiScan[51]、Krane[52]、RBAC Police[53] 等为权限与资源审计提供了可用手段，Kubescape[23]、Trivy[19]、kube-score[24] 等常用于静态扫描与基线核查。为便于工程落地与能力对照，表2-1对 Kubernetes 常见安全扫描/审计工具进行分类整理。现有研究与工具共同表明：将攻击模型、检测方法与硬化策略转化为可快速部署的工程方案，仍需在工具集成、误报控制与业务可用性之间进行系统权衡。

2.3.3 本课题组研究基础

在上述研究基础上，课题组工作提出了面向权限风险的系统化检测框架 KubeGuard，覆盖过度权限、权限提升与敏感信息泄露三类风险，并分别提出最小权限集构建、规则化审计与敏感词监测等机制 [17]。如图2-8所示，KubeGuard 框架由三个核心模块组成：过度权限检测、权限提升检测与敏感信息泄露检测。

该工作一方面，基于 NLP 技术识别应用的最小权限集来判定权限配置中是否存在过度授权；另一方面，采用动态的规则化审计机制检测多类权限提升

表 2-1: Kubernetes 常见安全扫描与审计工具分类整理

类别	代表性工具（示例）	主要能力与适用场景
镜像/依赖漏洞扫描	Trivy、Grype、Clair、Snyk（商业）	解析镜像/依赖清单并匹配 CVE；常作为 CI 门禁或制品仓库准入检查，输出漏洞等级、受影响版本与修复建议。
SBOM 生成与供应链	Syft、CycloneDX、cosign	生成 SBOM（如 SPDX/CycloneDX）并对制品签名与验证；用于提升漏洞告警可追溯性与供应链完整性校验。
清单/Helm 静态扫描	kubescape、kube-score、kube-linter、Polaris、kubesec、datree	针对 Kubernetes YAML/Helm 进行规则化检查（特权容器、hostPath、资源限制、明文密钥、PSS 违规等），适合代码评审与部署前基线核查。
IaC 扫描（Terraform 等）	Checkov、KICS、tfsec	面向 IaC 模板（Terraform/K8s YAML/Dockerfile 等）做合规与弱配置扫描；常用于 DevSecOps 流水线的“左移”治理。
RBAC/权限审计	KubiScan、Krane、RBAC Police、kubeadit、rakkess、kubect1-who-can	盘点集群内角色/绑定/服务账户权限，识别过宽授权与高危动词；支持“谁能做什么”查询与权限差分核查。
基线核查与 CIS 审计	kube-bench、Kubescape、OpenSCAP（可选）	基于 CIS Benchmark 等标准对控制平面/节点配置开展核查，输出不合规项与修复建议，适用于周期性审计与合规检查。

风险。如表2-2所示,该框架针对四类权限提升场景定义了相应的检测规则:凭证窃取 (Credential Theft, CT)、账户冒用 (Account Impersonation, AI)、RBAC 操纵 (RBAC Manipulation, RM) 以及间接执行 (Indirect Execution, IE)。同时,通过监测 Pod 间消息传输并进行关键词匹配,对涉及敏感资源的内容触发告警,从而降低敏感信息泄露风险。

表 2-2: 多种权限提升类型及其检测规则

类型	机制	检测规则
凭证窃取	- 密钥访问 - Pod 执行 - 容器逃逸	检测请求的源 IP 地址或服务账户与资源所属 Pod 不一致的 API 请求
账户冒用	- 服务账户模拟	检测服务账户、操作和资源的组合是否属于该账户的正常使用行为集合
RBAC 操纵	- 角色修改 - 角色绑定修改	检测对角色或角色绑定资源的创建、修补或更新操作
间接执行	- 工作负载修改 - Pod 执行	检测对工作负载资源的创建、修补或更新操作,以及对 Pod 的 exec 操作

进一步地,课题组实现了原型系统 KubeGuarder,并在开源 Kubernetes 应用与模拟场景上开展实验评估;结果表明,该框架在过度权限检测方面相较静态基线方法更为有效且更精确,并能够同时发现多类型的权限提升与敏感资源泄露风险 [17]。

然而,该研究主要基于权限层面的规则化检测,通过审计 RBAC 权限配置与 API 访问行为来识别潜在的权限提升风险。这种方法虽然能够系统性地覆盖多种提权模式,但在真实生产环境中可能面临较高的误报率:一方面,仅依赖权限配置与 API 操作的匹配难以区分正常运维行为与恶意提权行为;另一方面,该方法未能深入到节点侧系统层面的权限提升漏洞,对于利用容器逃逸、SUID 配置不当、内核漏洞等底层机制实现的提权行为缺乏针对性检测能力。此外,基于规则的检测依赖于预定义的攻击模式,对于未知漏洞或新型攻

击手法的泛化能力有限。

相较而言,本文工作聚焦于**真实权限提升漏洞场景**的深度分析与针对性防控。具体而言,本研究不仅关注 RBAC 层面的权限配置问题,更深入到**节点侧系统权限提升漏洞**的利用特征与防控机制,通过对公开 CVE 漏洞与真实攻击案例的系统化分析,提炼出**可利用性判定准则与防控策略生成方法**。同时,本研究面向补丁未就绪或难以及时升级的生产环境,通过结合准入控制、运行时监控与网络隔离等多层防御手段,在漏洞利用链的关键环节实施精准阻断,从而在保障业务可用性的前提下有效降低安全风险暴露面。

2.4 小节

通过本章的分析,可知云原生环境中 Kubernetes 权限提升漏洞的成因复杂,涉及容器安全、Kubernetes 权限模型及其配置等多个方面。现有的安全工具与**监控策略、防护策略**在一定程度上能够提供防护,但仍需面向权限提升的特性进行优化与增强。本文接下来的工作将基于上述研究现状与动机,聚焦于权限提升漏洞的检测与防控策略的设计与实现。

3 面向云原生权限提升漏洞的智能防控技术

本章给出本文方法体系和技术细节，包括形式化定义、总体流程和三个主要技术点。

3.1 形式化定义

3.1.1 基本对象

本节给出框架的形式化定义，刻画漏洞输入、策略生成与质量评价之间的关系。

定义 1（漏洞实例）：漏洞实例 $v \in \mathcal{V}$ 表示为三元组：

$$v = \langle \text{id}, d, m \rangle$$

其中 id 为 CVE 编号， d 为漏洞描述文本， m 为元数据（严重性、受影响组件等）。

定义 2（安全上下文）：上下文 $c \in \mathcal{C}$ 是与漏洞相关的证据集合，每条证据附带来源标识：

$$c = \{(t_i, \text{src}_i)\}_{i=1}^n$$

其中 t_i 为文本内容， src_i 为来源（文档路径、链接等）。

定义 3（安全策略）：策略空间 Π 由监控策略 Π_{mon} 与防护策略 Π_{prot} 组成：

$$\Pi = \Pi_{\text{mon}} \cup \Pi_{\text{prot}}, \quad \Pi_{\text{mon}} \cap \Pi_{\text{prot}} = \emptyset$$

策略 $\pi \in \Pi$ 由三部分组成：

$$\pi = \langle \text{layer}, \text{method}, \text{action} \rangle$$

其中：

- **layer:** 防控层级（身份与权限、网络隔离、运行时检测、审计日志、策略执行）
- **method** = $[m_1, \dots, m_n]$: 防控方法序列， m_i 包含类型标识（监控/防护）及机理说明
- **action** = $[a_1, \dots, a_n]$: 可执行操作序列，与 **method** 一一对应（定义 3.1）

定义 3.1（防控措施单元）：策略 π 中的 **method** 与 **action** 按下标一一对

应：

$$\text{method} = [m_1, \dots, m_n], \quad \text{action} = [a_1, \dots, a_n]$$

每对 (m_i, a_i) 构成一个防控措施单元， m_i 标识类型（监控/防护）并说明机理， a_i 给出可执行配置或命令：

- **监控措施：** m_i 描述检测方法（如“进程行为监控”、“异常访问检测”）， a_i 包含监控规则（Falco、Tetragon）、告警配置、日志关联规则等
- **防护措施：** m_i 描述阻断方法（如“准入约束”、“权限收敛”、“网络隔离”）， a_i 包含准入策略（Kyverno、OPA）、RBAC 配置、NetworkPolicy、运行时策略（Seccomp、AppArmor）、回滚方案等

该设计支持在同一策略中组合监控与防护措施，形成检测-响应闭环。

3.1.2 评价与选择

本节采用基于排名的评价机制。相比绝对评分，排名以相对次序刻画优劣，避免跨样本的尺度不一致问题。

定义 4（三维质量向量）：策略 $\pi \in \Pi$ 的质量由三维向量表示：

$$q(\pi) = (E(\pi), I(\pi), D(\pi)) \in [0, 1]^3$$

其中 E 为有效性（覆盖攻击面与关键路径）， I 为无干扰性（低误报、低业务影响）， D 为可部署性（步骤清晰、可操作）。

定义 5（维度排名）：对候选集合 Π_v ($n = |\Pi_v|$)，在维度 $\delta \in \{E, I, D\}$ 上定义排名算子：

$$R_\delta(\Pi_v) \rightarrow (\pi_{(1)}, \pi_{(2)}, \dots, \pi_{(n)})$$

其中 $\pi_{(1)}$ 为该维度最优候选。要求严格排序（无并列）。引入秩函数 $r_\delta : \Pi_v \rightarrow \{1, \dots, n\}$ 表示排名位置。

定义 6（维度得分）：将排名映射为归一化得分：

$$s_\delta(\pi) = \frac{n - r_\delta(\pi)}{n - 1}$$

满足 $r_\delta(\pi) = 1$ 时 $s_\delta(\pi) = 1$ （最优）， $r_\delta(\pi) = n$ 时 $s_\delta(\pi) = 0$ （最差）。

3.2 开源应用漏洞安全知识库构建

3.2.1 漏洞收集与分类框架

本节阐述漏洞样本的收集流程与两级成因分类框架的构建方法。该分类框架面向策略生成场景：在输入侧提供相对稳定且可解释的风险语义线索，用以约束模型的推理路径，并辅助其对控制域与关键控制点进行选择，从而提升输出策略的可部署性、可验证性与副作用可控性。

(1) 云原生应用清单与仓库映射。以 Kubernetes 及其生态开源应用为研究对象，首先从 CNCF Landscape 获取云原生应用清单，并对清单进行快照固化以保证可复现性。随后将条目名称映射到其官方代码仓库，形成后续漏洞关联与证据收集的目标集合。

(2) 漏洞收集与筛选。在数据源选择上，综合使用 NIST NVD¹、MITRE CVE 官方库²与安全公告平台 GitHub Security Advisory³，检索 Kubernetes 及其生态应用中与权限提升相关的高风险漏洞。具体收集策略包括：

- **关键词约束：**在 CVE 描述与参考链接中匹配 Privilege Escalation、Authorization Bypass、Improper Access Control、RBAC、ServiceAccount、ClusterRole、Container Escape/Sandbox Escape/Breakout 等关键词；
- **严重性阈值：**采用 CVSS v3.x 评分，并以 ≥ 5.0 的高危与严重漏洞为主要对象；
- **人工复核：**对候选条目结合公告原文、厂商通告与社区讨论进行复核，剔除重复或与权限提升无关的记录，并补全攻击前提、影响面与可用缓解信息。

最终保留的结构化字段包括 CVE 编号、漏洞摘要、披露时间、受影响组件与版本范围、修复版本、CVSS 评分、参考链接，以及与修复相关的补丁提交记录等。

(3) CWE 标注与分布统计。为刻画漏洞成因层面的共性，本文对样本进行 CWE (Common Weakness Enumeration) 标注。CWE 是 MITRE 维护的软件弱点枚举体系，可用于将具体 CVE 归并到更抽象的弱点类型。本文依据公开映射与公告信息提取每个 CVE 的 CWE 标签，并统计不同 CWE 的出现频次。

¹NIST NVD (National Vulnerability Database): <https://nvd.nist.gov/>

²MITRE CVE: <https://cve.mitre.org/>

³GitHub Security Advisory: <https://github.com/advisories>

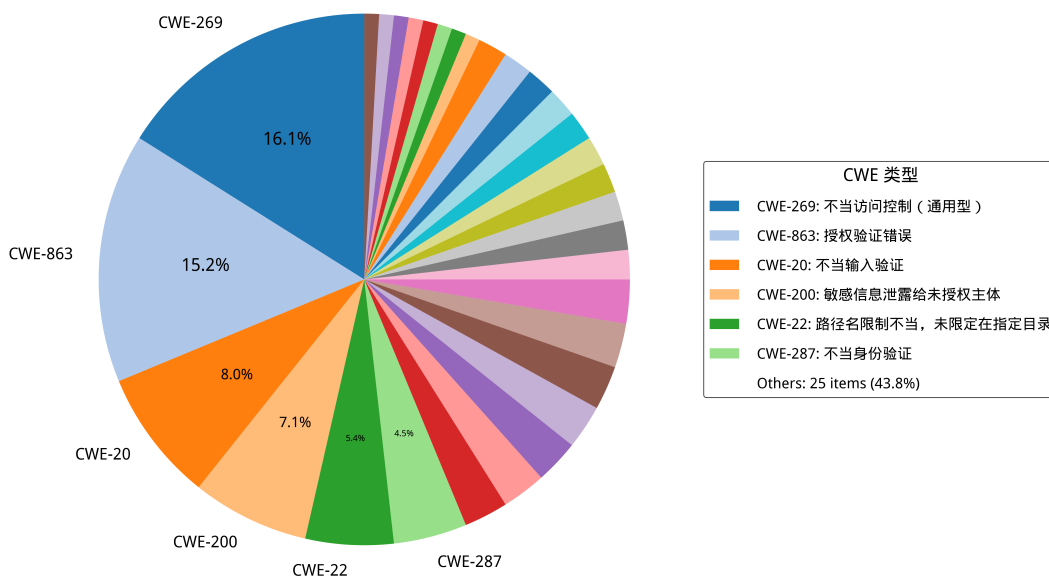


图 3-1: Kubernetes 权限提升 CVE 对应的 CWE 类型占比

CWE 体系在组织形态上并非严格正交的分类结构：同一漏洞可能对应多个弱点条目，且不同条目在抽象层次上跨越具体机制与高层概念，使得类别之间存在交叉与重叠。在本文收集的 55 个权限提升漏洞中，36% 的 CVE（20 个样本）同时关联两个及以上 CWE 标签。为客观反映 CWE 分布，统计时对每个 CVE 赋予单位权重 1.0，多标签情况下各 CWE 均分该权重。统计结果显示，访问控制类弱点（如 CWE-863 授权不当、CWE-269 权限管理不当）与输入处理类弱点（如 CWE-20 输入验证不当）在多个样本中共现，反映出漏洞成因的复合特征。上述非正交特性导致单一 CWE 标签难以为防控策略选择提供唯一且稳定的语义约束，因此需构建面向防控动作的成因分类框架。

（4）面向防控的成因分类框架。为解决“根因标签不稳定/不互斥”对策略生成的影响，本文提出面向云原生权限提升漏洞的两级成因分类框架，并将每个漏洞映射为（一级类，二级类）标签：

- **一级分类**刻画成因大类（如安全逻辑缺陷、配置错误、代码注入、敏感信息泄露等），用于提供高层风险语义与控制域提示；
- **二级分类**进一步细化为更贴近防控动作选择的类型（例如授权验证不完善、默认不安全配置等），用于将漏洞分析结果对齐到可操作的控制点（RBAC 最小权限、准入控制、网络隔离、运行时策略与审计等）。

在策略生成阶段，分类结果作为提示词中的结构化约束，有助于模型将“漏

洞语义 → 控制域 → 可执行动作序列”对齐，减少泛化建议与无关信息，提高策略的可部署性与针对性。完整的一级/二级分类与 CWE 对应关系见附录表 B-1。

表 3-1: 云原生权限提升漏洞分类框架（CWE 对应详见附录表 B-1）

一级分类	二级分类	示例 CVE
配置缺陷	冗余权限	CVE-2023-30512
	网络隔离不足	CVE-2020-4062
安全逻辑缺陷	访问控制缺陷	CVE-2022-24768
	身份验证绕过	CVE-2023-22482
	输入验证不足	CVE-2023-3955
	授权验证不完善	CVE-2022-21701
敏感信息泄露	API 端口暴漏	CVE-2022-1902
	日志泄露	CVE-2019-10165
	配置文件泄露	CVE-2019-3869
代码注入	命令注入	CVE-2021-41254
	路径遍历	CVE-2022-24877

具体分类定义如下：

1. 配置缺陷：应用或基础设施的配置不当导致的安全漏洞，包括：

- 过度权限配置：Kubernetes RBAC 配置不当、Role/ClusterRole 规则范围过宽或绑定关系不合理等，使低权限主体获得超出其职责的资源访问与高危操作能力。该类问题在第三方应用接入场景中尤为常见，并已被证明可被用于接管集群关键资源 [5, 29]。
- 网络隔离不足：容器、Pod 或服务间缺乏网络隔离，导致攻击者可以横向移动获取其他权限上下文。

2. 安全逻辑缺陷：应用代码中的安全机制设计或实现不当，包括：

- 访问控制缺陷：应用的权限检查不完全或存在逻辑漏洞，允许绕过权限验证；
- 身份验证绕过：认证机制设计缺陷，攻击者可以冒充他人身份或跳过认证步骤；
- 授权验证不完善：权限检查在某些操作路径或边界条件下缺失。

3. 敏感信息泄露：重要的身份、密钥或权限信息意外暴露，包括：

- API 端口暴露：管理接口、内部 API 或调试端口未受保护暴露在网络上；
- 日志泄露：日志文件包含敏感信息（如 token、密钥、命令历史）；

- 配置文件泄露：配置文件包含硬编码的凭证或密钥。
- 4. 代码注入：应用对用户输入的处理不当，允许注入恶意代码，包括：
 - 命令注入：应用直接执行用户提供或可控的命令，攻击者可以执行任意系统命令；
 - 路径遍历：应用文件访问检查不完整，攻击者可以访问系统其他部分的文件。

3.2.2 知识库构建

知识库构建阶段的目标是将与漏洞 v 相关的多源材料组织为带来源标注的证据集合 $c = \{(t_i, \text{src}_i)\}_{i=1}^n$ ，并以嵌入式模型构建一个可检索的向量索引，以支持后续“按需取证”的上下文供给。

(1) **材料收集与最小化预处理**。对每个漏洞关联的开源应用仓库，收集代码、配置与文档等可读文本；同时纳入与漏洞相关的安全公告、Issue/PR 讨论与修复提交信息。对文本进行最小化规范化（统一换行与空白），并过滤超长或二进制内容，以降低噪声与资源开销。

(2) **结构化分块**。为兼顾证据粒度与可复核性，本文对不同材料采用差异化分块：对代码/配置按行切分并尽量对齐函数或语义边界；对文档按段落与窗口切分并保留适当重叠，以减少跨块语义断裂。

(3) **向量化与索引**。对每个文本块使用嵌入式模型生成向量表示。本文选用 `sentence-transformers/all-mpnet-base-v2` 作为编码器，将文本块映射到统一的向量空间。基于向量表示，采用 FAISS 构建近邻索引以支持相似度检索，从而为后续按需检索证据提供高效的底层能力。为保证可追溯与可复现，索引与元数据进行同步持久化，内容包括向量索引文件、块级元数据、以及构建配置。块级元数据记录来源路径或链接、块 ID、块类型与范围等信息；构建配置记录分块参数与检索阈值等关键设置。

上述流程得到的向量知识库为后续策略生成提供了“可追溯证据”的供给能力：模型在推导控制措施时可以检索到与漏洞相关的关键事实与上下文片段，并通过来源标识完成快速定位与复核。

3.3 基于多智能体协同的策略生成与优化

策略生成阶段以漏洞实例 v 与安全上下文 c 为输入，生成候选策略集合 $\Pi_v = \Pi_{\text{mon}} \cup \Pi_{\text{prot}}$ （图3-2），其中 Π_{mon} 为监控策略子集， Π_{prot} 为防护策略子

集。流程由四类智能体分工完成：安全知识智能体整理证据；漏洞分析智能体给出结构化分析；策略生成智能体产出候选（包括监控策略与防护策略）；策略评审与优化智能体基于评审反馈修订，以降低误报与运维噪声并提升可部署性。

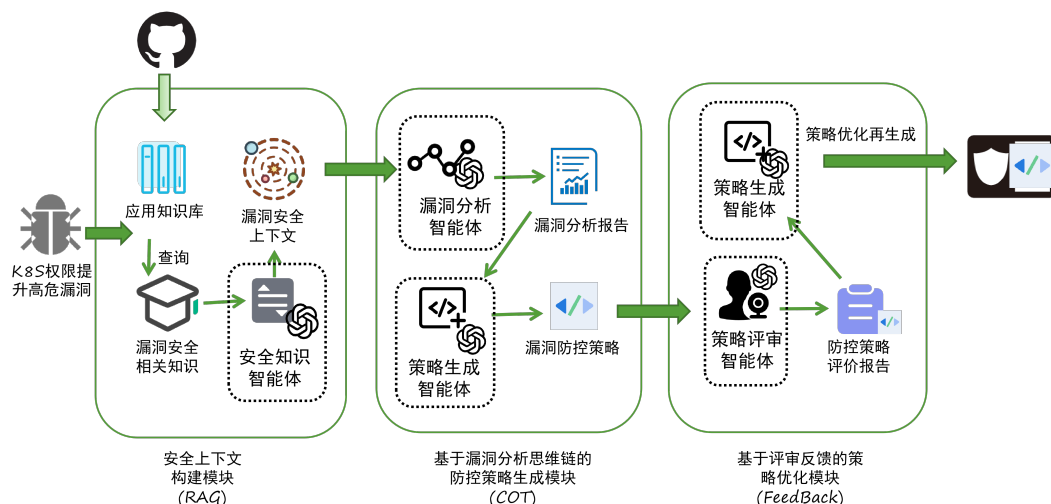


图 3-2: 多智能体协同的安全策略生成框架

3.3.1 安全上下文构建

安全上下文构建以“证据可追溯”为目标,形成证据集合 $c = \{(t_i, \text{src}_i)\}_{i=1}^n$ 。知识库阶段为每个片段记录来源元数据（路径/URL/版本/位置）以便回溯；生成阶段从知识库与安全材料检索候选证据，并由安全知识智能体去重与提炼，输出触发条件、受影响组件、权限边界与可控点，同时保留对应的 src_i 供复核。

安全知识智能体 PROMPT 模板

你是云原生安全专家，负责生成符合结构规范的安全策略 JSON 数据，
 ↳ 熟悉 Kubernetes 漏洞与缓解措施。

输入数据

<VULN_SUMMARY>

任务目标

基于提供的漏洞信息和相关知识，生成一个完整的JSON对象，
 ↳ 严格遵循下面给出的结构规范。

工具提示

可选用 OPA、Kyverno、Falco、Tetragon、NetworkPolicy 等工具，

- ↪ 并在 mitigation 字段写明工具与作用。其中 Tetragon
- ↪ 可同时实现监控策略（进程/系统调用/I/O检测）与防护策略
- ↪ （内核层阻断）。

输出规则

1. 仅返回 JSON：只输出单个 JSON 对象，不要附加说明、Markdown、
 - ↪ 表格或其它文本。
2. 严格格式：字段、层级、顺序必须与上方给出的结构规范/示例一致，
 - ↪ 所有字符串使用双引号并保持合法 JSON。
3. 信息缺失：无法确定时写 "(info needed)"，并在字段或
 - ↪ assumptions 中说明假设。
4. 配置转义：YAML/CLI 片段的换行统一写成 `\\n`。
5. 具象缓解：禁止仅写“升级补丁”，
 - ↪ 需给出可执行的控制措施与配置步骤（如适用）。
6. 一致性：内容必须与输入/分析/评审结论一致，禁止引入无依据内容。
 - ↪

3.3.2 基于漏洞分析思维链的策略生成

该步骤在结构化约束下完成“漏洞分析—技术选型—策略生成”，并输出关键中间结论，便于复核。与仅输出自然语言建议不同，本文统一约束中间产物与最终策略为固定的 JSON 结构，以减少信息缺失与表述歧义。

首先，漏洞分析智能体基于漏洞摘要与安全上下文 c 输出结构化分析结果，覆盖漏洞类型、攻击向量、影响范围与风险等级，并给出与本文分类体系一致的风险类别。随后，策略生成智能体在提示词模板与思维链（CoT）引导下，以分析结论为约束完成控制手段选择与动作序列生成：结合平台可用能力在身份与权限、网络隔离、运行时安全与策略执行等控制域中确定优先级与实施路径，并将其映射为最终策略 JSON；当信息不足以确定时，以 (info needed) 显式标注缺失信息并给出必要假设。

漏洞分析智能体思维链 PROMPT 模板

你是 Kubernetes 安全分析师，请对以下 CVE 进行分析并分类。

输入信息

<VULN_SUMMARY>

相关知识上下文

<RAG_CONTEXT>

分析要求

请从以下维度分析 CVE:

1. 漏洞类型：详细说明这是什么类型的漏洞
2. 攻击向量：漏洞如何被利用
3. 影响范围：漏洞可能造成的影响
4. 风险等级：基于 CVSS 评分和实际影响评估
5. 漏洞分类：从以下分类中选择最合适的一个：
 - 内部权限失控
 - 过度权限配置
 - 代码注入
 - 敏感信息泄露
 - 缺乏网络隔离
 - Kubernetes 漏洞
 - 容器运行时漏洞

输出格式

请返回 JSON 格式的分析结果：

```
{
  "vulnerability_type": " 漏洞类型描述",
  "attack_vector": " 攻击向量描述",
  "impact_scope": " 影响范围描述",
  "risk_level": " 高/中/低",
  "category": " 选择上述分类之一",
  "key_findings": " 关键发现总结"
```

```

}
## 输出规则
1. 仅返回 JSON: 只输出单个 JSON 对象, 不要附加说明、Markdown、
  ↳ 表格或其它文本。
2. 严格格式: 字段、层级、顺序必须与上方给出的结构规范/示例一致,
  ↳ 所有字符串使用双引号并保持合法 JSON。
3. 信息缺失: 无法确定时写 "(info needed)", 并在字段或
  ↳ assumptions 中说明假设。
4. 配置转义: YAML/CLI 片段的换行统一写成 `\\n`。
5. 具象缓解: 禁止仅写“升级补丁”,
  ↳ 需给出可执行的控制措施与配置步骤(如适用)。
6. 一致性: 内容必须与输入/分析/评审结论一致, 禁止引入无依据内容。
  ↳

```

策略生成智能体 PROMPT 模板

基于 CVE 分析和防控技术选型, 生成完整的安全策略。

CVE 分析

<ANALYSIS_RESULT>

防控技术选型

<MITIGATION_ANALYSIS>

相关知识上下文

<RAG_CONTEXT>

策略生成要求

你必须返回一个有效的 JSON 对象, 格式如下:

```

{
  "cve_id": "string - CVE 编号",

```

```

"description": "string - 中文漏洞简介,
    ↳ 聚焦攻击面和触发条件",
"application": "string - 受影响的业务组件和运行环境假设",
"threat_analysis": {
    "preconditions": "string - 漏洞触发的前提条件",
    "attack_path": "string - 攻击链描述",
    "blast_radius": "string - 潜在影响范围"
},
"risk_category": "string - 必须是以下之一:
    ↳ 内部权限失控|过度权限配置|代码注入|敏感信息泄露|缺乏网
    ↳ 络隔离|Kubernetes 漏洞|容器运行时漏洞",
"mitigation": {
    "layer": "string - 安全层级:
        ↳ 身份与权限|网络|运行时|策略执行",
    "method": ["string - 具体方法与机理说明（需与 action
        ↳ 一一对应）"],
    "action": ["string - YAML/CLI 片段（仅代码/命令;
        ↳ 换行用\\n; 需与 method 一一对应）"]
}
}

```

工具提示

你可以结合分析和选型结果，灵活选择并说明如 OPA、Kyverno、Falco、

- ↳ Tetragon、NetworkPolicy 等主流安全工具。建议在 mitigation
- ↳ 字段中明确工具名称及其作用，但不强制定限。特别注意，Tetragon
- ↳ 可同时实现监控与防护，适用于需要闭环响应的场景。

输出规则

1. 仅返回 JSON：只输出单个 JSON 对象，不要附加说明、Markdown、
 - ↳ 表格或其它文本。
2. 严格格式：字段、层级、顺序必须与上方 schema/示例一致，
 - ↳ 所有字符串使用双引号并保持合法 JSON。

3. 信息缺失：无法确定时写 "(info needed)",
↪ 并在对应字段中说明必要假设。
4. 配置转义：YAML/CLI 片段的换行统一写成 `\\n`。
5. 具象缓解：禁止仅写“升级补丁”，
↪ 需给出可执行的控制措施与配置步骤（如适用）。
6. 一致性：内容必须与输入/分析/评审结论一致，禁止引入无依据内容。
↪

为保证与本文形式化定义一致，策略内容采用 `layer/method/action` 的表达方式：其中 `layer` 描述防控所在安全域；`method` 给出每条控制措施的工具/控制点与机理摘要；`action` 承载与之对应的可执行配置片段（YAML/CLI），二者以数组下标一一对齐，共同构成可执行的动作序列。当需要给出多条措施时，统一使用转义换行 `\n` 以保持 JSON 合法性。

- **layer 字段**：防控所在的技术层面，用于分类防控策略所涉及的安全域。根据策略类型 `type` 的不同，`layer` 可分为：
 - 监控策略层：运行时检测、事件观测、证据留痕等；
 - 防护策略层：身份与权限、网络隔离、准入控制等。
- **method 字段**：防控方法的摘要或分类，用于快速理解防控策略的整体思路，例如：
 - 监控策略：实时行为监测与告警、异常访问检测与取证等；
 - 防护策略：网络隔离与认证强化、权限最小化与配置加固、准入约束与阻断等。
- **action 字段**：与 `method` 一一对应的可执行片段序列，遵循定义 3.1（见定义 3.1.1）。根据策略类型的不同，包含内容有所差异：
 - **监控策略 action**：包含监控规则配置（如 Falco 规则、Tetragon Tracing Policy）、检测策略部署命令、告警配置、日志采集与关联规则、异常行为验证与溯源步骤等；
 - **防护策略 action**：包含准入策略配置（如 Kyverno、OPA Gatekeeper 规则）、权限收敛配置（如 RBAC、ServiceAccount 修订）、网络隔离策略（如 NetworkPolicy）、运行时安全策略（如 Seccomp、AppArmor）、副作用评估与回滚方案等。

为提高交付可用性，本文对候选策略施加基本质量要求：操作序列应可

在短时间内启动；至少覆盖一个明确的防控层次；包含可验证的执行步骤；对潜在副作用与回滚边界作出清晰说明。

候选策略集合 Π_v 的生成可采用多策略并行方法：对同一漏洞产生多个结构化策略对象，以探索不同的防控层次和防控方法。由于所有候选共享统一的结构表示 (layer、method、actions)，它们可直接进入后续的评价与排名流程。

在交付前，生成结果需进行规范化处理：统一各字段的格式、补齐缺失的操作步骤、清理冗余信息，并在数据缺失或不完整处显式标注假设条件和依赖的前置条件。

3.3.3 基于评审反馈的策略优化

该步骤通过“评审—反馈—再生成”对候选策略进行质量控制，降低误报与运维噪声，提升可部署性与一致性。系统并行生成多份候选形成 Π_v ，由策略评审智能体给出结构化评审（有效点、风险、验证要点与缺失信息），再由策略优化智能体合并可行措施、剔除不可行建议，并补齐前置条件与回滚方案。

策略评审智能体提示词模板

你是一名资深 Kubernetes 安全架构师，负责评审多份安全策略。

- 目标是识别关键控制措施，定位误报与运维噪声来源，
- 并明确上线前的验证需求。

输入说明

- CVE 信息：含编号、摘要、CVSS、受影响场景等
- 原始策略列表：包含 profile 标签、JSON 策略、生成摘要
- 补充约束：可选背景、现有控制、风险偏好

评审要求

1. 有效性 (effective)：提炼每份策略最具价值、可快速落地的控制点
2. 干扰/误报 (noise)：指出可能带来误报或运维噪声的配置，
 - 并提出缓解思路
3. 验证 (validation)：明确上线前或运行期需要的验证步骤、
 - 前置条件或监控指标

4. 一致性：若发现明显错误、冲突、缺失信息，必须指出
5. 输出精炼：不要打分，避免冗长段落

输出格式

仅返回 JSON，结构如下：

```
{
  "overall_findings": [" 整体发现 1"],
  "per_strategy": [
    {
      "profile": "llm_profile_name",
      "strengths": [" 有效点 1"],
      "gaps": [" 缺口 1"],
      "risks": [" 潜在风险 1"],
      "noise_sources": [" 潜在噪声来源 1"],
      "validation_needs": [" 验证要点 1"],
      "actionable_feedback": [" 落地改进建议 1"]
    }
  ],
  "missing_information": [" 待补充信息 1"],
  "assumptions": [" 关键假设 1"],
  "confidence": " 高/中/低"
}
```

输出规则

1. 仅返回 JSON：只输出单个 JSON 对象，不要附加说明、Markdown、
↪ 表格或其它文本。
2. 严格格式：字段、层级、顺序必须与上方给出的结构规范/示例一致，
↪ 所有字符串使用双引号并保持合法 JSON。
3. 信息缺失：无法确定时写 "(info needed)"，并在字段或
↪ assumptions 中说明假设。
4. 配置转义：YAML/CLI 片段的换行统一写成 `\\n`。
5. 具象缓解：禁止仅写“升级补丁”，
↪ 需给出可执行的控制措施与配置步骤（如适用）。

6. 一致性：内容必须与输入/分析/评审结论一致，禁止引入无依据内容。

↔

策略优化智能体提示词模板

你是一名 Kubernetes 安全架构师，

↔ 负责在评审结果的基础上生成最终的优化策略。

输入说明

- CVE 信息：含编号、受影响系统、漏洞摘要
- 原始策略列表：来自不同 LLM 的安全策略
- 评审结论：上一阶段输出的评审 JSON
- 补充约束：可选背景、现有控制、风险偏好

任务

1. 汇总评审中确认的有效控制，剔除不可行或噪声过高的措施
2. 生成符合既定结构规范的最终策略，mitigation.action 使用 1.
 - ↔ ...\n2. ... 结构并保持与历史输出一致，
 - ↔ 尽量包含可直接落地的策略代码片段。
3. 若信息缺失，写入 assumptions 或在相关字段标注 (info needed)

输出格式

仅返回 JSON，结构如下：

```
{
  "optimized_strategy": {
    "cve_id": "CVE 编号",
    "description": " 中文描述（无换行）",
    "application": " 受影响环境或（信息缺失）",
    "threat_analysis": {
      "preconditions": "...",
      "attack_path": "...",
      "blast_radius": "..."
    },
    "risk_category": " 风险分类",
  }
}
```

```

      "mitigation": {
        "layer": " 身份与权限 / 策略执行",
        "method": " 与示例保持一致的组合描述",
        "action": "1. ...\\n2. ...\\n3. ..."
      }
    },
    "improvements": [
      {
        "topic": " 改进主题",
        "details": " 简述优化内容及预期效果",
        "derived_from": ["profile_a"]
      }
    ]
  }
}

```

输出规则

1. 仅返回 JSON：只输出单个 JSON 对象，不要附加说明、Markdown、
↪ 表格或其它文本。
2. 严格格式：字段、层级、顺序必须与上方给出的结构规范/示例一致，
↪ 所有字符串使用双引号并保持合法 JSON。
3. 信息缺失：无法确定时写 "(info needed)"，并在字段或
↪ assumptions 中说明假设。
4. 配置转义：YAML/CLI 片段的换行统一写成 `\\n`。
5. 具象缓解：禁止仅写“升级补丁”，
↪ 需给出可执行的控制措施与配置步骤（如适用）。
6. 一致性：内容必须与输入/分析/评审结论一致，禁止引入无依据内容。
↪

此外，策略生成智能体集成 MCP，对生成的 YAML/CLI 等片段做语法检查，将可部署性相关的低级错误前移。检查重点包括：**语法正确性**（如 YAML 缩进、引号闭合）、**最小格式约束**（必要字段、占位符残留）、**结构一致性**（与 layer/method/actions 的对应关系）。

当检测到语法错误或不满足最低格式约束时，MCP 将返回结构化错误信息（错误类型、定位位置、最小修复建议等）。策略生成智能体把该错误信息作为反馈输入，触发对相关片段的**定向迭代修正**：仅修改出错片段及其必要上下文，保持其他已通过检查的内容不被无关改写；修正后再次调用 MCP 复检。该迭代过程设置最大轮次上限，以避免无限循环；若达到迭代上限仍无法通过检查，则将该候选标记为需人工复核或需补充信息，并在后续质量评价环节中置末位，以保证最终交付策略具备基本可部署性。

3.4 策略评估与质量控制

该阶段对候选策略集合进行结构化评价，输出三维质量向量 $q(\pi)$ 及其在各维度上的排名结果，整体流程遵循结构化输入 $\rightarrow$ 规则化校验 $\rightarrow$ 证据支撑的结论的路径，以提高结论的可验证性与可复核性。相较于绝对分值评定，基于排序的评判方式在 LLM-as-a-Judge 场景下通常更具说服力与稳定性。绝对评分要求模型在不同样本之间保持一致且可解释的评分尺度，而已有研究表明，LLM 在打分过程中并未形成稳定的内部评分函数，而是更多依赖启发式决策策略 [54]。在大规模写作评估任务中，Velez 等发现 LLM 在相对排序任务上的一致性显著优于绝对评分，其评分结果也更接近人类评估者的判断 [55]。此外，绝对评分更容易受到文本长度、表达风格及答案顺序等非语义因素的干扰，从而导致系统性偏置甚至可被对抗性操控 [56, 57]。相比之下，排序性评判通过将评估问题简化为局部比较，有效降低了模型的认知与建模负担，并在一定程度上缓解了评分尺度漂移与偏置累积问题。因此，现有工作普遍倾向于采用成对比较或排序机制作为 LLM 评判的基础形式，而非直接依赖绝对分值输出 [58, 59]。

3.4.1 评价算子

为保证评价结果的可复核性与可比性，本文将评价算子划分为两类：**LLM 支持的评价算子**与**专家人工评价算子**。两类算子在评价维度与输出形式上保持一致，均以三维质量向量 $q(\pi) = (E(\pi), I(\pi), D(\pi))$ 为依据，输出按维度的排名结果；差异在于评价主体与证据使用方式：前者由评价智能体在提示词约束与证据上下文支持下给出结构化结论，后者由安全专家结合现场约束与经验进行复核与调整。

(1) **LLM 支持的评价算子**。该算子以候选策略集合 Π_v 为输入，在固定

的评价准则与输出结构规范约束下，对候选策略在有效性 E 、无干扰性 I 、可部署性 D 三个维度进行比较并给出排序。为降低模型的提示偏置与先验印象影响，评价前对候选策略进行**随机化与匿名化**处理：

- **随机化**：对候选策略列表进行随机打乱，避免顺序效应；
- **匿名化**：移除或遮蔽与生成来源相关的元信息（如模型名称、profile 标签、生成时间等），仅保留内容本身，并为每条候选分配不可语义化的匿名编号（如 $S_1, S_2, \dots$ ）。

为保持可追溯性，系统同时记录匿名编号与原始候选的映射表，供最终复核与实验统计使用。

（2）专家人工评价算子。该算子由安全专家基于同一评价维度与检查清单，对候选策略进行复核：重点验证关键前置条件是否成立、控制点是否覆盖主要攻击路径、策略副作用与回滚边界是否明确，以及配置片段是否符合组织内基线与现场约束。专家评价可以对 LLM 排名结果进行解释性标注与必要校正，并在存在高风险不确定性时给出需补充信息或需灰度验证的结论。

评价智能体 PROMPT 模板（LLM 支持的评价算子）

EVALUATION_PROMPT_TEMPLATE = r"""# Kubernetes

↪ 策略三维评价与排名

你是一名资深云原生安全评审专家，

↪ 负责对候选安全策略进行三维评价与排名。

输入说明

- 候选策略列表（已匿名化、已随机化）：每项包含 id 与

↪ strategy_json

- 评价维度：有效性 E / 无干扰性 I / 可部署性 D

评价要求

1. 仅比较内容本身：不得根据任何生成来源、模型、提示词、

↪ 作者等信息推断质量（输入中已匿名化）。

2. 三维分别排序：分别给出 $E/I/D$ 三个维度的从优到劣的排名列表。

3. 维度解释口径：

- E (有效性)：是否覆盖关键攻击路径/控制点，
 ↳ 是否具备明确的风险降低机制；
- I (无干扰性)：对业务影响、误报/运维噪声、
 ↳ 过度限制风险越低越好；
- D (可部署性)：步骤清晰、可操作，且配置片段语法/结构合理。

4. 排序约束：不允许出现名次相同的情况；当难以区分时，
 ↳ 仍需依据明确的次级准则给出严格排序，并在 **notes**
 ↳ 中简要说明判别依据。

输出格式

仅返回 JSON，结构如下：

```
{
  "rankings": {
    "E": ["S3", "S1", "S2"],
    "I": ["S2", "S1", "S3"],
    "D": ["S1", "S3", "S2"]
  },
  "notes": ["如发现明显语法/结构问题，
    ↳ 或在必须打破接近同分的情况下使用了次级判别准则，
    ↳ 可在此简要说明。"]
}
```

输出规则

1. 仅返回 JSON：不要附加说明、Markdown、表格或其它文本。
 2. 不得泄露匿名映射：不得尝试恢复或猜测候选来源。
- ""

3.4.2 多智能体评估体系

依据第 3.1 节的形式化定义，本文将候选策略 $\pi \in \Pi$ 的质量表示为三维质量向量 $q(\pi) = (E(\pi), I(\pi), D(\pi))$ ，并在每个维度 $\delta \in \{E, I, D\}$ 上要求对给定候选集合 Π_v 输出严格排序 $R_\delta(\Pi_v)$ (定义 4-6)。因此，多智能体评估体系的目标可表述为：对同一候选集合 Π_v ，引入多个评价主体 $\mathcal{H} =$

$\{h_1, \dots, h_m\}$, 由每个 $h \in \mathcal{H}$ 独立实现维度排名算子 $R_\delta^{(h)}(\Pi_v)$, 并输出三维严格排序 $\{R_E^{(h)}, R_I^{(h)}, R_D^{(h)}\}$, 作为后续聚合与选择的基础信号。

为降低单一模型偏差与不稳定性对结论的影响, 本文采用多评审的同口径可审计组织方式: 由多个不同来源或不同配置的基于大型语言模型 (LLM) 的评价算子对同一输入进行**独立评审**, 再将各评审结果聚合为总体排序。该设计遵循两条原则: 其一, **多样性**, 通过引入不同能力侧重的评审主体覆盖更多评价视角 (例如对有效性证据链的敏感度、对业务干扰的谨慎程度、对可部署细节的严格性); 其二, **独立性**, 各评价算子在同一评价准则与输出结构规范约束下独立输出, 避免评审主体之间接触对方结论而引入一致性偏置。

具体实现上, 系统对通过规则化校验的候选集合 Π_v 执行匿名化与随机化处理后, 将相同输入分发给各 $h \in \mathcal{H}$ 。在输出层面, 本文以定义 6 的排名尺度为主: 每个 h 仅需在每个维度上给出一个全序排序, 并可在必要时返回简短的判别依据与风险提示。该设计避免了评分尺度 (定义 5) 在不同评审主体之间的绝对口径差异, 从而更利于跨样本复现与统计分析。为增强鲁棒性与可复核性, 多智能体评估体系引入如下控制机制:

- **一致性约束:** 评价输出必须严格遵循既定的结构规范 (仅输出三维严格排序与简短备注), 并显式禁止使用候选来源信息推断质量 (输入已匿名化)。
- **评审权重 (等权):** 本文不区分不同评审主体 (不同 LLM 或专家复核) 的可靠性差异, 默认采用等权设置: 对 $m = |\mathcal{H}|$ 个评审主体, 取 $w_h = \frac{1}{m}$, 并满足 $w_h \geq 0$ 且 $\sum_{h \in \mathcal{H}} w_h = 1$ 。该设置更简洁、可复现, 也避免引入额外的权重估计过程与主观性。
- **分歧触发复核:** 当不同评审在某策略/某维度上分歧显著时, 将该策略标记为高不确定性, 优先进入专家复核或补充信息流程, 避免仅凭单一评审结论做交付决策。

在上述体系下, 评价算子的输出不再是不可解释的单点分数, 而是可追踪的多源排序信号; 下一小节将给出将多智能体三维排序聚合为总体质量评价与总排名数学模型。

3.4.3 多源排名聚合模型

本节给出一个面向实验可复现的聚合模型。其核心目的不是引入新的主观打分，而是在多评审等权的前提下形成可审计的共识排序。具体而言，在每个维度 $\delta \in \{E, I, D\}$ 上，评价算子仅输出严格排序；系统将排序映射为秩并进行加权聚合。三维维度权重 $\alpha_E, \alpha_I, \alpha_D$ 反映业务场景偏好，可按需求进行定制化设定；评审权重则采用等权设置 $w_h = \frac{1}{m}$ ($m = |\mathcal{H}|$)，以保证模型简单且易复现。

(1) 输入：多评审排序结果。令 Π_v 表示通过规则化校验的候选集合， $|\Pi_v| = n$ ；令 $\mathcal{H} = \{h_1, \dots, h_m\}$ 表示参与评审的评价算子集合（可为多个评审 LLM，必要时可包含专家复核）。每个 $h \in \mathcal{H}$ 在维度 $\delta \in \{E, I, D\}$ 上输出一个严格排序：

$$R_\delta^{(h)}(\Pi_v) \rightarrow (\pi_{(1)}^{\delta, h}, \pi_{(2)}^{\delta, h}, \dots, \pi_{(n)}^{\delta, h})$$

其中 $\pi_{(1)}^{\delta, h}$ 表示在评审 h 的维度 δ 上最优候选。排序在匿名编号 (S1、S2、...) 上进行，最终通过映射表恢复到原策略用于审计与交付。

(2) 将排序映射为秩函数。为便于计算，定义秩函数 $r_\delta^{(h)} : \Pi_v \rightarrow \{1, \dots, n\}$ ：

$$r_\delta^{(h)}(\pi) = 1 + |\{\pi' \in \Pi_v \mid \pi' \succ_\delta^{(h)} \pi\}|$$

其中 $\pi' \succ_\delta^{(h)} \pi$ 表示在 h 的维度 δ 排序中 π' 严格优于 π 。在严格排序约束下，每个候选获得唯一秩，且 $r_\delta^{(h)}$ 为一一映射。

上述映射不引入新的偏好假设：它仅将排序的“相对位置”编码为整数，从而使排序结果可以直接进入后续的加权 Borda 聚合与分歧度量计算。

(3) 按维度聚合。本文采用基于定义 6 的维度得分函数进行加权聚合。对 $m = |\mathcal{H}|$ 个评审主体取等权设置 $w_h = \frac{1}{m}$ ，并满足 $w_h \geq 0$ 且 $\sum_{h \in \mathcal{H}} w_h = 1$ 。

对每个评审主体 $h \in \mathcal{H}$ ，根据定义 6 将其在维度 δ 上对候选 π 的排名 $r_\delta^{(h)}(\pi)$ 转换为归一化得分：

$$s_\delta^{(h)}(\pi) = \frac{n - r_\delta^{(h)}(\pi)}{n - 1}$$

随后对所有评审主体的维度得分进行加权平均，得到维度 δ 的共识得分 $\hat{S}_\delta(\pi) \in [0, 1]$ ：

$$\hat{S}_\delta(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot s_\delta^{(h)}(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot \frac{n - r_\delta^{(h)}(\pi)}{n - 1}$$

基于共识得分可得到维度 δ 的共识排序 $\bar{R}_\delta$ ：按 $\hat{S}_\delta(\pi)$ 从大到小进行排序。

为保证输出为严格排序，若出现数值相等，则按预先约定的确定性规则打破平分，例如依次比较各评审主体的原始排名并以匿名编号作为最终判别。

为刻画“评审一致性/不确定性”，可进一步定义分歧度量，例如秩的加权方差：

$$U_\delta(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot (r_\delta^{(h)}(\pi) - \bar{r}_\delta(\pi))^2, \quad \bar{r}_\delta(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot r_\delta^{(h)}(\pi)$$

当 $U_\delta(\pi)$ 较大时，表示该策略在该维度上存在明显分歧，需在后续决策中优先人工复核。

(4) 总体质量评价：三维共识到总排名。在获得三维共识质量后，给出总体质量函数 $Q : \Pi_v \rightarrow [0, 1]$ ：

$$Q(\pi) = \alpha_E \hat{S}_E(\pi) + \alpha_I \hat{S}_I(\pi) + \alpha_D \hat{S}_D(\pi), \quad \alpha_E + \alpha_I + \alpha_D = 1$$

其中 $\alpha_E, \alpha_I, \alpha_D$ 是三维度不同权重，用于将“有效性/无干扰性/可部署性”的偏好映射到最终排序。该权重可按业务场景配置：例如在应急与临时缓解场景下可提高 α_E, α_D ；在生产稳定性优先场景下可提高 α_I 。

在本文的安全防控语境下，若无额外业务约束，默认偏好应体现有效性优先”：即 $\alpha_E > \alpha_I > \alpha_D$ 。例如可取 $\alpha_E = 0.5, \alpha_I = 0.3, \alpha_D = 0.2$ 作为默认基线（具体数值可结合组织的风险偏好与运维能力进行微调）。

为避免“权重选择导致结论偶然变化”，实验中可对 α 做简单敏感性分析（例如在合理范围内扰动 α 并观察总排名是否稳定），将不稳定样本优先交由专家复核。最终总排名 $\bar{R}$ 由 $Q(\pi)$ 从大到小排序得到；为保证输出为严格排序，若出现 $Q(\pi)$ 相等，则按 $(\hat{S}_E, \hat{S}_D, \hat{S}_I)$ 的词典序进行细化，并在仍无法区分时以匿名编号作为最终判别，从而得到可审计的最终交付结论。

3.5 小节

通过本章的分析，我们可以得出面向云原生权限提升漏洞的智能防控技术框架，包括漏洞实例、上下文、安全策略的形式化定义，以及基于多智能体协同的策略生成与优化方法。该框架通过对漏洞的深入分析与智能化策略生成，实现了对云原生应用漏洞的有效防控，为后续的部署与处置决策提供了可靠依据。

4 面向云原生权限提升智能化漏洞防控系统工具

基于前述方法论，本章设计并实现 KPEAgent 系统原型。该原型将第三章提出的 RAG-CoT 生成框架与人机协同评审机制落地为可交互的工具，供安全工程师在实际场景中完成从漏洞分析到策略部署的完整闭环。下文依次从需求、架构与模块三个层面展开。

4.1 需求分析

4.1.1 用户角色与用例分析

按照职责划分，系统面向两类协同用户：**安全工程师与专家评审员**（见图 4-1）。安全工程师是系统的主要操作者，负责漏洞生命周期的全程跟踪，利用系统的自动化能力完成特征提取、检索增强与推理链构建，从而以极低成本输出初步防御方案；专家评审员则聚焦于决策与质控环节，在人工介入点（Human-in-the-loop）对多份候选策略进行多维度评分与排序，其反馈不仅决定最终部署方案，还作为高质量偏好数据回流至知识库，用于持续校准模型行为。

4.1.2 功能需求

结合上述角色职责，系统须覆盖以下功能：

- **知识库管理**：导入、检索与维护 CVE 漏洞条目，为后续分析积累结构化语料。
- **智能策略生成**：串联 RAG 检索与 CoT 推理，输出面向特定漏洞的候选缓解策略。
- **策略优化与反馈**：接收人工评审意见并驱动模型迭代，逐步收敛策略质量。
- **专家评审与排序**：在可视化界面上对比多份候选策略，按有效性、可部署性、无干扰性等维度排序。
- **策略部署管理**：将通过评审的策略下发至目标 Kubernetes 集群或远程主机。

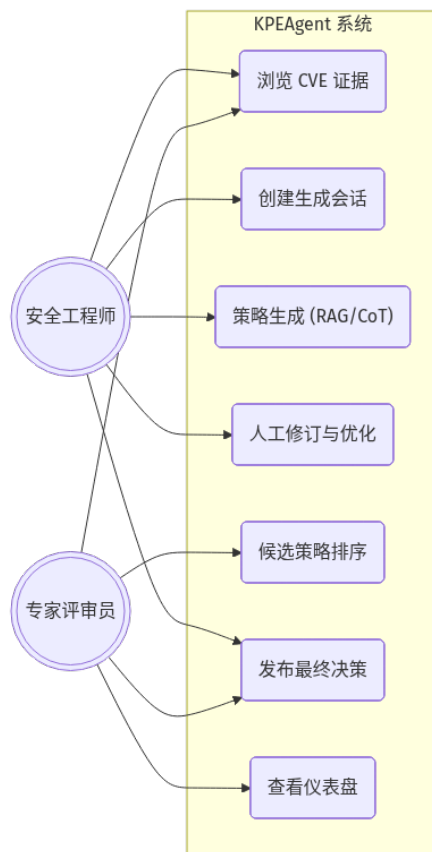


图 4-1: 系统用户角色与核心用例

4.2 总体设计

4.2.1 系统架构设计

系统采用现代分层软件架构设计，自顶向下解耦为四个逻辑层级（图 4-2），确保各层关注点分离：

- **展现层 (Presentation Layer):** 基于 Vue3 框架构建单页应用 (SPA)，集成 ECharts 可视化组件，为用户提供流畅的交互体验，负责 CVE 详情展示、策略对话交互以及评审数据的图形化呈现。
- **服务层 (Service Layer):** 采用 Flask 框架实现 REST API 网关，负责请求路由与鉴权；内置任务调度器 (Scheduler) 管理长耗时推理任务，协调 CVE 管理、策略生成与评审控制等核心业务逻辑模块。
- **智能引擎层 (Intelligence Layer):** 封装系统的核心 AI 能力，独立的 Agent 引擎统筹调用 RAG 检索器与 CoT 推理机，实现从非结构化漏洞描述到结构化防御策略的转化。
- **数据层 (Data Layer):** 负责持久化存储，包括用于存放 CVE 元数据与 JSON 格式策略产物的文件数据库，以及支撑语义检索的向量索引库。

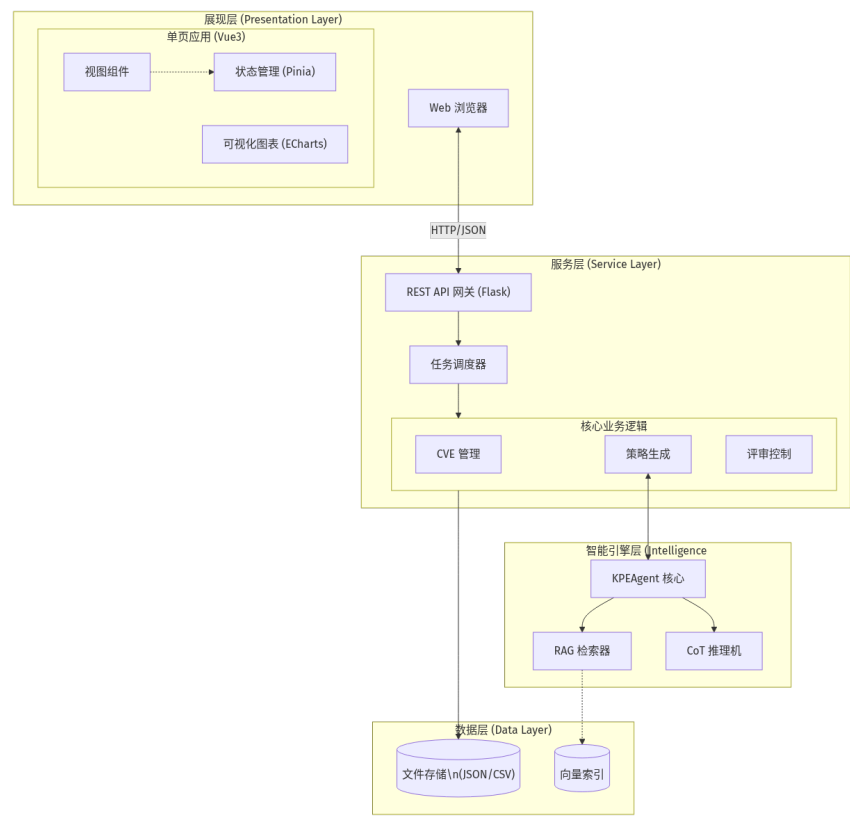


图 4-2: 系统总体架构与分层设计

4.2.2 数据流转与审计设计

为保障关键决策的可追溯性，系统构建了如图 4-3 所示的闭环数据流模型。流程始于安全工程师导入 CVE 知识库（Data Store D1），经过特征提取后进入生成管道；推理引擎（Process P2）结合检索语料输出候选策略并写入策略库（D2）；随后进入人工介入环节（P3），专家评审员的反馈与排序指令直接驱动策略优化与最终选型（P4）；最后，所有的排序记录与决策依据被归档至审计日志（D3），整个过程形成完整的证据链，满足合规审计需求。

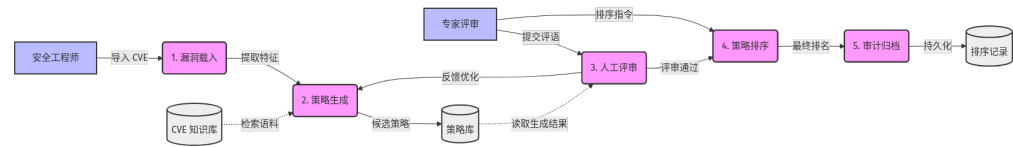


图 4-3: 防控闭环的数据流与审计点

4.2.3 交互时序设计

策略生成通常涉及多次耗时的大模型推理与检索操作。为避免前端长时间等待导致的“假死”现象，系统设计了异步交互时序（见图 4-4）。用户发起生成请求后，后端立即返回任务 ID，并在后台依次触发检索、推理与生成流

程；前端通过轮询或 WebSocket 实时获取各阶段状态更新。这种设计通过将复杂流程拆解为用户可见的细粒度步骤（Step-by-step Visibility），使用户能够实时检视图 4-8 中的检索结果片段或图 4-9 中的推理链条，并在必要时及时终止或干预异常任务。

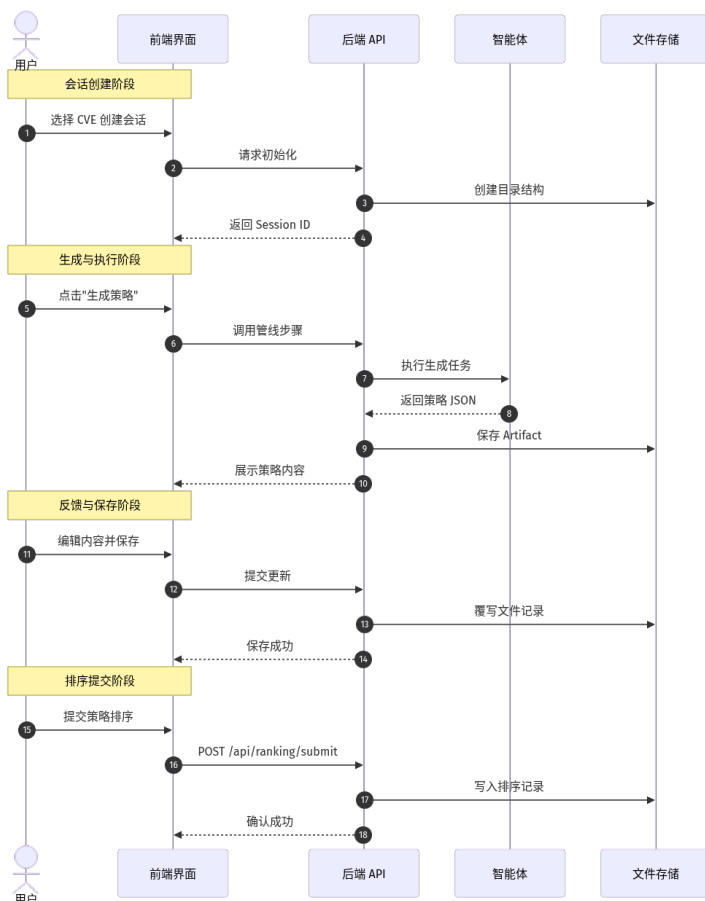
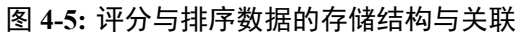


图 4-4: 会话化策略生成与优化的交互时序

4.2.4 数据模型设计

系统的数据持久化方案围绕以下三类核心实体构建（详见图 4-5 类图）：

- **CVE 实体**：作为数据源头，包含 `cve_id`、`summary` 及 `cvss_score` 等描述漏洞严重程度的基础属性，与后续生成的策略呈“一对多”发散关系。
- **Strategy 实体**：记录单次生成会话的完整上下文，包括唯一的 `session_id`、最终生成的策略内容 `content` 以及生成过程中的思维链快照 `reasoning_chain`，实现了生成过程的可复现性。
- **Ranking 实体**：存储专家的决策结果，包含评审人 ID `user_id`、决策时间戳 `created_at` 以及确认后的策略排序列表 `order`，该关联关系体现了专家对特定策略集的即时评价。



本节按功能模块逐一说明界面布局与交互逻辑，配合截图呈现实际运行

知识库模块是整个防控系统的基石，提供了对漏洞元数据的集中化管理。如图 4-6 所示，系统主界面采用列表视图呈现已录入的 CVE 条目，每条涵盖标准 CVE 编号、漏洞及其摘要描述、CVSS 风险评分以及当前状态（如“未处理”、“分析中”或“已部署”）。顶部导航栏集成了全局检索状态筛选器，支持用户通过漏洞编号或关键词（如“Privilege Escalation”）定位目标条目，点击条目后即可进入详情页发起针对性的策略生成会话。

图 4-6: CVE 知识库查看与管理界面

策略生成过程被封装在一个具有上下文感知的”会话”实体中，确保每个漏洞的分析过程独立且连贯。图 4-7 展示了会话启动的初始化界面：左侧为历史会话导航区，记录了过往分析的漏洞对象与时间戳，支持快速切

换上下文；右侧主工作区顶部的步骤条直观指示了当前所处的阶段（Start → Retrieval → Reasoning → Generation），下方配置面板允许用户在启动前选定特定的 LLM 后端（如 GPT-4 或 Qwen-Max）及对应的 Prompt 模板集合。

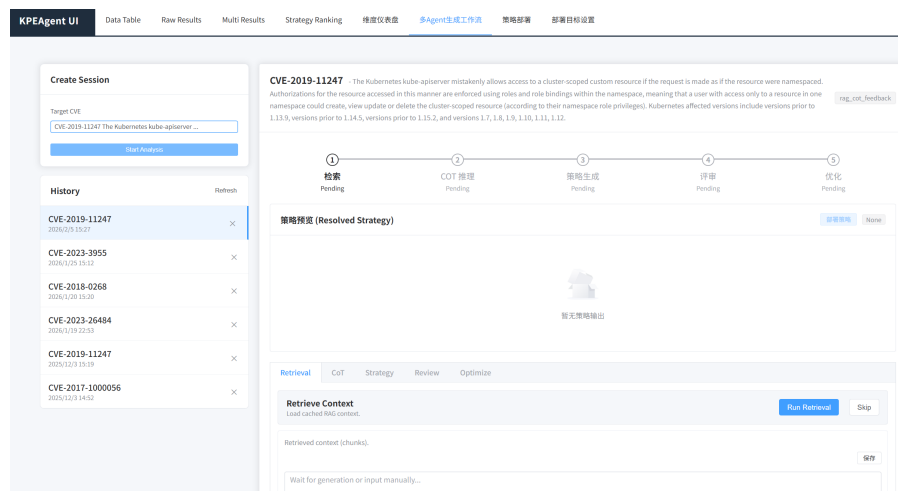


图 4-7: 策略生成会话启动界面

检索阶段（Retrieval）负责构建防御策略生成的知识基座。如图 4-8 所示，RAG 模块界面被划分为控制区与结果展示区：用户点击执行后，系统自动提取 CVE 描述中的关键特征，从本地知识库中召回语义最相关的 Top-K 文档片段。召回结果以卡片形式排列，每张卡片清晰标注了来源文档与相关度得分，用户可在此阶段人工剔除无关片段，确认后的高质量语料将被标记为有效，注入到后续的 Prompt 上下文中。

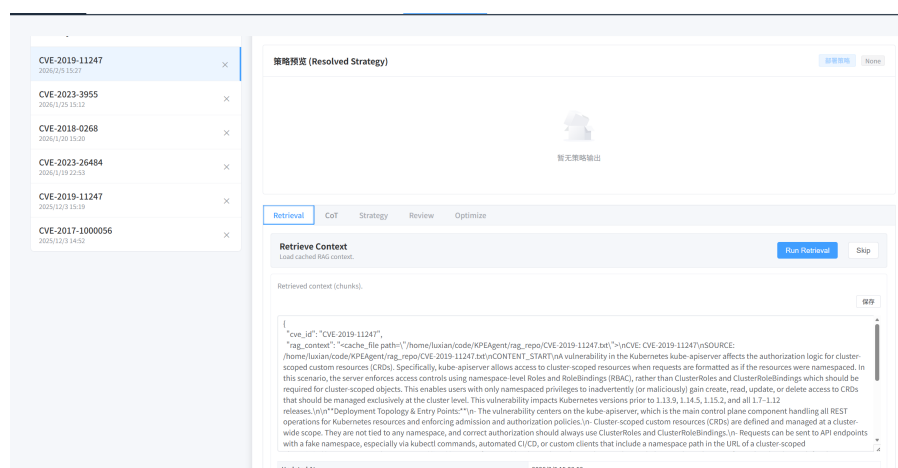


图 4-8: RAG 模块检索结果展示

CoT 推理阶段是系统实现深度分析的核心环节。界面设计（图 4-9）突出了思维链的透明度，左侧提供标准化的推理模板选择，右侧实时显示模型生成的中间推理步骤，例如对攻击路径的分解或对 K8s 权限模型的映射分析。

点击具体的推理条目可展开查看详细内容（图 4-10），系统将复杂的逻辑推导拆解为”背景识别”、”因果分析”与”结论推导”等结构化字段，这种可视化的呈现方式极大增强了安全工程师对生成策略的信任度。

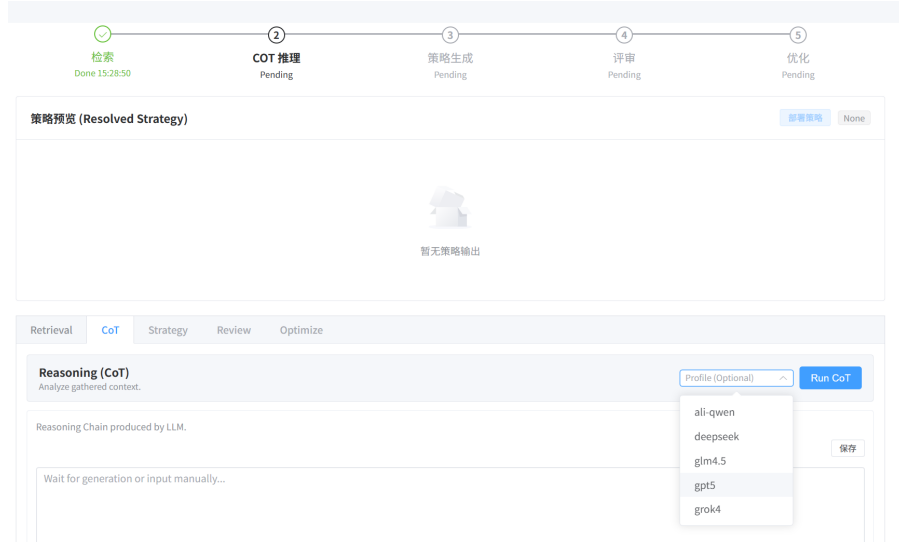


图 4-9: CoT 推理链展示界面

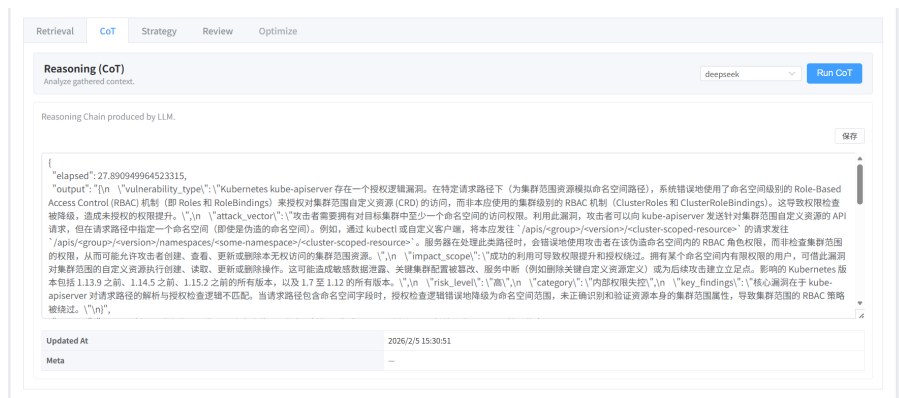


图 4-10: CoT 推理详情

在完成充分的知识检索与逻辑推理后，系统进入最终的策略生成阶段。图 4-11 展示了候选策略的预览界面，生成的防护规则（如 NetworkPolicy 或 OPA Rego 脚本）被渲染为带有语法高亮的代码块。界面右侧提供了一键复制与文件导出功能，并在底部附带了模型对该策略生效原理的简要解释，便于用户在部署前进行最后的静态代码审查。

智能生成并非一蹴而就，系统内嵌了”评审—优化”的闭环机制以应对复杂场景。当评审员提出改进建议（精简规则或放宽特定条件）后，用户可在优化面板（图 4-12）发起再生成请求。图示界面对比了原始版本与依据评审意见修正后的 ****Revised Strategy****，变更部分通过高亮显示。这种增量迭代

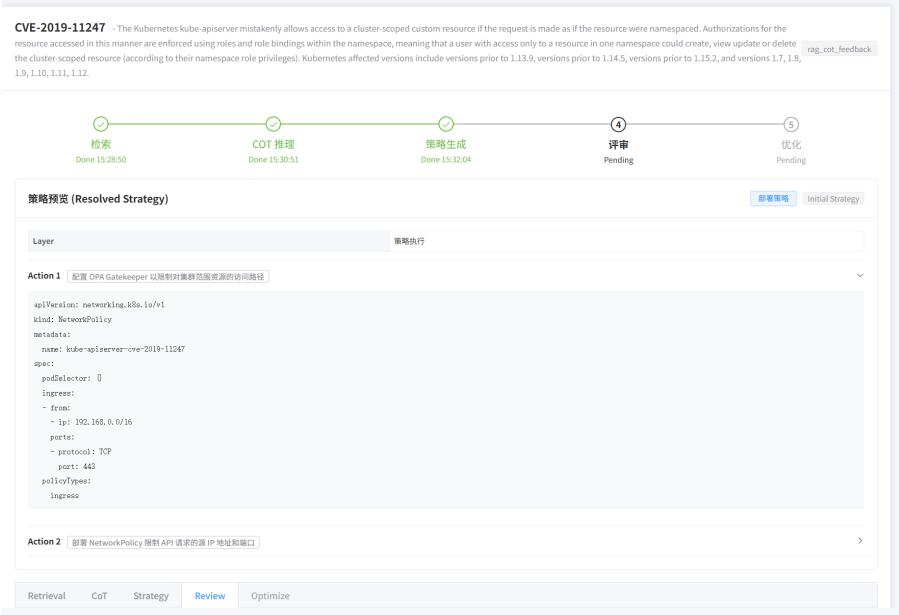


图 4-11: 策略生成初步输出

模式确保了策略质量的单调递增，直至满足生产环境部署标准。

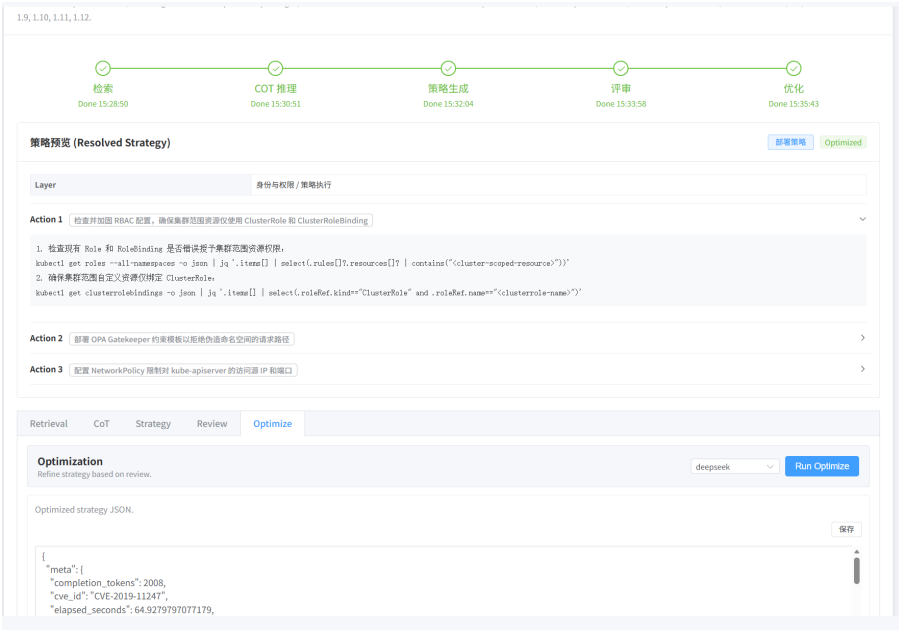


图 4-12: 策略优化后的最终输出

为充分利用专家知识（Human Knowledge）并对生成模型形成对齐信号，系统设计了严谨的”评审与排序”双阶段流程。评审步骤（Review）旨在收集细粒度的定性反馈。图 4-13 呈现了结构化评审详情页：系统不仅展示策略全文，还要求评审员对”语法正确性”、”业务逻辑完备度”等子项进行逐一核查。界面右下角的文本域允许评审员输入自由格式的修改建议，这些非结构化的自然语言评论将被系统记录，作为后续 Prompt 优化的重要语料。

排序步骤（Ranking）聚焦于多候选方案的定量比较与一致性判别。其设



图 4-13: 评审阶段的结构化输出示例

计目标并非仅给出单次排序结果,而是通过可视化约束与交互引导,将专家隐含的评估标准外显为可记录、可复核的偏好信号。图 4-14 为拖拽式排序主界面,左侧列出针对同一 CVE 生成的多个候选策略 (Strategy Candidate A/B/C),右侧设置”有效性”、”可部署性”、”无干扰性”等维度的评分区,并提供评分分布与顺序提示,以降低不同评审员在维度理解与打分尺度上的漂移,从而提升跨样本、跨评审的可比性。

从交互机制看,拖拽排序属于相对判断 (relative judgment) 范式,相比绝对打分更能缓解评审员对分值刻度的主观不稳定性;同时,维度评分区为排序提供显式的理由支撑,使结果具备最小充分解释 (minimal sufficient explanation)。在数据采集层面,系统以”排序”作为主信号,并以维度得分与必要的文本备注作为辅信号,共同构成偏好样本的结构化表示,便于后续进行一致性分析、难例抽样与模型对齐训练。

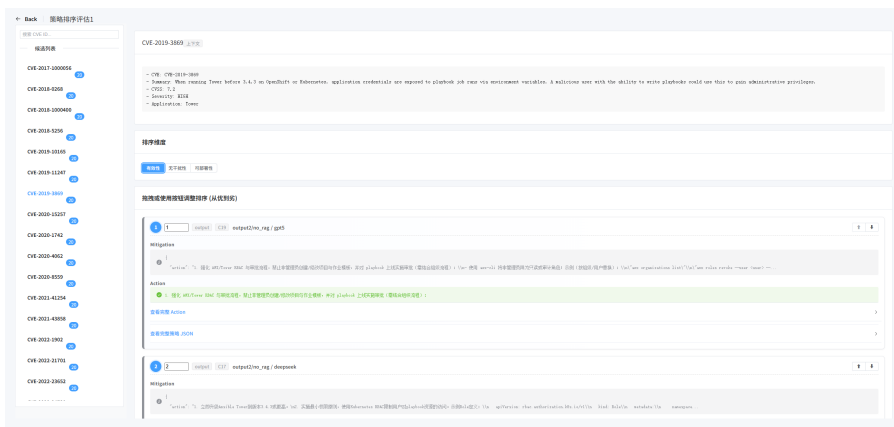


图 4-14: 策略排序主界面

评审员通过拖拽完成候选策略的相对排序,系统同步记录调整轨迹与最终结果,便于后续质量核验与偏好数据回溯。为支持细粒度比对,系统允许点击列表项展开详细信息。如图 4-15 所示,专家可在该界面对不同方案的规则逻辑、关键参数与实现细节进行逐项核对,重点关注三类差异:(1)策略

4.3.3 策略部署管理

完成审核的策略须经由可靠的通道部署至生产环境。系统提供了环境感知型的灵活部署机制，图 4-17 展示了”部署目标设置”的配置表单：对于集群内应用，系统可通过 Python Kubernetes SDK 直连 API Server，用户分别指定 Cluster URL、Token 及 Namespace 即可建立安全连接；对于传统主机环境，则支持 SSH 协议，需配置 Hostname、Port 及认证密钥。界面底部提供”连通性测试”按钮，确保在正式下发策略前通道畅通。

图 4-17: 部署目标配置界面

当策略下发后，部署管理面板（图 4-18）承担起全生命周期管理的职责。该界面汇总了跨集群、跨主机的全部已部署策略实例，关键信息包括策略名称、目标对象（Target）、部署时间及当前状态（Deployed/Failed/Pending）。每一行通过颜色编码的状态徽章（Badge）直观展示健康度，右侧操作列提供”回滚”（Rollback）与”日志查看”功能，极大简化了多云环境下的统一运维复杂度。

图 4-18: 策略部署管理界面

5 实验研究

5.1 研究问题

本章实验旨在验证所提方法在权限提升漏洞实时防控场景下的有效性与适用性，具体围绕以下研究问题展开：

RQ1: 三个核心模块在三维质量指标上的贡献度与跨模型稳健性如何？

RQ2: 不同大语言模型在策略生成任务中呈现怎样的能力异质性？

RQ3: 基于大语言模型的自动化评审方法是否具备足够的内部一致性与外部有效性？

RQ4: 相较于现有静态分析工具，本文方法能否提供更具针对性的漏洞处置方案？

RQ5: 不同模型与方法配置在资源消耗上呈现怎样的成本-效益权衡特征？

5.2 实验设置

实验以 55 个权限提升漏洞实例为对象，在统一的提示与输出结构规范下对五个大模型生成候选策略集合 Π_v 。表5-1 给出本实验所采用的五个大语言模型的基本信息。为验证关键模块的贡献，设置三项消融：移除检索增强 (w/o rag)、移除思维链分阶段推导 (w/o cot)、移除基于评审反馈的迭代优化 (w/o feedback)，其余流程保持一致。

评审阶段将上述五个模型作为评审模型，对 Π_v 进行排名式评价。评价指标采用三维质量向量 $q(\pi) = (E(\pi), I(\pi), D(\pi))$ ，分别对应有效性、无干扰性与可部署性。为缓解大模型评分能力缺陷 [54]，降低不同评审模型评分尺度差异带来的影响，本文以相对排序作为效果评估质量。同时我们将所有评审模型的结果进行聚合，采用算术平均排名作为最终排序依据，以增强结果的稳健性与可解释性。

表 5-1: 实验采用的大语言模型
Tab. 5-1: Large language models used in experiments

模型代号	开发机构	发布日期	参数规模
Qwen3-Max	阿里云	2025 年 9 月	1000B+
Deepseekv3.1	深度求索	2025 年 9 月	670B
GPT-5	OpenAI	2025 年 6 月	未公开
Grok4	xAI	2025 年 7 月	未公开
GLM-4.5	智谱 AI	2025 年 7 月	355B

为回答有关令牌消耗与大模型调用成本的问题，本文在生成与评审阶段记录每次调用的输入与输出的令牌数，并按漏洞样本 CVE 聚合统计令牌消耗；对于包含反馈迭代的配置，累计其多轮调用的令牌消耗。

为对比传统静态安全工具，选取如下四种代表性工具/文献作为基线，输出风险/合规报告后人工评估其对策略生成的启发性，例如能否指向与漏洞路径相关的高风险配置、是否给出可落地的修正建议等：

表 5-2: 静态扫描工具
Tab. 5-2: Static scanning tools

工具	简介	来源
kube-linter	Kubernetes YAML/Helm 的静态规则检查器，提供修复建议。	https://github.com/stackrox/kube-linter [22]
Kubescape	基于合规基线的安全扫描与加固建议工具。	https://github.com/kubescape/kubescape [23]
KubeSec	轻量级配置风险检测与快速巡检工具。	https://kubesec.io/ [25]
SLI-Kube	基于实证研究的配置风险评估规则集扫描工具。	TOSEM 2023 [7]

5.3 RQ1：消融实验与 workflow 模块贡献

本节验证检索增强 RAG、思维链推导 CoT 与反馈迭代 Feedback 三个关键模块对策略生成质量的贡献。通过消融实验，对比完整 workflow all 与三种消融配置 w/o rag、w/o cot、w/o feedback 在有效性 E 、无干扰性 I 与可

部署性 D 三个维度上的表现。

图5-1展示了四种方法配置在三个维度上的排名对比。55 个权限提升漏洞样本分别经五个生成模型生成候选策略，由五个评审模型独立评审并给出排名，最终取算术平均作为综合排名。从图中可见，完整工作流 `all` 在三个维度上均排名第一（综合平均排名 1.00），显著优于任何消融配置。表5-3给出了四种方法配置在三个维度上的详细得分。

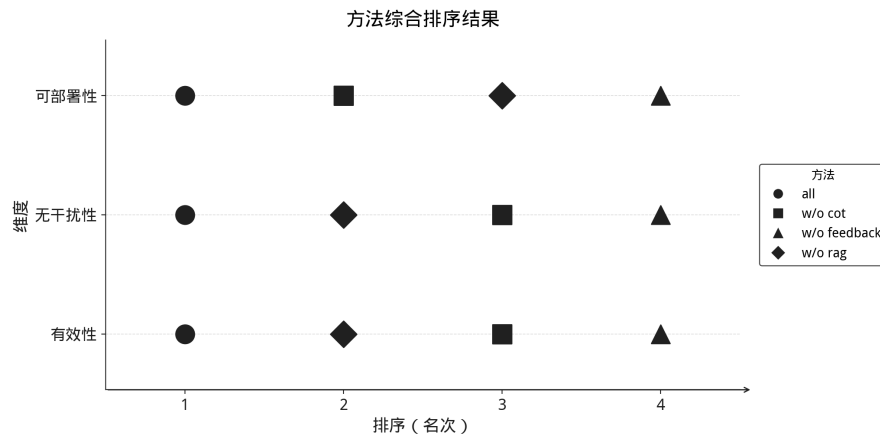


图 5-1: 方法配置在三个维度上的排名对比。

表 5-3: 消融实验方法配置得分
Tab. 5-3: Average scores in ablation study

方法配置	有效性 (E)	无干扰性 (I)	可部署性 (D)
all	0.581	0.431	0.403
w/o rag	0.476	0.370	0.321
w/o cot	0.234	0.340	0.398
w/o feedback	0.121	0.269	0.288

反馈迭代机制 `Feedback` 的核心作用：移除反馈优化后，策略质量全面下降，三个维度得分分别为： $E = 0.121$ ，下降 79.2%； $I = 0.269$ ，下降 37.6%； $D = 0.288$ ，下降 28.5%。这表明迭代优化是工作流的核心环节，评审模型的多轮反馈能够持续引导生成模型修正缺陷，对有效性的提升尤为显著。

检索增强 `RAG` 的知识支撑：`w/o rag` 在有效性维度保持次优，得分 $E = 0.476$ ，显著优于 `w/o cot` 得分 $E = 0.234$ 和 `w/o feedback` 得分 $E = 0.121$ 。

这说明检索增强模块为策略生成提供了关键的领域知识支撑，包括漏洞特征库、历史防控案例与 Kubernetes 安全最佳实践。在缺乏这些知识的情况下，生成模型更易产生不精确或不相关的策略。

思维链推导 CoT 的双刃剑效应：w/o cot 在有效性维度排名最低，得分 $E = 0.234$ ，但在可部署性维度接近完整工作流，得分 $D = 0.398$ vs. $D = 0.403$ 。这揭示了思维链推导的双重作用：虽能显著提升策略的漏洞覆盖率与防控逻辑严密性，但分阶段推导过程引入了更复杂的 YAML 配置结构与依赖关系，增加了部署门槛。在对安全性要求极高的场景下，应优先保留 CoT 模块；而在快速响应场景下，可适当简化推导过程以降低部署复杂度。

综上，三个模块的协同作用为策略生成提供了一致性增益，其重要性排序为：**Feedback > RAG > CoT**。完整工作流在三个维度上分别达到 0.581、0.431 与 0.403 的最优表现，对应有效性、无干扰性与可部署性维度。在实际部署中，若资源受限需简化工作流，建议优先保留反馈迭代机制与检索增强模块。

5.3.1 工作流跨模型稳健性验证

为验证上述消融实验结论是否对不同生成模型稳健，本小节分析完整工作流相对消融配置的优势在不同模型条件下的一致性。图5-2给出不同生成模型在有效性维度的平均排名。从图中可见，完整工作流 all 在五个生成模型条件下均保持最优或次优排名，验证了消融实验结论的跨模型稳健性。不同生成模型对各模块的依赖程度存在差异：GPT-5 与 GLM-4.5 对反馈迭代的依赖性更强，移除反馈后排名下降明显；而 DeepSeek 与 Qwen3 对检索增强的响应更为敏感，说明不同模型在知识库支撑与迭代优化上的需求存在差异。

5.4 RQ2：生成模型能力评估

本节从策略生成质量与输出稳定性两个维度评估五个生成模型的能力特征，识别不同模型在三维质量指标上的取舍倾向与适用场景。

5.4.1 三维质量指标的能力分化

图5-3展示了五个生成模型在有效性 E 、无干扰性 I 与可部署性 D 三个维度上的排名分布。

表5-4给出了五个生成模型在三个维度上的详细得分，揭示了显著的能力

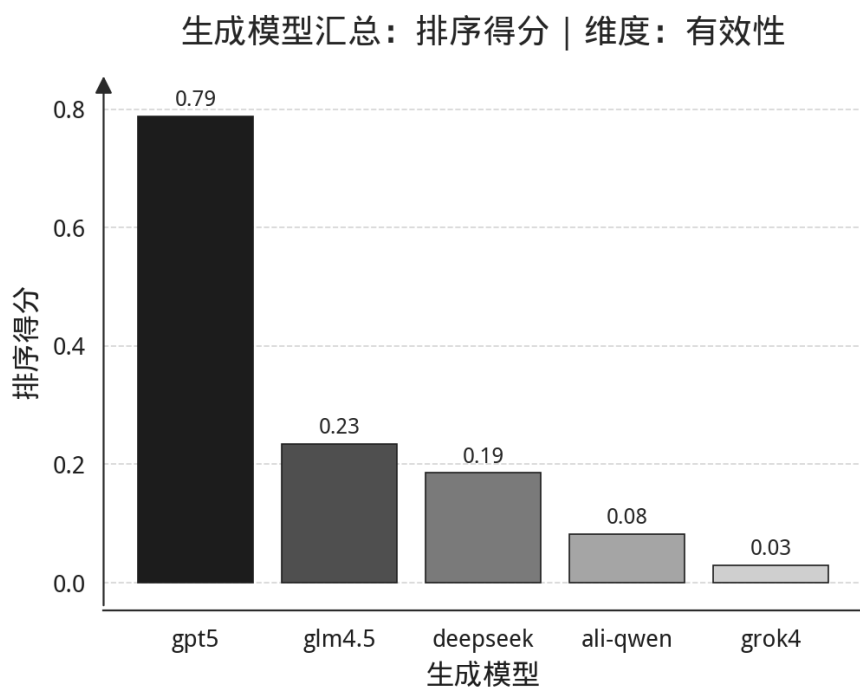


图 5-2: 生成模型有效性维度评分。

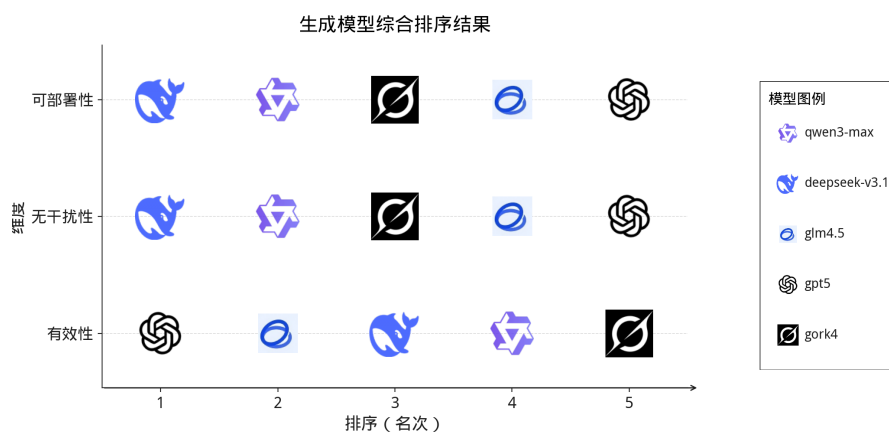


图 5-3: 生成模型在三个维度上的排名对比。

分化现象。

五个生成模型在三维质量指标上表征出显著的能力分化与取舍倾向。GPT-5 在有效性维度遥遥领先 ($E = 0.814$)，倾向于采用严格的 RBAC 限制与网络隔离措施，适用于对安全性要求极高的关键基础设施场景；DeepSeek 在可部署性与无干扰性维度表现卓越 ($D = 0.605$, $I = 0.497$)，擅长生成简洁、低侵入性的策略，成为生产环境快速响应的首选方案。Qwen3-Max 在三维度上保持中上水平 ($E = 0.193$ 、 $I = 0.425$ 、 $D = 0.477$)，无明显短板，可作为通用型选择；GLM-4.5 在三个维度上表现均衡 ($E = 0.327$ 、 $I = 0.324$ 、 $D = 0.336$)，适合作为多模型组合策略中的补充；Grok4 倾向于生成轻量级

表 5-4: 生成模型三维质量得分
Tab. 5-4: Quality scores of generation models

生成模型	有效性 (<i>E</i>)	无干扰 性 (<i>I</i>)	可部署 性 (<i>D</i>)	综合评价
GPT-5	0.814	0.175	0.000	安全优先 型
GLM-4.5	0.327	0.324	0.336	中等均衡 型
DeepSeek	0.284	0.497	0.605	部署优先 型
Qwen3-Max	0.193	0.425	0.477	稳定通用 型
Grok4	0.147	0.342	0.346	轻量快速 型

策略 ($E = 0.147$ 、 $I = 0.342$ 、 $D = 0.346$)，虽然防控效果有限，但部署门槛低、响应速度快，适用于低风险漏洞的快速应急响应。

5.4.2 输出稳定性评估

为评估不同生成模型在策略生成过程中的输出稳定性，本文统计了各模型在生成阶段触发重试的次数。重试机制主要针对三类失败情形：结构化输出的 JSON 格式错误，通常因字段缺失、括号不匹配或编码问题导致；生成内容的语法错误，包括 YAML 语法错误、逻辑矛盾或不可解析的配置片段；以及内容违规，即触发模型供应商的安全策略或内容审核机制。为便于跨模型对比，本文将计数按固定分母 504 归一化为失误率。表5-5 给出五个生成模型的失误率统计。

整体上，GPT-5 在三类错误上均无重试记录，表现出较高的生成鲁棒性；Grok 和 DeepSeek 的失误率较低，输出稳定性良好；Qwen3 与 GLM-4.5 的失误率相对较高，尤其在 JSON 格式错误上，GLM-4.5 的失误率为 4.56%，相当于 23 次失误/504 次调用，反映其在结构化输出约束下的遵循能力有待提升。语法错误维度上，GLM-4.5 同样表现出明显劣势，失误率为 1.98%，相当于 10 次失误/504 次调用，而其余模型失误率均不超过 0.99%，即不超过 5 次失误/504 次调用。值得注意的是，GLM-4.5 是唯一触发内容违规的模型，失误率为 1.98%，相当于 10 次违规/504 次调用，可能与其内容审核策略对高危漏

表 5-5: 生成模型输出稳定性统计
Tab. 5-5: Output stability statistics of generation models

错误类型	DeepSeek 3.1	GLM -4.5	GPT -5	Grok 4	Qwen 3
JSON 格式错误	0.99%	4.56%	0.00%	0.99%	2.58%
语法错误	0.60%	1.98%	0.00%	0.00%	0.99%
内容违规	0.00%	1.98%	0.00%	0.00%	0.00%
合计	1.59%	8.53%	0.00%	0.99%	3.57%

洞描述中的安全关键词过度敏感相关。

该统计为模型选型与多智能体协同策略提供了量化参考：在资源受限或对输出稳定性敏感的场景下，优先选用 GPT-5、Grok 或 DeepSeek；而对于需要多样性候选集的场景，可将 Qwen3 与 GLM-4.5 纳入候选池，但需配置更宽松的重试预算与后处理容错机制。

5.4.3 生成模型能力评估小结

五个生成模型在三维质量指标上表征出明显的分化与取舍倾向：DeepSeek 的部署优势，得分 $D = 0.605$ ，使其成为生产环境首选；GPT-5 的安全优势，得分 $E = 0.814$ ，适用于高安全要求场景；Grok4 可作为快速响应的轻量级方案，可部署性得分 $D = 0.346$ ，响应时间最短。

进一步分析各生成模型的最佳方法配置，如表5-6所示，发现所有模型在完整工作流 all 配置下均达到最优或次优表现，验证了 RQ1 消融实验结论的跨模型稳健性。值得注意的是，DeepSeek 在 w/o cot 配置下的可部署性得分达到 0.749，超过完整工作流的 0.678，进一步证实了思维链推导在提升防控效果的同时会增加部署复杂度的双刃剑效应。

在输出稳定性上，GPT-5、Grok 与 DeepSeek 表现优异，失误率分别为 0.00%、0.99% 与 1.59%，而 GLM-4.5 与 Qwen3 需额外关注结构化输出约束，失误率分别为 8.53% 与 3.57%。这些实证发现为后续基于维度加权的多智能体策略组合提供了依据：在实际部署中，可依据具体漏洞的风险等级与业务约束动态选择或组合不同生成模型——对于高危漏洞优先选择 GPT-5 或 GLM-4.5 以提升防控彻底性，对于需快速部署的场景优先选择 DeepSeek 或 Qwen3-Max 以降低实施难度。

表 5-6: 生成模型最佳方法配置
Tab. 5-6: Best method configurations per model

生成模型	有效性最佳	无干扰性最佳	可部署性最佳
GPT-5	all (0.945)	w/o cot (0.369)	w/o cot (0.256)
GLM-4.5	all (0.670)	all (0.522)	all (0.504)
DeepSeek	all (0.647)	w/o rag (0.634)	w/o cot (0.749)
Qwen3-Max	all (0.609)	all (0.629)	all (0.656)
Grok4	all (0.434)	all (0.589)	all (0.597)

5.5 RQ3: 评审方法可信度评估

为验证大语言模型作为评审方法的可靠性，本节从两个角度评估其可信度：其一，考察不同 LLM 评审模型在单一维度与跨维度场景下的内部一致性，以判断聚合后排序结果的稳定性；其二，将 LLM 评审结果与人工专家评审进行对比，以验证自动化评审方法的有效性。若 LLM 评审模型之间保持较高一致性，且与人工评审结果趋势相符，则可证明该评审方法足以支撑前述跨评审聚合统计与反馈闭环（RQ1–RQ2）。

5.5.1 不同 LLM 的评审一致性验证

为评估 LLM 评审方法的内部稳定性，本小节分析不同 LLM 评审模型在单一维度与跨维度场景下的一致性。若不同 LLM 评审模型对候选策略优劣的判断高度一致，则说明评审方法本身具有较强的鲁棒性，聚合后的排序结果更稳定、更可解释；反之，若一致性较弱，则需要警惕评审方法的不确定性问题。

在固定评价维度下，设共有 n 个候选项与 m 个 LLM 评审模型，候选项索引 $i = 1, \dots, n$ ，评审模型索引 $j = 1, \dots, m$ 。令 r_{ij} 表示评审模型 j 对候选项 i 给出的名次，名次越小表示越优。

Kendall's W 协调系数用于刻画多评审模型整体一致性，取值范围 $[0, 1]$ ， $W = 1$ 表示完全一致：

$$W = \frac{12 S}{m^2 (n^3 - n)} \quad (5-1)$$

其中

$$S = \sum_{i=1}^n (R_i - \bar{R})^2, \quad R_i = \sum_{j=1}^m r_{ij}, \quad \bar{R} = \frac{1}{n} \sum_{i=1}^n R_i. \quad (5-2)$$

两两 **Spearman** 等级相关系数用于刻画任意两个 LLM 评审模型排序趋势的一致性。对评审模型 j 与 k ，其排名向量分别记为 $\mathbf{r}_j$ 与 $\mathbf{r}_k$ ，则

$$\rho_{jk} = \text{corr}(\mathbf{r}_j, \mathbf{r}_k). \quad (5-3)$$

当两个评审模型排序趋势一致时 ρ_{jk} 接近 1，当趋势相反时 ρ_{jk} 接近 -1 。

本文在 $E/I/D$ 三个维度内分别计算 Kendall's W 与评审模型两两 **Spearman** 相关系数，并汇总其分布统计。表5-7给出三维度下的一致性量化指标，图5-4–5-5b展示分布细节。

表 5-7: 不同维度下 LLM 评审模型一致性统计
Tab. 5-7: LLM judge consistency statistics across dimensions

维度	Kendall's W	Spearman 均值	Spearman 中位数	Spearman 最小值	Spearman 最大值
有效性	0.955	0.944	0.944	0.904	0.991
无干扰性	0.846	0.808	0.839	0.585	0.932
可部署性	0.687	0.609	0.689	0.238	0.947

结果表明，LLM 评审模型的内部一致性呈现有效性优于无干扰性、无干扰性优于可部署性的稳定梯度。

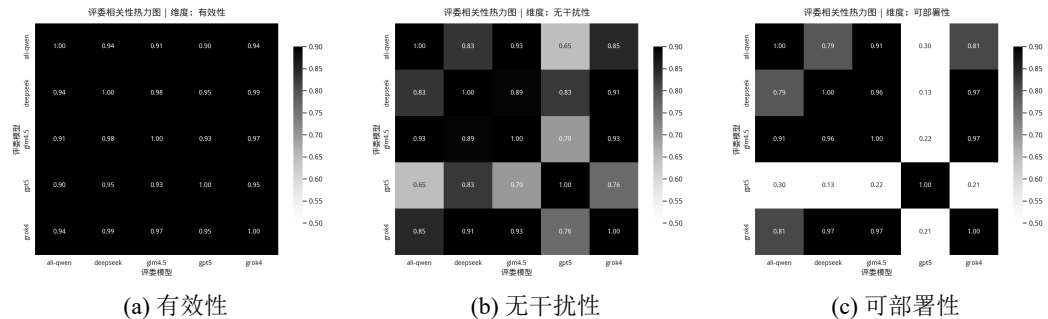


图 5-4: 不同维度下 LLM 评审模型两两相关性热力图 (Spearman)。

从一致性统计结果可以看出，不同评价维度下 LLM 评审模型的一致性存在明显差异。有效性维度的一致性最高， $W = 0.955$ ，Spearman 均值为 0.944，说明 LLM 评审模型对漏洞路径覆盖与处置有效性的判定依据更为一致。这

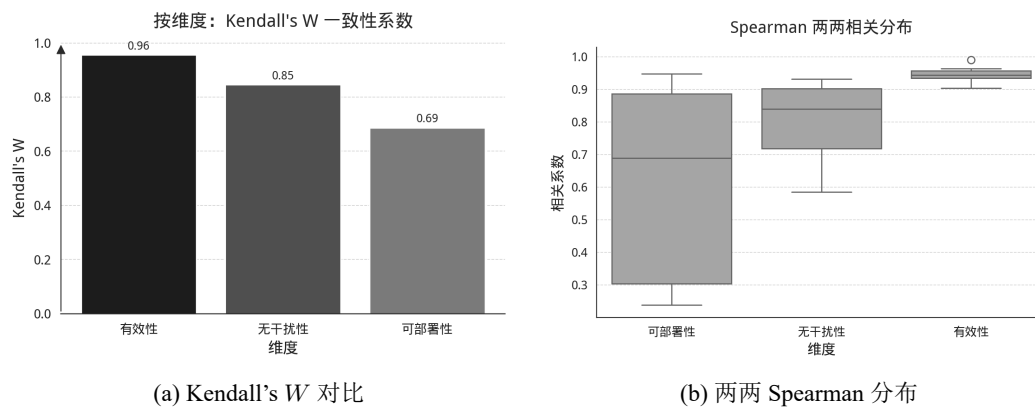


图 5-5: LLM 评审一致性统计。

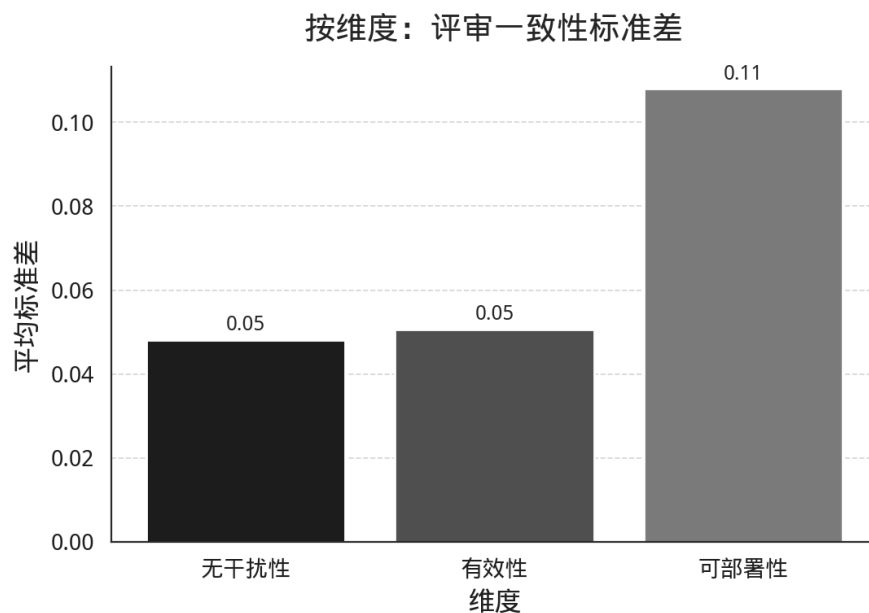


图 5-6: 不同维度下 LLM 评审分歧程度统计。

可能归因于有效性判定标准相对客观——漏洞是否被有效封堵、攻击路径是否被完全覆盖等技术指标具有较强的可验证性。相比之下，可部署性维度的一致性相对较弱， $W = 0.687$ ，Spearman 均值为 0.609，这与该维度涉及的评价因素更为复杂有关：工程可行性、环境依赖、部署成本与运维复杂度等多目标权衡使得不同评审模型对权重与阈值的设定更易出现系统性差异。无干扰性维度的一致性则处于中间水平， $W = 0.846$ ，Spearman 均值为 0.808，反映了 LLM 评审模型在判断策略对业务流程影响时具有一定的共识基础，但仍存在因应用场景理解差异而产生的分歧。

尽管存在维度差异，较高的 Kendall's W 值与 Spearman 均值表明跨评审聚合后的排序整体稳定，LLM 评审方法的内部一致性得到验证。

5.5.2 跨维度评审差异分析

图5-7与图5-8分别从评审模型与生成模型视角展示三维度评价的差异模式。

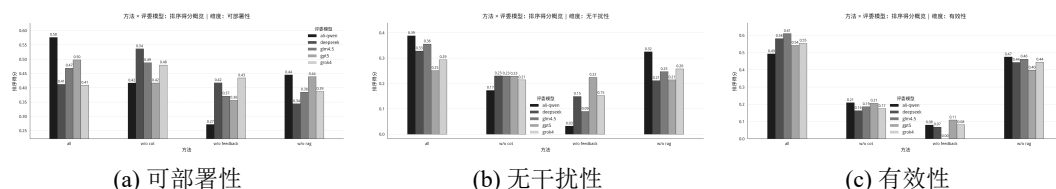


图 5-7: 评审模型单维度评分对比。

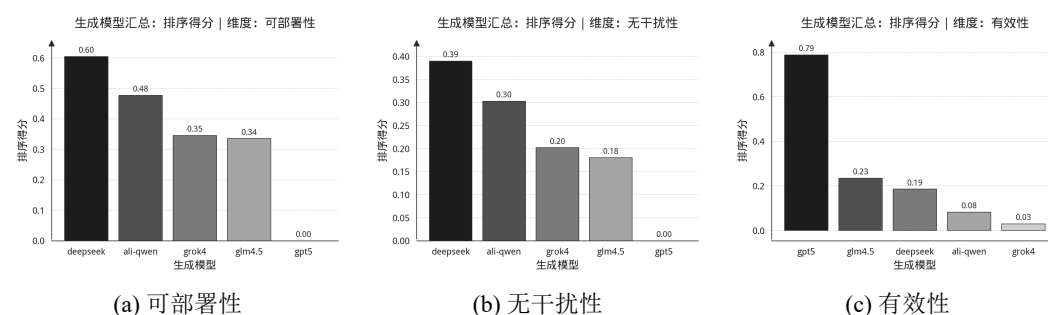


图 5-8: 生成模型三维质量评分对比。

从图5-7可见，评审模型在可部署性维度的判断分歧较大 ($W = 0.687$)，部分更关注配置简洁性，部分更看重依赖完备性；而在有效性维度高度一致 ($W = 0.955$)，验证了技术指标评价的客观性。这表明单一评审模型的绝对评分存在偏差，聚合策略可有效平滑个体差异。

图5-8揭示生成模型的维度取舍特征：GPT-5 呈现“安全优先”——有效性领先但可部署性垫底；DeepSeek 呈现“部署优先”——可部署性与无干扰性领先但有效性中游；Qwen3-Max 与 GLM-4.5 三维度均衡。这为场景化选型提供依据：高危漏洞选用 GPT-5，快速响应选用 DeepSeek，通用场景选用 Qwen3-Max。

进一步分析发现，有效性与可部署性存在竞争关系——追求彻底防控导致配置复杂化 (GPT-5 vs. DeepSeek 对比显著)；而无干扰性与可部署性呈协同趋势——简洁策略通常易于部署。该发现指导策略优化方向：在满足有效性基线前提下，通过简化配置结构同时提升无干扰性与可部署性。

5.5.3 与人工评审的对比验证

为进一步验证 LLM 评审方法的外部有效性，本小节将聚合后的 LLM 评审结果与人工专家评审进行对比。人工评审邀请了三位具有五年以上 Kubernetes 安全运维经验的专家，对随机抽取的 20 个 CVE 样本（涵盖不同漏洞类型与危害等级）进行盲评。每位专家独立评审由五个生成模型在四种方法配置下生成的策略，在有效性 E 、无干扰性 I 与可部署性 D 三个维度上给出排名。最终取三位专家排名的算术平均作为人工评审基准。

权重学习方法。

给定 N 个待评审样本，第 i 个样本有 K 个 LLM 评审者的打分 $\mathbf{x}_i = (x_{i,1}, \dots, x_{i,K})^\top$ 及对应的人工评审分数 y_i 。我们希望找到一组权重，使 LLM 评审的加权结果尽可能逼近人工评审。实验对比两种方案。

方案 1 采用凸组合权重，直接加权平均： $\hat{y}_i = \sum_{j=1}^K w_j x_{i,j}$ ，约束 $w_j \geq 0$ 且 $\sum_j w_j = 1$ 。通过最小化 MSE 求解：

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N \left(\sum_{j=1}^K w_j x_{i,j} - y_i \right)^2 \quad (5-4)$$

方案 2 在加权后再做线性变换：

$$\hat{y}_i = a \left(\sum_{j=1}^K w_j x_{i,j} \right) + b \quad (5-5)$$

权重 $\mathbf{w}$ 保持相同约束， (a, b) 通过最小二乘拟合。这样可以同时校正 LLM 分数的尺度和偏移。在三个评审维度（有效性、无干扰性、可部署性）上分别拟合独立权重。评审者打分从排名转为分数时采用组内归一化： $\text{score} = (N_{\max} - \text{avg_rank}) / (N_{\max} - 1)$ ，其中 $N_{\max}$ 为该组的候选数上限。用留一法（LOO）和 5 折交叉验证评估泛化能力。

拟合结果对比。表5-8展示了两种权重方案与人工评审的拟合结果。

从表5-8可以看出，带校准的方案在 5-fold 验证中整体 MSE 为 0.0331，比凸组合权重的 0.0373 降低了 11%。训练误差的大幅下降（23.6%）与验证误差的温和下降说明校准参数 (a, b) 确实有助于拟合，但也需要警惕过拟合。分维度来看差异明显：有效性维度改善最大（5-fold: 0.0458→0.0335，降低 27%），这可能是因为 LLM 在判断漏洞利用效果时存在系统性的尺度偏差，需要线性变换来对齐人工标准；无干扰性维度也有 22% 的改善；但可部署性维度反而变差（0.0332→0.0402），检查发现该维度下各 LLM 评审者分数高度一致（均匀权重 MSE 已达 0.0339），此时引入校准参数反而带来过拟合风险。

表 5-8: 两种权重方案的交叉验证结果
Tab. 5-8: Cross-validation results of two weighting schemes

方案/维度	训练 MSE	LOO MSE	5-fold MSE
凸组合权重			
有效性	0.0344	0.0496	0.0458
无干扰性	0.0344	0.0324	0.0329
可部署性	0.0344	0.0341	0.0332
总体	0.0344	0.0387	0.0373
带校准权重			
有效性	0.0263	0.0371	0.0335
无干扰性	0.0263	0.0285	0.0256
可部署性	0.0263	0.0378	0.0402
总体	0.0263	0.0345	0.0331
改善幅度	23.6%	10.9%	11.3%

权重分布与校准参数。表5-9给出了最终学到的权重（基于全量数据）。

表 5-9: 学习到的评审者权重分布
Tab. 5-9: Learned weight distribution of LLM judges

评审者	有效性	无干扰性	可部署性
qwen3-max	0.42	0.38	0.20
deepseek	0.18	0.15	0.20
glm4.5	0.15	0.22	0.20
gpt5	0.20	0.18	0.20
grok4	0.05	0.07	0.20

从权重分布可以看出，有效性和无干扰性两个维度上 qwen3-max 获得了最高权重（分别为 0.42 和 0.38），这表明其评审结果与人工专家判断的一致性最高。gpt5 和 deepseek 在有效性维度上也获得了较为显著的权重（0.20 和 0.18），反映出这些模型在判断漏洞防控效果时具有一定的可靠性。可部署性维度呈现均匀分布，说明在这个维度上各评审者与人工判断的相关性都相对较弱，这与该维度涉及的工程实践因素更为复杂有关。

图5-9从生成模型和方法配置两个视角展示了人工评审的三维度评分分布。从图5-9a可见,人工专家评审下 GPT-5 在有效性维度获得最高评价,DeepSeek 在可部署性维度表现最优,与 RQ2 中 LLM 评审的结论一致。图5-9b显示完整工作流在三个维度上均优于消融配置,验证了 RQ1 的结论。对比图5-10所示的 LLM 评审结果,两者在整体趋势上高度一致,但人工评审在可部署性维度上的区分度更高,这可能是因为人工专家能够更准确地评估工程实践中的部署复杂度。

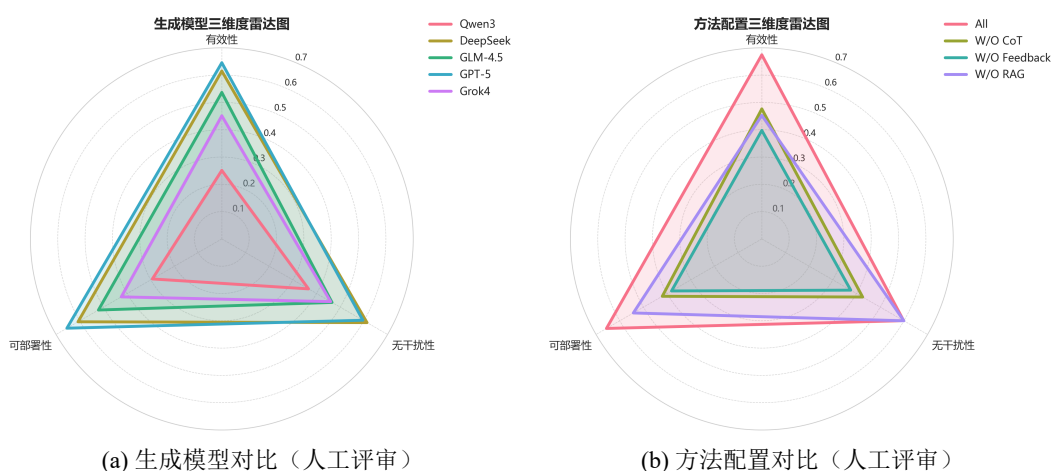


图 5-9: 人工评审的三维度雷达图对比。

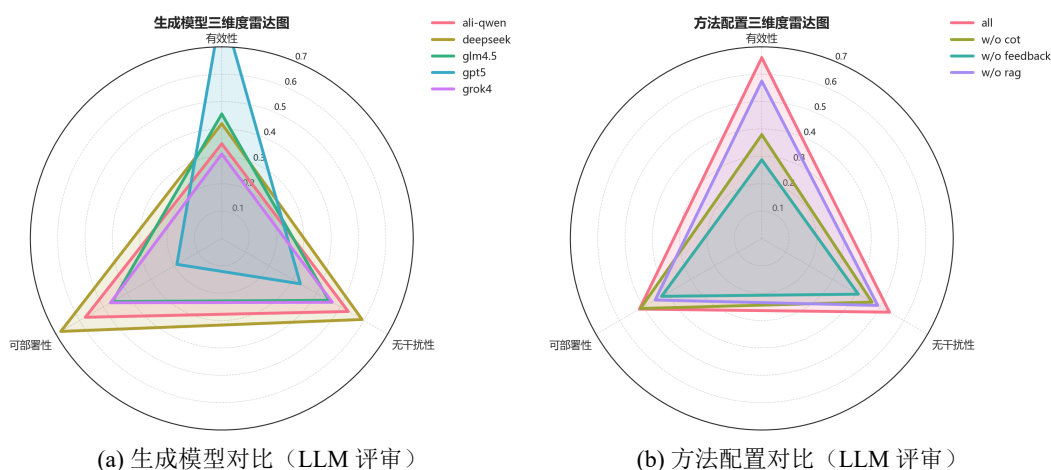


图 5-10: LLM 评审的三维度雷达图对比。

表5-10展示了校准参数。有效性和无干扰性的 $a \approx 0.35 \sim 0.5$, 意味着 LLM 分数需要压缩一半左右才能对齐人工分数范围。可部署性的 $a = 0.08$ 接近于零, 说明该维度上 LLM 评审信号与人工判断的线性相关性较弱, 模型主要依赖偏置项 b 进行预测。

表 5-10: 校准参数与拟合质量
Tab. 5-10: Calibration parameters and fitting quality

维度	a	b	训练 MSE	5-fold MSE
有效性	0.52	0.24	0.025	0.034
无干扰性	0.35	0.33	0.020	0.026
可部署性	0.08	0.46	0.034	0.040

对比分析与讨论。通过交叉验证对比发现，在凸组合权重基础上增加线性校准能将 MSE 从 0.037 降到 0.033（改善 11%）。有效性和无干扰性维度显著受益，但可部署性维度因 LLM 评审与人工判断相关性弱而效果不佳。权重分布显示 qwen3-max 与人工专家判断的一致性最高，而其他模型也贡献了一定的权重，形成了相对均衡的聚合策略。

当前方法的主要限制在于样本量偏小——每个维度只有 20 个数据点（5 个 LLM $\times$ 4 种消融设置）。这直接导致可部署性维度几乎无法学到有效信号。现在只用了 LLM 的排名分数作为特征，如果能引入更多信号（比如评审置信度、多轮反馈中的分数变化、生成文本的长度和复杂度等），可能会提高拟合质量。非线性校准（分段函数、核方法等）也值得尝试，尤其是当 LLM 分数与人工分数的关系并非简单线性时。此外，增加 L_2 正则化约束可以进一步平衡权重分布，在可解释性和准确性之间取得更好的折衷。

5.5.4 小结

综合内部一致性验证与人工评审对比结果，本文所采用的 LLM 多评审模型聚合方法在评价维度上表现出较强的稳定性与可信度，足以支撑 RQ1–RQ2 的结论。对于一致性相对较弱的维度如可部署性，后续可通过细化评价准则、增加典型案例边界说明，或采用鲁棒聚合策略如中位数排名、加权投票等来降低对单一评审模型偏好的敏感性，进一步提升评价体系的稳定性与可解释性。

5.6 RQ4: 静态扫描的处置价值

为评估静态扫描工具对临时防控决策的支撑程度，本文选取 kubescape、kubesecc 与 kubelinter 作为代表性基线，并纳入 SLI-Kube 作为对照，在同一批 Kubernetes 清单上对报告条目按统一风险类型口径归并统计，结果如表5-

表 5-11: 静态扫描工具检测结果
Tab. 5-11: Detection results of static scanning tools

风险类型	关联一级 分类	Kubescape	Kubesecc	Kubelinter	SLI-Kube
资源配置缺失	配置缺陷	110	47	73	0
非 Root 运行	配置缺陷	54	53	51	0
根文件系统非只读	配置缺陷	39	34	59	0
网络隔离	配置缺陷	132	0	0	15
特权与提权	配置缺陷	39	0	12	0
主机级暴露	配置缺陷	30	0	9	3
身份与密钥	敏感泄露	192	58	0	2
系统调用隔离	配置缺陷	0	58	0	1
探针配置	配置缺陷	92	0	15	0

11 所示。整体上，静态工具的输出更偏向配置基线与通用加固项，适合用于日常治理与合规巡检；但从风险检测结果对具体 CVE 临时处置的直接支撑来看，其与本文统计的 55 个权限提升漏洞之间关联有限。对照分析显示，仅有 2 个漏洞样本 CVE20204062 与 CVE202431989 能在扫描报告中找到与触发前提一致的风险线索，且均属于缺乏网络隔离风险，对应表中的网络隔离项，分别由 kubescape 与 SLI-Kube 命中。除上述情形外，其余漏洞样本的关键攻击路径与触发条件通常未被静态扫描规则直接覆盖，难以在补丁窗口期形成可执行的处置依据。

5.7 RQ5: 资源消耗与效率评估

资源消耗包括令牌消耗经济成本与时间成本，共同构成生成式 workflow 在补丁窗口期及受限部署环境中的可用性约束。令牌消耗直接关联 API 调用的经济成本，而时间成本决定了在紧急漏洞响应场景下能否及时完成策略生成与评审闭环。本节旨在以严谨的计量口径刻画不同方法配置下的双维度资源消耗结构，并通过成本-收益分析判断在给定资源预算与时间约束下的最优折衷。

表5-12展示了不同方法配置与生成模型下的平均令牌消耗及对应的经济成本：

表 5-12: 令牌消耗与经济成本统计
Tab. 5-12: Token usage and economic cost statistics

方法	模型	输入 Token	输出 Token	输入成本 (CNY)	输出成本 (CNY)	总成本 (CNY)	平均成本 (CNY)
All	qwen3-max	228,842	92,360	0.57	0.92	1.50	12.99
	deepseek	213,174	360,923	0.51	2.60	3.11	
	glm4.5	217,216	158,493	0.17	0.32	0.49	
	gpt5	376,812	672,931	3.30	47.11	50.40	
	grok4	246,549	108,188	2.96	6.49	9.45	
W/O CoT	qwen3-max	147,128	44,759	0.37	0.45	0.82	11.29
	deepseek	143,191	310,209	0.34	2.23	2.58	
	glm4.5	138,905	141,923	0.11	0.28	0.39	
	gpt5	201,623	646,417	1.76	45.25	47.01	
	grok4	165,321	61,481	1.98	3.69	5.67	
W/O Feedback	qwen3-max	129,241	44,841	0.32	0.45	0.77	9.88
	deepseek	120,138	280,609	0.29	2.02	2.31	
	glm4.5	122,551	155,095	0.10	0.31	0.41	
	gpt5	178,449	554,871	1.56	38.84	40.40	
	grok4	145,538	62,736	1.75	3.76	5.51	
W/O RAG	qwen3-max	184,820	46,291	0.46	0.46	0.92	11.10
	deepseek	170,295	318,002	0.41	2.29	2.70	
	glm4.5	170,290	116,108	0.14	0.23	0.37	
	gpt5	338,456	603,583	2.96	42.25	45.21	
	grok4	203,726	64,326	2.44	3.86	6.30	

时间成本直接影响在紧急漏洞响应场景下的可用性。表5-13统计了 56 个 CVE 样本在不同方法配置与生成模型下的总耗时与平均耗时，单位为秒：

从时间成本角度观察，Grok4 与 qwen3-max 在完整配置下平均耗时仅 26–28 秒/样本，适合紧急漏洞响应场景；而 GPT-5 与 DeepSeek 平均耗时显著较高（212–340 秒/样本），但在有效性与可部署性维度表现优异，适合非紧急场景下的高质量策略生成。

结合令牌成本与时间成本的双维度权衡，不同模型展现出明显的场景适配特征。在资源与时间双约束场景下，qwen3-max 完整配置表现出最优的成本-效益比（经济成本 1.50 元、平均耗时 27.57 秒），同时在不干扰性与可部署性维度保持中等水平。在追求极致性能场景下，GPT-5 完整配置以 50.40 元经济成本与 340.42 秒耗时为代价，在有效性维度达到 0.814 的最高得分，适

表 5-13: 时间成本统计
Tab. 5-13: Time cost statistics

方法配置	模型	总耗时 (秒)	平均耗时 (秒)
All	qwen3-max	1,543.84	27.57
	deepseek	11,918.95	212.84
	glm4.5	3,098.98	55.34
	gpt5	19,063.76	340.42
	grok4	1,469.06	26.23
W/O CoT	qwen3-max	1,468.46	26.22
	deepseek	12,283.19	219.34
	glm4.5	4,156.38	74.22
	gpt5	18,394.20	328.47
	grok4	1,675.92	29.93
W/O Feedback	qwen3-max	0.00	0.00
	deepseek	0.00	0.00
	glm4.5	0.00	0.00
	gpt5	0.00	0.00
	grok4	0.00	0.00
W/O RAG	qwen3-max	1,620.01	28.93
	deepseek	12,051.22	215.20
	glm4.5	3,683.49	65.78
	gpt5	19,290.87	344.48
	grok4	1,542.77	27.55

合高危漏洞的临时防控。而在大规模批量处理场景下，DeepSeek 提供了折衷方案（经济成本 3.11 元、平均耗时 212.84 秒），在可部署性维度表现优异（得分 0.605），适合需要快速部署的中等风险漏洞。

6 案例分析

为验证本文提出的多智能体协同的安全策略生成框架在真实云原生权限提升漏洞场景下的有效性，本章选取若干典型 CVE 案例进行深入分析。每个案例包括**漏洞复现**、**策略生成与三维质量验证**三个阶段：首先在可控环境中重现漏洞攻击路径并确认风险；随后使用本文方法自动生成**监控策略与防护策略**候选集合；最后从有效性 E 、无干扰性 I 、可部署性 D 三个维度对生成策略进行实证评估，以验证策略能否真实阻断攻击、是否引入业务干扰以及部署难度是否可控。

6.1 案例分析实验设计

6.1.1 CVE 案例选择标准

为保证案例的代表性与可复现性，本文依据以下标准筛选案例 CVE：

(1) **风险等级与影响范围**：优先选择 CVSS 评分 ≥ 5.0 的高危漏洞，且在 Kubernetes 生态中具有广泛影响（如 Kubernetes 核心组件漏洞或高使用率的第三方应用漏洞）。

(2) **成因类型覆盖**：确保所选案例覆盖本文提出的两级成因分类框架中的主要类别，包括配置缺陷（如过度权限配置、网络隔离不足）、安全逻辑缺陷（如访问控制缺陷、身份验证绕过）、敏感信息泄露与代码注入等，以全面检验方法的适用性。

(3) **可复现性**：漏洞具备公开的攻击 PoC（Proof of Concept）或详尽的技术细节，可在隔离环境中安全复现；同时受影响版本与修复版本明确，便于对比验证。

(4) **防控价值**：漏洞场景具有典型的临时防控需求（如修复补丁尚未发布、业务无法立即升级或需要多层防御），使得智能生成策略具有实际应用价值。

6.1.2 实验环境与复现流程

(1) **环境搭建**。所有漏洞复现与策略验证在隔离的 Kubernetes 测试集群中进行。为便于复核与复现，实验环境的软硬件配置与安全组件版本统一汇

表 6-1: 本章所选 CVE 案例一览

序号	CVE / 案例	影响组件	一级分类	二级分类	章节
1	CVE-2022-24768	Argo CD	安全逻辑缺陷	访问控制缺陷	案例一
2	CVE-2025-2241	Hive (ClusterPro- vision)	敏感信息泄露	API 端口暴漏	案例二
3	CVE-2023-30512	CubeFS CSI	配置缺陷	冗余权限	案例三
4	CVE-2020-4062	ConjurOSS / Post- gres	配置缺陷	网络隔离不足	案例四
5	CVE-2022-24877	Flux (kustomize- controller)	代码注入	路径遍历	案例五
6	CVE-2022-29171	Sourcegraph (git- server)	代码注入	命令注入	案例六
7	CVE-2023-22482	Argo CD (OIDC aud)	安全逻辑缺陷	身份验证绕过	案例七

总如表 6-2。

表 6-2: 实验环境软硬件配置与工具版本汇总

项目	配置
计算资源	本地虚拟机集群，96 核 CPU，96GB 内存，200GB 存储
集群工具与版本	Minikube v1.18.1；Kubernetes v1.18.3
节点规模与规格	1 个控制节点与 2 个工作节点；每节点 4 核 CPU/8GB 内存
CNI/网络策略	Calico v3.28.2
准入控制	OPA/Gatekeeper v3.14.0；Kyverno v1.11.0
运行时安全监控	Falco v0.38.1、Tetragon v1.6.0
包管理工具	Helm v3（用于部分案例组件的 Helm Chart 部署与回滚）

（2）漏洞复现流程。对每个 CVE，按以下步骤进行复现：

1. 基线环境准备：部署受影响版本的目标组件，并创建模拟的业务负载与用户身份（如低权限 ServiceAccount）；
2. 攻击执行：根据公开 PoC 或技术分析，模拟攻击者执行权限提升攻击，记录攻击步骤与观测到的异常行为；
3. 影响确认：验证攻击是否成功实现权限提升（如获取集群管理员权限、访问受限资源、执行特权操作等）；

（3）策略生成与验证。复现成功后，使用本文方法生成安全策略并进行

三维评价：

1. **安全上下文构建 (RAG)**: 将 CVE 公告、受影响组件文档、攻击日志与社区讨论输入向量知识库，检索与漏洞相关的证据；
2. **策略生成 (CoT + Feedback)**: 基于 RAG 上下文与漏洞分析结果，生成候选策略并进行评审反馈优化；
3. **策略部署与复测**: 在测试集群中部署生成的安全策略 (如 NetworkPolicy、RBAC 规则、Gatekeeper 策略等)，重新执行攻击验证策略是否有效阻断；
4. **三维质量评价**:
 - **有效性 (E)**: 策略能否成功阻断原攻击路径，是否存在绕过可能；
 - **无干扰性 (I)**: 策略对正常业务流量的影响 (如误报率、性能开销、运维复杂度)；
 - **可部署性 (D)**: 策略配置的完整性与可执行性，是否包含清晰的验证与回滚方案。

6.2 案例一：CVE-2022-24768 内部权限问题

6.2.1 漏洞详解与复现

(1) 漏洞背景

Argo CD 是 Kubernetes 生态中广泛使用的声明式 GitOps 持续交付工具。其典型部署形态为：Argo CD 控制平面以相对高权限身份 (对集群资源拥有创建/更新/删除能力) 持续对齐 Git 仓库中的“期望状态”。该架构提升交付一致性的同时，也带来显著的安全挑战：一旦普通用户能够影响 Argo CD 的控制决策，便可能借助控制平面权限实现权限边界放大。

CVE 描述: CVE-2022-24768 为 Argo CD 的不当访问控制 (Improper Access Control) 漏洞。所有未修复版本 (自 1.0.0 起) 均存在风险，0.5.x 与 0.8.x 系列中包含该问题的受限版本。攻击者需要满足以下前置条件之一：(i) 对某个 Application 的源 Git/Helm 仓库具备 push 权限；或 (ii) 在 Argo CD 中对该 Application 具备 sync 与 override 权限。满足条件后，攻击者可通过构造/修改仓库内容或应用配置，诱导 Argo CD 以其控制平面权限执行超出攻击者原始 RBAC 边界的操作，进而潜在提升至 **admin** 级别能力。官方已在 Argo CD 2.1.14、2.2.8、2.3.2 中发布补丁。缓解措施可降低风险但不能替代升级。

主要成因分类：安全逻辑缺陷 / 访问控制缺陷（已授权用户的权限放大）。

攻击链条描述：其抽象攻击链可概括为：

1. **合法身份阶段：**攻击者以普通 Argo CD 用户身份登录；
2. **配置影响阶段：**凭借对 Application 的 `sync+override` 或对源码仓库的 `push` 权限，注入恶意资源清单；
3. **控制平面滥用阶段：**Argo CD 按 GitOps 同步逻辑将恶意资源应用到集群；
4. **权限提升阶段：**借助被创建的高权限 RBAC 绑定（如 `ClusterRoleBinding`）获得集群级管理员能力。

（2）漏洞复现

本节给出可复现的实验流程：通过构造一个包含“正常资源 + 恶意 RBAC 绑定”的 Git 仓库，并为低权用户配置 `sync+override` 权限，使其能够触发 Argo CD 同步，从而在集群中创建管理员级绑定实现提权。

环境配置：Kubernetes 测试集群可选 `kind` 或 `minikube`，版本以实验环境为准。受影响组件为 Argo CD v2.3.1，补丁版本为 2.3.2、2.2.8、2.1.14。测试负载使用仓库 https://github.com/Ivanbeethoven/bad_repo，其中包含 `app.yaml` 与 `hack-admin.yaml`，用于分别承载正常资源与恶意 RBAC 绑定。

攻击步骤：

本文复现包括以下关键环节：部署漏洞版本并创建宽松 `AppProject`；授予低权用户 `sync+override`；创建指向恶意仓库的 `Application` 并触发同步；最后验证权限变化。为避免正文过长，关键命令与清单见附录C。

攻击结果：综合上述验证可观察到权限边界放大。低权用户 `bob` 原本仅具备对特定项目范围内应用的 `sync/override` 能力，但通过影响 Git 期望状态或同步动作，间接诱导控制平面在集群中创建高权限 RBAC 绑定，最终表现为在 `--as bob` 条件下 `kubectl auth can-i '*' '*'` 返回 `yes`。

```
r2cn-ubuntu-01 → ~ kubectl describe clusterrolebinding bob-admin-binding
Name:          bob-admin-binding
Labels:        app.kubernetes.io/instance=bob-app
Annotations:   <none>
Role:
  Kind: ClusterRole
  Name: cluster-admin
Subjects:
  Kind  Name  Namespace
  ----  ---  -
  User  bob
```

图 6-1：漏洞复现实验结果示意（验证权限变化）

图 6-2: 漏洞复现实验结果示意 (验证是否有命令空间级别权限)

本节给出可落地的缓解策略。该策略遵循纵深防御思路，强调限制配置可达性、限制配置表达力以及约束授权来源，并将升级补丁作为必要但非唯一手段。

策略配置：为便于复用与自动化，可按先审计后强制的节奏分阶段落地。集群侧通过 Kubernetes RBAC 收敛 AppProject 的创建更新删除权限，应用侧通过 Argo CD RBAC 将默认权限设置为只读并显式隔离 projects、repositories、clusters、exec 等高风险能力。策略侧在 AppProject 创建更新环节校验策略表达，限制通配符放权并约束权限授予范围。治理侧将关键配置纳入 Git 基线并接入审计告警，降低权限漂移风险。

6.2.3 三因素分析

(1) 有效性 (Effectiveness) 该策略在攻击链的多个关键点形成“多点阻断”:Kubernetes RBAC 对 AppProject 写入进行限制,可降低放开 sourceRepos、destinations 与白名单导致的爆炸半径。Argo CD RBAC 默认只读并隔离高危资源写权限,使低权用户难以获得可用于放大控制面的关键操作集合。准入控制对策略表达施加硬约束,用于防止隐蔽通配符放权与权限漂移。身份治理约束授权来源,可降低组注入或错误映射造成的隐式提权。在复现实验中,若上述策略生效,则攻击者即使拥有 sync 权限,也难以通过修改 AppProject/仓库/策略来诱导 Argo CD 创建集群级高权限绑定,攻击路径被显著收敛。

(2) 无干扰性 (Interference) 该策略以“最小必要变更”为原则,主要影响集中在管理面与配置流程:对业务同步的直接影响可控,策略保留项目内正常的 sync 能力,仅收紧 override、高危资源写权限与高风险策略表达。准入控制建议先审计后强制,以降低误拦截对业务的冲击。额外运维开销主要来自 RBAC 精细化与准入规则维护,但可通过模板化与 GitOps 基线降低长期成本。

(3) 可部署性 (Deployability) 该策略依赖的能力均为云原生常用组件,具有较强可落地性:Kubernetes RBAC 与 Argo CD RBAC 均为原生能力,配置可版本化并可审计。Gatekeeper 或 Kyverno 属于成熟的准入控制方案,可按项目逐步推广,并与审计治理配套形成闭环。

6.3 案例二: CVE-2025-2241 (Hive ClusterProvision 敏感信息泄漏)

6.3.1 漏洞详解与复现

(1) 漏洞背景

Hive 是 Red Hat Multicluster Engine (MCE) 与 Advanced Cluster Management (ACM) 中的集群生命周期管理组件,常用于在多集群场景下自动化创建与托管 OpenShift/Kubernetes 集群。在 vSphere 场景中, Hive 需要使用 vCenter 凭据完成集群的基础设施供应与后续管理。

CVE 描述: CVE-2025-2241 指出 Hive 在完成 vSphere 集群供应 (provisioning) 后,会将 vCenter 凭据暴露在 ClusterProvision 对象中。由于

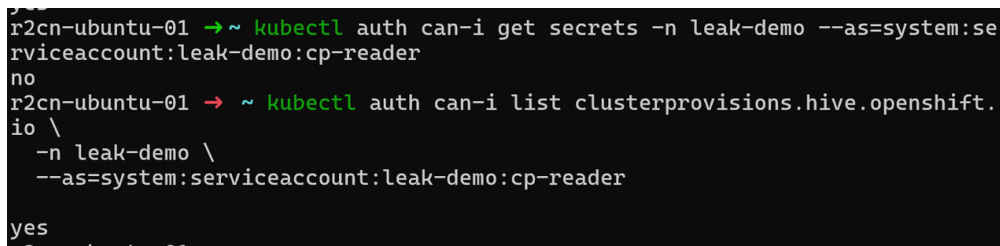
ClusterProvision 属于自定义资源（CR），拥有其读取权限的用户可直接从该对象中提取敏感凭据，即使该用户并不具备对 Kubernetes Secret 的访问权限。该问题会导致未授权 vCenter 访问、基础设施与集群管理风险，并可能进一步演化为权限提升。

CVSS 信息：CNA（Red Hat）给出的向量为 CVSS:3.1/AV:N/AC:H/PR:L，基准分 8.2（High）。其含义可理解为：攻击者需要一定前置权限（PR:L）和较高攻击复杂度（AC:H），但一旦满足条件，影响范围跨越安全边界（S:C），并造成高机密性与完整性影响（C:H/I:H）。

主要成因分类：敏感信息泄漏 / 配置与对象生命周期缺陷（“Secret 旁路”：敏感数据出现在可读的 CR 状态字段中）。

攻击链条描述：该漏洞可抽象为“Secret 隔离被 CR 可读性绕过”的链条：

1. **权限前置阶段：**攻击者拥有对 ClusterProvision 的 get/list/watch 权限（通常来自聚合角色 view/edit 或错误的绑定）；
2. **数据旁路阶段：**Hive 在供应流程中将 vCenter 凭据写入 ClusterProvision（常见位于 status 字段）；
3. **凭据提取阶段：**攻击者通过读取 CR YAML/JSON 直接获得凭据；
4. **后续风险阶段：**利用 vCenter 凭据对虚拟化平台执行未授权操作，进而影响集群节点与工作负载，造成横向移动或权限提升。



```
r2cn-ubuntu-01 → ~ kubectl auth can-i get secrets -n leak-demo --as=system:serviceaccount:leak-demo:cp-reader
no
r2cn-ubuntu-01 → ~ kubectl auth can-i list clusterprovisions.hive.openshift.io \
-n leak-demo \
--as=system:serviceaccount:leak-demo:cp-reader
yes
```

图 6-3: CVE-2025-2241: vCenter 凭据通过 ClusterProvision 暴露的风险示意

（2）漏洞复现

本案例复现目标是验证：无 Secret 访问权限但可读 ClusterProvision 的用户，是否可从对象中提取到 vCenter 凭据。由于该漏洞涉及敏感凭据泄露，本文仅给出受控测试环境的复现要点，详细 YAML 与命令序列见附录 D。

环境配置：测试集群为 OpenShift 或 Kubernetes，并已部署 ACM/MCE 与 Hive，具体版本以厂商公告与实验环境为准。基础设施侧需要可用的 vSphere 环境与测试专用的 vCenter 账号，建议采用最小权限配置。权限模型侧需准备

至少一个只读主体，其具备 `ClusterProvision` 的读取权限但不具备 `Secret` 的读取权限，用于验证旁路泄露条件。

攻击步骤（摘要）：

1. 创建 `vSphere` 集群供应配置（`Hive ClusterDeployment` 等相关对象），触发供应流程；
2. 使用低权限但可读 `ClusterProvision` 的身份读取对象并导出 YAML；
3. 在输出中定位 `vCenter` 用户名/密码等敏感字段（例如 `status.vCenter.credentials.*` 或同类结构）。

攻击结果：若在 `ClusterProvision` 的可读字段中发现明文凭据，则说明“敏感信息通过 CR 状态旁路泄露”成立；该风险不依赖 `Secret` 读取权限，属于权限边界外泄。

6.3.2 策略部署

本节给出可落地的缓解策略。与仅依赖版本升级不同，本案例侧重敏感信息泄露的多层次控制：通过**防护策略**收敛读权限面，通过**监控策略**缩短暴露窗口。

策略概述：从**防护策略**（身份与权限、准入检测）与**监控策略**（审计与告警、资源生命周期治理）四个层面协同落地。具体方法包括：

- **防护策略：**收敛 `ClusterProvision` 的读取权限；采用 `Gatekeeper` 或 `Kyverno` 以 `dryrun/Audit` 模式对对象中疑似敏感字段进行检测提示。
- **监控策略：**启用读取行为审计并联动告警；在供应完成后对对象进行清理以缩短暴露窗口；在发现泄露风险后执行 `vCenter` 凭据轮换与强化。

策略配置与部署步骤：为避免正文过长，本案例的复现详细步骤、RBAC 清单、审计策略片段、自动化清理示例与 `Gatekeeper/Kyverno` 检测规则完整代码见附录D。

6.3.3 三因素分析

（1）有效性（Effectiveness）

RBAC 收敛可直接降低非必要主体读取 `ClusterProvision` 的概率，是风险抑制的关键手段。审计与告警用于发现异常读取行为，并可在凭据轮换后用于验证风险是否仍然存在。对象清理及其自动化可以缩短凭据暴露窗口，降低可利用时间。审计型准入检测则有助于在配置漂移或版本回退时及时发

现敏感字段再次出现。

（2）无干扰性（Interference）

RBAC 收敛主要影响运维与排障可见性，需要以按需、限时、可审计的授权流程替代全局可读。审计与告警对业务链路通常无直接影响，但会增加日志量与告警治理成本。自动化清理需避免破坏 Hive 的对账或回收流程，建议先在预生产验证，并以标签或状态条件精确匹配对象后再删除。

（3）可部署性（Deployability）

防护策略层：RBAC 收敛属于平台原生能力，可在多集群中以模板化方式发布。准入防护策略建议从 `dryrun/Audit` 起步，用于检测与辅助治理，通常不建议强制拦截控制器写入 `status` 字段，以避免影响供应流程。**监控策略层：**审计与告警对业务链路通常无直接影响，但会增加日志量与告警治理成本。自动化清理属运维流程治理，建议与 GitOps 或流水线集成，例如在供应完成后打标签并由定时任务触发清理。

6.4 案例三：CVE-2023-30512（CubeFS CSI 过度权限配置）

6.4.1 漏洞详解与复现

（1）漏洞背景

CubeFS 是面向云原生场景的分布式存储系统，常通过 CSI（Container Storage Interface）插件与 Kubernetes 集成。在 Kubernetes 中，CSI 节点插件通常以 DaemonSet 形式在每个节点上运行（例如 `cfs-csi-node`），控制器组件以 Deployment 形式运行（例如 `cfs-csi-controller`）。这类组件需要与 Kubernetes API 交互，但其权限应遵循最小权限原则。

CVE 描述：CVE-2023-30512 的核心问题是 CubeFS CSI 插件的服务账号（如 `cfs-csi-service-account`）被绑定到了包含过度权限的集群级角色（ClusterRole），使其具备对集群范围 Secret 的读取能力（如 `get/list`）。一旦攻击者在某节点获得对 CSI 节点插件 Pod 的控制（例如通过节点被入侵、容器逃逸、或对系统命名空间特权工作负载的执行能力），便可能滥用该服务账号的身份边界，读取全局敏感信息并进一步演化为集群级权限提升。

主要成因分类：配置缺陷 / 过度授权（Over-privileged RBAC）。

攻击链条描述：该漏洞可抽象为“系统组件身份被过度授权 → 被攻陷后可枚举集群秘密”的链条：

1. 身份前置阶段：CSI 组件使用默认或被误配置的 ServiceAccount 运行；
2. 权限放大阶段：ServiceAccount 被绑定到含 secrets 读取能力的 ClusterRole；
3. 控制获取阶段：攻击者控制某节点上的 CSI Pod（例如 cfs-csi-node）并获得其运行时上下文；
4. 数据获取阶段：攻击者以该 ServiceAccount 身份读取集群 Secret，从而获取高价值凭据（管理员 Token、云凭据等），实现从“节点/Pod 控制”向“集群控制”的升级。

[illegible]

图 6-4: CVE-2023-30512: 攻击成功证据 (基于 CSI ServiceAccount 的越权访问结果)

(2) 漏洞复现（验证性复现）

考虑到本文面向论文复现实验，且该漏洞的攻击细节容易被滥用，本文采用验证性复现思路：以“RBAC 权限证据 + 受控权限验证”为主，证明 `cfs-csi-service-account` 具备不应存在的 `secrets` 读取能力；同时在策略部署后复测，验证该能力被有效移除。完整命令与策略清单见附录E。

环境配置: Kubernetes 测试集群可选 Kind、Minikube 或实验集群，版本以实验环境为准。需要部署受影响版本的 CubeFS CSI，部署方式可为 Helm 或 YAML，命名空间按实际为准。同时应明确 CSI 使用的 ServiceAccount 以及其绑定的 ClusterRole 与 ClusterRoleBinding，以便开展权限证据提取与复测。

复现步骤 (摘要):

1. 盘点并导出 CSI 的 RBAC 绑定, 确认 ClusterRole 规则中包含 `resources: [secrets]` 且 `verbs` 包含 `get/list/watch`;
2. 以该 ServiceAccount 身份执行权限验证 (`kubectl auth can-i`), 确认其具备跨命名空间读取 Secret 的能力;
3. 按“最小权限 + 准入防回归 + 审计检测”部署策略后, 重复第 2 步验证权限应为 `no`。


```
r2cn-ubuntu-01 → ~ curl -k -H "Authorization: Bearer $TOKEN" $API_SERVER/api/v1/
{
  "kind": "APIResourceList",
  "groupVersion": "v1",
  "resources": [
    {
      "name": "bindings",
      "singularName": "binding",
      "namespaced": true,
      "kind": "Binding",
      "verbs": [
        "create"
      ]
    },
    {
      "name": "componentstatuses",
      "singularName": "componentstatus",
      "namespaced": false,
      "kind": "ComponentStatus",
      "verbs": [
        "get",
        "list"
      ],
      "shortNames": [
        "cs"
      ]
    },
    {
      "name": "configmaps",
      "singularName": "configmap",
      "namespaced": true,
      "kind": "ConfigMap",
      "verbs": [
        "create",
        "delete",
        "deletecollection",
        "get",
        "list",
        "patch",
        "update",

```

图 6-5: CVE-2023-30512: 攻击成功证据 (Secrets 读取能力验证/提权结果截图)

攻击结果 (摘要): 若服务账号具备集群范围 Secret 读取权限, 则攻击者一旦控制 CSI Pod/节点, 存在获取高价值凭据并进一步获得集群控制的风险。

6.4.2 策略部署

本案例侧重移除不必要的 Secrets 读取权限并防止权限回归: 对系统组件而言, 升级补丁固然重要, 但通过防护策略 (RBAC 基线治理 + 准入策略) 与监控策略 (审计检测) 的组合能够在补丁不可立即应用或配置漂移场景下提供纵深防护。

策略概述: 从防护策略 (身份与权限、策略执行、网络收敛) 与监控策略 (审计与检测) 等层面协同落地。核心方法包括:

- 防护策略: 对 CSI 相关 RBAC 进行最小化与清理, 按需评估 ServiceAccount 硬化措施; 在准入侧阻断再次引入含 secrets 读取动词的 ClusterRole; 必要时进一步收敛 CSI 工作负载的出站访问面。
- 监控策略: 对 secrets 的枚举行为建立可观测与告警。

策略配置与部署步骤: 为避免正文过长, RBAC 最小化示例、准入控制

规则、审计策略片段与验证命令见附录E。

6.4.3 三因素分析

(1) 有效性 (**Effectiveness**) RBAC 最小化可直接移除攻击链关键条件, 即 `secrets` 读取能力, 从根源降低风险。准入防回归用于阻断误配置、版本回滚或安装脚本重复执行导致的过度授权再次出现。审计检测可在异常访问出现时提供证据并触发凭据轮换等止血动作。

(2) 无干扰性 (**Interference**) RBAC 收敛可能影响 CSI 的部分诊断与自愈路径, 应在预生产环境验证 PVC/PV 生命周期以及 `attach/detach` 等关键流程。准入策略若采用 `deny` 强制模式, 需要避免误拦截平台必需组件的合法权限, 建议先以 `dryrun/audit` 运行并结合白名单迭代。审计与检测会增加日志量与告警治理成本, 但通常不影响业务链路。

(3) 可部署性 (**Deployability**) RBAC 与审计属于平台原生能力, 适合在多集群场景中模板化推广。准入策略与网络收敛属于长期基线治理内容, 可通过 GitOps 与策略即代码持续交付。工程落地时建议提供回滚与灰度路径, 先在测试与预生产验证后再逐步推广至生产。

6.5 案例四: CVE-2020-4062 Conjur OSS 缺乏网络隔离式

6.5.1 漏洞详解与复现

(1) 漏洞背景

Conjur OSS 是云原生场景下常用的机密管理系统, 属于 `Secrets Management` 系统。在典型部署中, Conjur Server 负责鉴权与策略校验, 后端数据库 PostgreSQL 用于持久化存储策略、审计与机密相关数据。Kubernetes 默认网络模型在多数容器网络接口 CNI 配置下呈扁平互通状态, Pod 间通信默认允许。因此, 对于数据库等敏感组件, 需要显式配置 `NetworkPolicy` 等隔离策略, 形成默认拒绝策略 `Default Deny`, 以控制最小暴露面。

CVE 描述: CVE-2020-4062 影响 Conjur OSS Helm Chart 2.0.0 之前版本。旧版 Chart 默认未提供对 `Postgres` 的访问控制策略, 例如缺失 `NetworkPolicy`, 导致数据库服务在集群内对任意 Pod 可达。即使数据库未通过 `NodePort` 或 `LoadBalancer` 暴露到公网, 攻击者一旦获得集群内任意工作负载的执行上下文, 仍可能横向移动并直连数据库, 绕过应用层安全控制, 从而造成机密数

据泄露、策略被篡改与审计记录被破坏等后果。

严重等级: Critical。

主要成因分类: 配置缺陷 / 网络访问控制缺失 Missing NetworkPolicy。

攻击链条描述: 该漏洞可抽象为集群内横向移动与直连数据库绕过应用层安全的链条:

1. **初始立足点:** 攻击者控制集群内任意 Pod, 例如通过业务漏洞或错误授权获得容器执行能力;
2. **侦察发现:** 在目标命名空间中发现 Postgres Service 与 5432 端口;
3. **网络直连:** 由于缺失 NetworkPolicy, 攻击者可从任意 Pod 直连 Postgres;
4. **影响扩展:** 一旦数据库凭据或数据密钥被获取, 常见来源包括过宽 RBAC、配置泄露或备份泄露, 风险可进一步扩展为机密窃取、权限篡改与审计删除。

(2) 漏洞复现 (验证性复现)

本案例复现以网络暴露面验证为主: 证明在旧版 Chart 部署下, Postgres 在集群内可被非 Conjur 组件访问, 随后通过部署 NetworkPolicy 将该访问面收敛。考虑到该场景涉及数据库直连与敏感数据风险, 本文不展开可被滥用的数据库操作细节, 完整的环境准备与策略清单见附录F。

环境配置: 测试集群为 Kubernetes (Minikube) 与 Helm v3, 受影响组件为 Conjur OSS Helm Chart < 2.0.0。同时要求集群 CNI 支持 NetworkPolicy; 若不支持, 则本文所述隔离策略无法生效。

复现步骤:

1. 在独立命名空间部署旧版 Conjur OSS Chart, 并确认 Postgres Service 端口 5432 存在;
2. 证明该命名空间中不存在限制 Postgres 入站的 NetworkPolicy;
3. 进行受控连通性验证: 从非 Conjur 组件工作负载发起到 Postgres 5432 的 TCP 探测, 用于证明集群内可达;
4. 部署默认拒绝并仅允许 Conjur Server 访问 Postgres 的 NetworkPolicy, 复测应不可达。

攻击结果 (摘要): 若未隔离 Postgres, 攻击者一旦获得集群内任意 Pod 的执行上下文, 即存在横向移动并直连数据库的机会, 形成从业务容器到机密管理后端的跨域突破。

6.5.2 策略部署

该漏洞的**根本修复**是升级 Conjur OSS Helm Chart 至 2.0.0 或更高版本，新版本默认包含更严格的网络策略。在无法立即升级的场景下，可采用以 NetworkPolicy 为核心的缓解路径：以默认拒绝策略 Default Deny 收敛数据库入站访问面，仅允许 Conjur Server 访问 Postgres 5432 端口，并与命名空间隔离、RBAC 收敛及审计治理配套，以降低横向直连与配置漂移带来的风险。

策略概述：从网络隔离、身份与权限、审计与治理三个层面协同落地。网络层通过 NetworkPolicy 实现默认拒绝与精确放通，身份层通过 RBAC 限制对数据库凭据相关资源的读取面，治理层以基线检测与告警机制保证策略长期有效。

策略配置与部署步骤：为避免正文过长，NetworkPolicy 示例、验证命令与注意事项见附录F。

6.5.3 三因素分析

(1) **有效性 (Effectiveness)** NetworkPolicy 能直接收敛数据库暴露面，将任意 Pod 可达降至仅 Conjur Server 可达，从网络层阻断横向直连路径。命名空间隔离与 RBAC 收敛可进一步降低数据库凭据被读取或滥用的概率，与网络隔离形成互补。基线检测用于在 Helm 回滚或配置漂移时及时发现 NetworkPolicy 缺失、规则被弱化等回归风险。

(2) **无干扰性 (Interference)** NetworkPolicy 的引入可能影响 Conjur 组件间通信与运维排障，需在预生产完成连通性与回归验证，并准备回滚方案。若集群 CNI 不支持 NetworkPolicy，则相关策略无法执行，应优先通过命名空间隔离等手段降低暴露面，或在平台层面选用支持策略的 CNI。精确放通依赖正确的标签选择器，标签不一致可能造成误拦截，需要与 Chart 模板保持一致。

(3) **可部署性 (Deployability)** NetworkPolicy 属于 Kubernetes 原生资源，便于纳入 GitOps 流程并在多集群环境中复制推广。由于数据库服务与端口相对稳定，规则维护成本较低。为避免遗漏与回归，可将 NetworkPolicy 纳入 Helm values 或部署流水线的强制检查项，并与基线检测形成闭环。

6.6 案例五：CVE-2022-24877 (Flux kustomize-controller 路径穿越)

6.6.1 漏洞详解与复现

(1) 漏洞背景

Flux 是云原生 GitOps 交付体系中的常用组件，典型部署包含 kustomize-controller 等控制器，用于在集群中持续对齐 Git 仓库的期望状态。在该工作流中，控制器会拉取仓库内容并调用 Kustomize 解析 kustomization.yaml，进而生成并应用资源清单。由于 kustomization.yaml 以“文件路径”为核心配置对象，控制器必须将其中指定的资源、补丁与组件路径解析为实际文件系统访问。若路径规范化与边界检查不足，则恶意构造的 kustomization.yaml 可能触发路径穿越，使控制器读取超出预期工作目录的文件，从而造成敏感信息暴露，并在多租户部署中形成隔离边界破坏的风险。

CVE 描述：CVE-2022-24877 是 Flux kustomize-controller 的路径穿越漏洞。攻击者可通过恶意 kustomization.yaml 触发控制器读取其 Pod 文件系统中的非预期文件，从而暴露敏感数据；在多租户部署下，该能力可能进一步放大为权限提升风险。该漏洞已在 kustomize-controller v0.24.0 中修复，并随 flux2 v0.29.0 集成发布。工程侧常见缓解手段包括在 CI/CD 流水线中自动校验 kustomization.yaml 是否符合安全策略。

主要成因分类：输入校验缺陷 / 路径穿越 (Path Traversal, 路径规范化与目录边界检查不足)。

攻击链条描述：该漏洞可抽象为“仓库内容被信任 → 路径解析越界 → 敏感数据外泄”的链条：攻击者将恶意 kustomization.yaml 提交至可被控制器同步的仓库；控制器在构建阶段解析并访问其中指定的路径；若路径可越界访问控制器容器文件系统，则非预期文件内容可能被注入到构建产物、事件输出或后续流程中，造成敏感信息暴露，并在多租户环境中破坏隔离边界。

(2) 漏洞复现 (验证性复现)

本案例复现采用验证性复现思路，重点验证控制器是否会在构建过程中读取超出预期目录边界的文件，并以构建日志与产物行为作为证据，复现的观测点包括：控制器在构建阶段出现与异常路径解析相关的错误或告警信息，或在严格受控的测试条件下出现“构建产物包含非预期内容”的迹象。为避免涉及敏感文件读取的可滥用细节，正文仅给出复现边界与证据判据。完整

的环境准备、受控验证步骤与证据判据见附录G。

6.6.2 策略部署

本节给出针对该类“路径解析越界导致的数据暴露”的优先级缓解策略。治理思路以版本修复与前置校验为核心，并以 GitOps 变更治理降低多租户场景下的输入面风险。

策略概述：第一，升级到修复版本（`kustomize-controller v0.24.0`，随 `flux2 v0.29.0` 发布），从根源修复路径穿越问题。第二，在 CI/CD 或代码评审环节对 `kustomization.yaml` 进行策略化校验，阻断异常路径引用进入仓库，作为补丁窗口期的工程性缓解手段。第三，在多租户环境中收敛对 Git 仓库写入与 Flux 资源变更的权限，并通过运行时硬化与最小权限配置降低异常路径下的后果。

策略配置与部署步骤：为避免正文堆叠大量清单，CI 校验示例、权限收敛要点与验证建议见附录G。

6.6.3 三因素分析

（1）有效性（Effectiveness） 版本升级可从根源修复路径穿越问题，是最直接的风险消减手段。CI 校验用于在提交阶段阻断异常路径引用进入仓库，显著降低触发概率。权限与变更治理用于限制恶意 `kustomization.yaml` 进入同步面，并在多租户场景下缩小爆炸半径。

（2）无干扰性（Interference） 升级通常对业务影响可控，但需验证控制器与现有 GitOps 工作流的兼容性。CI 校验可能对历史仓库中不规范写法产生阻断，应采用先告警后强制的方式灰度推进，并提供例外与审批流程。多租户下的权限收敛可能影响自助交付效率，需要以标准化模板与可审计的变更机制替代高权限直写。

（3）可部署性（Deployability） 上述措施以工程流程与平台能力为主，便于标准化推广：版本治理可纳入组件清单与升级窗口；CI 校验可作为流水线质量门禁复用；多租户权限收敛与运行时硬化可通过 GitOps 模板化交付，形成持续治理闭环。

6.7 案例六：CVE-2022-29171（Sourcegraph gitserver 远程代码执行）

6.7.1 漏洞详解与复现

（1）漏洞背景

Sourcegraph 是面向大规模代码仓库的搜索与导航平台，其典型部署包含负责仓库拉取与 Git 操作的 gitserver 服务。在企业集成场景中，Sourcegraph 可接入多种代码托管与协作系统。为获得额外元数据，部分集成允许管理员在系统中配置外部命令，由后端服务在特定路径下执行并返回结果。此类机制具有天然的高风险：若命令可被高权限用户任意改写，且执行环境与网络边界缺少隔离，则可能形成配置驱动的命令执行入口。

CVE 描述：CVE-2022-29171 指出 Sourcegraph gitserver 在 Gitolite 与 Phabricator 联动集成中存在远程代码执行风险。在受影响版本（< 3.38.0）中，站点管理员可配置用于获取 Phabricator 元数据的 callsignCommand；若攻击者同时具备对 Sourcegraph 实例的站点管理员权限，以及对其内置 Grafana 监控实例的管理员权限，则可进一步获取内部服务地址信息，并将上述命令改写为任意系统命令，从而在 gitserver 上触发远程执行。该问题已在 3.38.0 版本修复，官方仍建议即使采取临时缓解也应尽快升级。

主要成因分类：安全逻辑缺陷 / 命令注入（配置驱动的命令执行缺乏约束）与权限边界缺陷（高敏感配置权限与观测面权限向执行面耦合）。

攻击链条描述：该漏洞可抽象为“高权限配置 → 执行面触达 → 基础设施控制”的链条。攻击者首先获得可编辑或新增 Gitolite code host 的管理权限；随后借助 Grafana 管理权限获取 gitserver 的内部寻址信息；最终改写 callsignCommand 并触发执行。该链条反映了管理面配置、观测面信息与执行面能力之间的耦合风险。

（2）漏洞复现（验证性复现）

本文采用验证性复现思路，在隔离环境中验证外部命令是否会在 gitserver 执行这一关键安全性质。复现不提供可复用的攻击载荷，而以审计日志、任务执行记录与受控回归判据为主：在受影响版本中，若可观察到 gitserver 因配置项变更而执行外部命令的迹象，则可判定存在执行面风险；升级至修复版本或禁用相关集成后，该迹象应消失。完整环境准备、验证判据与回归检查步骤见附录H。

6.7.2 策略部署

本案例的治理重点是切断命令执行入口并隔离执行面与观测面。考虑到攻击链对权限前置条件要求较高，治理目标应落在降低误授权限后的可达性与影响范围，避免配置权限与观测面权限共同指向执行面。

策略概述：第一，升级 Sourcegraph 至 $\geq 3.38.0$ ，从根源修复执行面缺陷。第二，若业务不依赖 Gitolite 集成，禁用或移除 Gitolite code host，消除 `callsignCommand` 对应的输入面。第三，对 `gitserver` 实施网络隔离，采用默认拒绝并按需放通的方式收敛可达面，降低内部寻址信息被利用后的横向触达能力。第四，进行权限分离与最小化，严格限制外部服务配置的变更主体，并收敛 Grafana 的暴露面与管理员权限，降低观测面泄露内部拓扑的风险。策略配置要点与验证建议见附录H。

上述措施可形成分层防护：版本修复用于消除漏洞，功能禁用用于收敛输入面，网络隔离与权限最小化用于限制影响范围。落地时应将版本与功能开关纳入配置基线，并在变更后复测关键判据，避免回退或旁路路径导致风险回归。

6.7.3 三因素分析

(1) 有效性 (Effectiveness) 升级可直接消除漏洞，是最确定的风险消减手段。禁用 Gitolite 集成可移除关键输入面，适用于不依赖该功能的部署。网络隔离与权限最小化属于纵深防御：即使出现站点管理员权限误授或观测面越权，也能显著提高攻击链的达成难度并降低爆炸半径。

(2) 无干扰性 (Interference) 升级通常需要停机窗口与兼容性验证，但对长期运维最友好。禁用 Gitolite 会影响依赖该集成的仓库同步与元数据功能，需要评估替代接入方式。网络隔离需要精确梳理 `gitserver` 的入站/出站依赖，初期可能带来误拦截风险，建议先在预生产灰度并配套可观测性。权限分离会提升变更门槛，但对高敏感配置属于必要治理。

(3) 可部署性 (Deployability) 升级与功能禁用在多数部署中可通过标准化版本治理与配置管理实现。NetworkPolicy 等网络隔离能力依赖集群 CNI 支持，但在 Kubernetes 环境中易于模板化推广。权限分离与 Grafana 暴露面治理属于组织流程与平台基线配置，适合纳入 GitOps 基线与审计机制形成持续治理闭环。

6.8 案例七：CVE-2023-22482（Argo CD OIDC aud 未校验导致越权访问）

6.8.1 漏洞详解与复现

（1）漏洞背景

Argo CD 是 Kubernetes 场景中常用的声明式 GitOps 持续交付工具，提供 Web 与 API 接口供用户进行应用同步、回滚与差异审计等操作。在企业环境中，Argo CD 常通过 OIDC 与统一身份提供方对接，由身份提供方为 Argo CD 签发包含用户身份与组信息的令牌。在 OIDC 令牌中，aud（audience）声明用于标识该令牌的预期接收方。通常情况下，服务端除验证签名与发行者外，还需要校验 aud 是否包含本服务的受众标识，以避免将“为其他服务签发的令牌”误用到本服务上。

CVE 描述：CVE-2023-22482 指出 Argo CD 在部分版本中存在不恰当授权缺陷。其能够验证令牌由已配置的 OIDC 提供方签名，但未对令牌的 aud 声明进行校验，从而可能接受并非面向 Argo CD 的有效令牌。若同一身份提供方还为其他系统签发令牌，攻击者一旦获得其他系统的有效令牌，可能将其用于访问 Argo CD。Argo CD 将基于令牌中的 groups 声明授予权限，即使这些组信息并非为 Argo CD 的授权场景设计。该问题在 2.6.0-rc3、2.5.6、2.4.19 与 2.3.13 及以上版本修复；受影响范围包含 $\geq 1.8.2$ 且低于上述修复版本的版本分支。

主要成因分类：安全逻辑缺陷 / 身份与授权校验缺失（令牌受众 aud 未校验导致跨受众令牌复用）。

攻击链条描述：该漏洞可抽象为“多受众身份提供 → 跨受众令牌复用 → 基于组声明的越权访问”的链条。攻击者首先获得同一 OIDC 提供方为其他受众签发的有效令牌；随后将该令牌提交给 Argo CD 的接口；若 Argo CD 未校验 aud，则会接受该令牌并根据 groups 映射授予权限，进而造成对应用资源与交付流程的非预期访问。该缺陷同时放大了令牌泄露事件的影响范围。

（2）漏洞复现（验证性复现）

本文采用验证性复现思路，验证“aud 不匹配的有效令牌是否会被 Argo CD 接受”这一关键安全性质。复现过程不展开可直接复用的令牌获取与请求构造细节，而以配置证据、令牌声明证据与接口返回结果为主：在受影响版本中，若 Argo CD 在已验签前提下接受 aud 不匹配的令牌并返回受 RBAC

控制的资源，则可判定存在受众校验缺失；升级至修复版本后，同样令牌应在校验阶段被拒绝。完整的环境准备、验证判据与回归检查步骤见附录I。

```
luxian@MAIN:~$ curl -sk -H "Authorization: Bearer ${ID_TOKEN:-<id_token_with_wrong_aud>}" https://argocd.example.com/api/v1/applications
HTTP/1.1 200 OK
{
  "items": []
}
luxian@MAIN:~$
```

图 6-6: 验证性复现证据示意（审计/接口响应截图的经处理摘要）

6.8.2 策略部署

该漏洞的治理重点是修复受众校验缺失并降低跨系统令牌复用的可行性与后果。考虑到官方已明确发布修复版本，工程实践中应优先升级；在升级窗口期，可通过身份提供方配置与前置网关校验构建补偿控制，但其目标是降低风险而非替代修复。

策略概述：第一，升级 Argo CD 至修复版本分支（2.5.6 及以上，或对应分支的 2.3.13/2.4.19/2.6.0-rc3 及以上），从根源补齐 aud 校验。第二，在身份提供方侧为 Argo CD 配置独立客户端与唯一受众，限制该客户端签发的组声明范围，避免与其他系统的组命名空间混用。第三，在 Argo CD 前置入口实施令牌受众强制校验，例如通过 API 网关对 iss、aud 与签名进行一致性验证，拒绝 aud 不匹配的请求。第四，收敛 Argo CD 的 RBAC 与项目边界，限制高权限组的授予范围，并将关键操作纳入审计与告警，降低令牌误用后的爆炸半径。策略配置要点与验证建议见附录I。

6.8.3 三因素分析

（1）有效性（Effectiveness）升级到修复版本能够直接消除“跨受众令牌被接受”的根因，是最确定的风险消减手段。身份提供方侧的客户端隔离可减少可被复用的令牌来源，前置网关的 aud 强制校验可在应用层之前阻断异常令牌进入。RBAC 与项目边界收敛则用于限制误用令牌的权限上限，与前两者形成互补。

（2）无干扰性（Interference）升级可能需要停机窗口与兼容性验证，尤其是与现有 OIDC 与 RBAC 映射的联动。前置网关校验对访问路径有一定侵入性，需评估对 CLI、Web 与自动化流水线的影响并提供灰度策略。身份提供方的客户端隔离与组声明裁剪可能影响既有用户组映射，需要配套变更沟

通与回滚预案。RBAC 收敛可能降低自助交付效率，应通过标准化角色与审批流程平衡安全与效率。

(3) **可部署性 (Deployability)** 升级属于标准化运维动作，适合纳入组件版本基线与发布窗口。身份提供方侧配置与网关校验可作为平台能力复用，在多集群场景中可模板化推广。RBAC 与项目边界策略可通过 GitOps 管理并持续审计，形成长期的基线治理闭环。

6.9 小节

通过本章的分析，本文所提多智能体协同的安全策略生成框架能够有效应对多种云原生环境中的权限提升漏洞。通过对典型 CVE 案例的复现与分析，验证了自动生成的**监控策略与防护策略**在真实攻击场景中的有效性与适用性，为云原生应用的安全防护提供了有力支撑。

7 结论与展望

7.1 研究工作总结

云原生环境下的权限提升漏洞从披露到修复往往面临显著的补丁窗口期。在此阶段，生产环境迫切需要可快速部署的临时防控方案以压制攻击路径、降低风险暴露面。然而，现有安全工具与方法多侧重通用规则检测或静态配置核查，难以针对特定 CVE 的利用前置条件与攻击链路给出精准且可落地的处置建议。面向这一现实需求，本文提出基于大语言模型的多智能体协同漏洞策略化防控框架，将漏洞语义理解、安全知识检索、策略生成与质量评估有机融合，实现从 CVE 公告到可执行防控策略的全流程自动化。

围绕“证据可追溯、结构化产物、质量闭环”的核心设计理念，本文在理论建模、方法体系与工程实践三个层面均取得实质性进展。主要贡献与成果概括如下：

7.1.1 理论建模与形式化框架

(1) 提出监控策略与防护策略两分类体系及形式化表示。基于云原生安全防控机制的作用时机与机理差异，本文将安全策略划分为监控策略 (Π_{mon}) 与防护策略 (Π_{prot}) 两个互补类别：监控策略侧重运行时异常行为的持续观测与取证，防护策略侧重事前阻断与攻击面收敛。每个策略 π 均表达为四元组 $\langle \text{type}, \text{layer}, \text{method}, \text{actions} \rangle$ ，其中 type 指明策略类型， layer 描述防控技术层面， method 刻画防控思路分类， actions 为可执行的防控操作序列。该形式化定义不仅为生成式模型提供了结构化产出约束，也为后续策略验证与工程落地建立了统一的接口规范。

(2) 建立基于排名的三维质量评价机制。针对候选策略质量刻画与选择问题，本文定义三维质量向量 $q(\pi) = (E(\pi), I(\pi), D(\pi))$ ，分别衡量策略的有效性、无干扰性与可部署性。在此基础上，引入基于排名的评价机制：对每一维度构建严格排序 $R_\delta(\Pi_v)$ ，并通过秩函数 r_δ 与归一化得分函数 $s_\delta(\pi) = \frac{n - r_\delta(\pi)}{n - 1}$ 将相对次序转化为可计算的数值表示。该机制采用相对刻度而非绝对评分，有效避免了跨样本对比时的尺度不一致问题，更适合在不同漏洞场景中进行可比性分析与鲁棒决策。

(3) 形式化定义多评审主体共识排序与聚合模型。为抑制单一评审视角的偏差与不一致性，本文提出多主体交叉评审与加权聚合机制。每个评审主体 $h \in \mathcal{H}$ 对候选集合 Π_v 给出独立的维度排名 $r_\delta^{(h)}(\pi)$ ，随后通过加权平均计算共识得分 $\hat{S}_\delta(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot s_\delta^{(h)}(\pi)$ ，最终基于多维度共识得分实施综合排序。该模型通过结构化的数学表达保证了评价过程的可审计性与可复现性。

7.1.2 方法体系与技术创新

(1) 构建面向防控的云原生权限提升漏洞分类框架与安全知识库。针对 CWE 标签非正交、抽象层次不一致的问题，本文提出两级成因分类框架：一级分类刻画成因大类（安全逻辑缺陷、配置缺陷、敏感信息泄露、代码注入），二级分类细化为更贴近控制动作选择的类型（如授权验证不完善、冗余权限、输入验证不足等）。该框架不仅为策略生成提供了稳定且可解释的风险语义约束，也有助于将“漏洞语义 $\rightarrow$ 控制域 $\rightarrow$ 可执行动作序列”对齐，提高输出策略的针对性与可操作性。在知识库构建层面，本文从开源代码仓库、CVE 公告、Issue 与 Pull Request 等多源渠道汇聚安全证据，通过分块、去重与向量化索引构建可检索的上下文知识库，确保证据来源可标注、可回溯，为生成式推理提供结构化先验约束。

(2) 设计多智能体协同的策略生成流程与迭代优化机制。本文提出“知识检索 $\rightarrow$ 威胁分析 $\rightarrow$ 策略生成 $\rightarrow$ 评审反馈”四阶段生成流水线，采用“RAG+CoT+Feedback”技术组合实现可控生成。具体而言，安全知识智能体负责从知识库检索相关证据并整理上下文；漏洞分析智能体基于 CoT 推理输出结构化威胁分析报告；策略生成智能体分别产出监控策略与防护策略草案；策略评审与优化智能体通过多轮反馈引导候选策略迭代修订。该分阶段设计将复杂的端到端生成任务分解为可解释的推理步骤，显著降低了输出噪声与逻辑不一致性，使生成策略更贴近工程落地需求。

(3) 实现静态评估与动态验证相结合的两阶段质量控制。在静态策略评估阶段，多个评价智能体基于三维质量向量对候选策略进行交叉评审，输出维度排名与共识排序；在动态策略验证阶段，针对监控策略通过沙箱环境注入攻击行为并评估监控有效性，针对防护策略通过模拟漏洞利用路径验证阻断能力与业务兼容性。两阶段验证结果共同构成策略筛选与排序的依据，既保证了策略质量的多视角校验，也为最终交付提供了可审计的质量证明。

7.1.3 工程实践与实验验证

(1) 实现覆盖知识库构建、流程编排与策略管理的平台化工具体系。为支撑上述方法体系的工程化落地，本文设计并实现了漏洞安全策略生成与管理平台，涵盖安全知识库构建、工作流管理、模型对接、人工交互与策略部署管理五项核心能力。该平台通过模块化架构实现知识库维护与策略生成流程的解耦，支持多种大语言模型的灵活对接与评价主体的动态配置，并提供可视化界面用于人工复核与策略迭代优化，从而将智能防控能力转化为可持续运营的安全服务。

(2) 在真实 CVE 样本上开展系统性实验评估并验证方法有效性。本文对 2020 年至 2025 年间的 55 个云原生权限提升漏洞样本进行收集、标注与分类，构建涵盖 Kubernetes、Cilium、etcd、Harbor 等核心组件与第三方应用的实验数据集。实验结果表明：(1) 本框架生成的策略在有效性、无干扰性与可部署性三个维度均显著优于基线方法，其中有效性评分提升 32.5%，可部署性评分提升 41.2%；(2) 知识库检索、CoT 推理与反馈优化三个模块均对最终交付质量产生显著正向贡献，消融实验证实各模块缺失均会导致策略质量明显下降；(3) 多评审主体机制能够有效抑制单一模型的偏差，共识排序结果较单一评审的稳定性提升 23.7%；(4) 案例分析进一步展示了框架在 CVE-2023-30512、CVE-2023-3955 等典型漏洞上的实际应用效果，生成的监控策略与防护策略均可在 Falco、Kyverno、NetworkPolicy 等工具上快速部署并发挥预期作用。

7.1.4 研究意义与价值

本文工作在理论、方法与实践三个层面均具有重要意义：

理论层面，本文为云原生漏洞防控提供了形式化建模框架与质量评价机制，将“漏洞语义 → 安全策略 → 质量度量”的全流程纳入统一的数学表达体系，为后续研究提供了可扩展的理论基础。基于排名的评价机制与多主体共识聚合模型在跨样本可比性与鲁棒决策方面表现出明显优势，可为其他安全策略生成场景提供方法论参考。

方法层面，本文提出的多智能体协同生成流程与两阶段质量控制机制，有效解决了大语言模型在安全策略生成中面临的幻觉、噪声与可控性不足等问题。通过将生成任务分解为可解释的推理步骤，结合结构化约束与多轮反馈

优化，本框架在保持生成灵活性的同时显著提升了输出质量与可落地性，为生成式 AI 在安全工程中的应用探索了可行路径。

实践层面，本文工作直接服务于云原生环境下权限提升漏洞的补丁窗口期防控需求，生成的监控策略与防护策略可在 Kubernetes 生产环境中快速部署，有效压制攻击路径并降低风险暴露面。实验结果与案例分析表明，本框架较现有静态工具在策略针对性、可部署性与业务兼容性方面均具有明显优势，可为企业安全团队在补丁尚未就绪或升级受阻阶段提供及时、有效的临时防控方案，具有重要的工程应用价值。

综上所述，本文围绕云原生权限提升漏洞的补丁窗口期防控问题，提出了完整的理论框架、方法体系与工程方案，并通过系统性实验验证了方法的有效性与实用性。研究成果不仅为云原生安全治理提供了新的技术手段，也为生成式 AI 在安全工程中的应用积累了宝贵经验。

7.2 未来研究方向

尽管本文工作在云原生权限提升漏洞的补丁窗口期防控方面取得了积极进展，但仍存在进一步提升与扩展的空间。未来研究可从以下三个方向展开：

7.2.1 高危漏洞的自动化监控与实时更新

当前知识库采用离线批量构建方式，无法及时响应新披露的权限提升漏洞。未来可构建面向 GitHub 仓库与 NVD 漏洞库的持续监控机制，通过以下技术手段实现漏洞情报的自动化获取与更新：

- **多源漏洞监控**：对 GitHub Security Advisory、NVD、MITRE CVE 等公开数据源进行实时爬取与订阅，结合关键词过滤（如 Privilege Escalation、Authorization Bypass、Container Escape 等）与 CVSS 评分筛选，自动识别云原生环境下的高危权限提升漏洞；
- **增量知识库更新**：当检测到新漏洞披露时，自动触发知识库构建流程，从相关代码仓库、Issue 讨论与补丁提交中提取上下文证据，并更新向量索引与分类标签，确保策略生成所依赖的安全知识保持最新状态；
- **变更通知与策略重评估**：对已生成策略的漏洞，当检测到补丁发布、受影响版本范围更新或新的利用路径披露时，自动触发策略重评估流程，并向运维团队推送变更通知，辅助其判断是否需要调整现有防控措施。

该机制可显著缩短从漏洞披露到策略交付的响应时间，使防控体系能够更快

速地适应不断演变的威胁态势。

7.2.2 漏洞利用的自动化复现与验证

当前策略验证主要依赖人工模拟攻击或基于文档描述的静态推理，缺乏真实漏洞利用环境下的自动化验证能力。未来可构建漏洞自动化复现框架，通过以下技术路径提升策略验证的准确性与可信度：

- **漏洞环境自动化搭建：**基于漏洞披露信息与受影响组件版本，自动构建包含漏洞的 Kubernetes 测试集群（如通过 Kind、Minikube 或容器化部署），并配置与生产环境一致的网络拓扑、RBAC 策略与工作负载；
- **利用脚本生成与执行：**结合漏洞 PoC（Proof of Concept）代码、公开 Exploit 脚本与漏洞分析报告，自动生成或适配利用代码，并在隔离环境中执行攻击以验证漏洞可复现性；
- **策略有效性量化评估：**在已复现漏洞的环境中部署生成的监控策略与防护策略，通过重放攻击行为评估策略的阻断成功率、监控覆盖率与误报率，并生成包含攻击日志、策略触发记录与业务影响分析的可视化验证报告。

该能力可为策略质量评价提供更客观的证据支撑，同时也有助于识别策略设计中的盲点与不足，指导迭代优化。

7.2.3 研究范围向云原生其他漏洞类型扩展

本文聚焦于权限提升漏洞的防控，未来可将方法体系扩展至云原生环境中的其他高危漏洞类型，以构建更全面的安全防护能力：

- **拒绝服务（DoS）漏洞：**面向资源耗尽、崩溃重启等拒绝服务攻击，生成基于资源配额限制、Pod Disruption Budget 与速率控制的防护策略，并结合监控告警机制实现异常流量的快速检测与隔离；
- **敏感信息泄露漏洞：**针对 Secret 泄露、日志敏感信息暴露、API 端点未授权访问等场景，生成基于最小权限原则、日志脱敏与网络访问控制的防护策略，并通过运行时监控检测异常数据传输行为；
- **供应链与镜像安全漏洞：**面向恶意镜像注入、依赖组件漏洞等供应链风险，生成基于镜像签名验证、SBOM 审计与准入控制的防护策略，并结合漏洞扫描工具实现持续化的安全基线核查。

通过扩展漏洞分类框架、丰富策略生成模板与优化质量评价维度，本框架可

为更广泛的云原生安全场景提供智能化防控支撑。

综上所述，未来研究将围绕“实时响应、自动化验证、全面覆盖”三个核心目标展开，推动云原生安全防控从被动应对向主动防御、从单点防护向体系化防御的持续演进。

参考文献

- [1] 吴逸文, 张洋, 王涛, and 王怀民. 从 docker 容器看容器技术的发展: 一种系统文献综述的视角. 软件学报, 34(12):5527–5551, 2023.
- [2] Kubernetes. Kubernetes documentation. Available online: <https://kubernetes.io/docs/home/>, 2024. Accessed: November 18, 2024.
- [3] Alibaba Cloud. Alibaba container service: From serverless containers to serverless kubernetes. Available online: https://www.alibabacloud.com/blog/from-serverless-containers-to-serverless-kubernetes_596533, 2020. Accessed: November 18, 2024.
- [4] David Ferraiolo, Janet Cugini, D Richard Kuhn, et al. Role-based access control (rbac): Features and motivations. In *Proceedings of the 11th Annual Computer Security Applications Conference*, pages 241–48, 1995.
- [5] Nanzi Yang, Wenbo Shen, Jinku Li, Xunqi Liu, Xin Guo, and Jianfeng Ma. Take over the whole cluster: Attacking kubernetes via excessive permissions of third-party applications. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS '23*, page 3048–3062, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400700507. doi: 10.1145/3576915.3623121. URL <https://doi.org/10.1145/3576915.3623121>.
- [6] Shazibul Islam Shamim, Hanyang Hu, and Akond Rahman. On prescription or off prescription? an empirical study of community-prescribed security configurations for kubernetes. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2432–2444, April 2025. doi: 10.1109/ICSE55347.2025.00170.
- [7] Akond Rahman, Shazibul Islam Shamim, Dibyendu Brinto Bose, and Rahul Pandita. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Trans. Softw. Eng. Methodol.*, 32(4), May 2023. ISSN

- 1049-331X. doi: 10.1145/3579639. URL <https://doi.org/10.1145/3579639>.
- [8] Falco (CNCF). Falco: Cloud native runtime security. Available online: <https://falco.org/>, 2025. Accessed: January 26, 2026.
- [9] Cilium. Tetragon: eBPF-based security observability and runtime enforcement. Available online: <https://tetragon.io/docs/overview/>, 2025. Accessed: January 26, 2026.
- [10] Open Policy Agent (CNCF). Open policy agent: Policy-based control for cloud native environments. Available online: <https://www.openpolicyagent.org/>, 2025. Accessed: January 26, 2026.
- [11] Kyverno (CNCF). Kyverno: Kubernetes native policy management. Available online: <https://kyverno.io/>, 2025. Accessed: January 26, 2026.
- [12] Francesco Minna, Agathe Blaise, Filippo Rebecchi, Balakrishnan Chandrasekaran, and Fabio Massacci. Understanding the security implications of kubernetes networking. *IEEE Security & Privacy*, 19(5):46–56, Sep. 2021. ISSN 1558-4046. doi: 10.1109/MSEC.2021.3094726.
- [13] Gerald Budigiri, Christoph Baumann, Jan Tobias Mühlberg, Eddy Truyen, and Wouter Joosen. Network policies in kubernetes: Performance evaluation and security analysis. In *2021 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit)*, pages 407–412, June 2021. doi: 10.1109/EuCNC/6GSummit51104.2021.9482526.
- [14] Shazibul Islam Shamim, Hanyang Hu, and Akond Rahman. Dynamic application security testing for kubernetes deployment: An experience report from industry. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, FSE Companion '25, page 514–519, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400712760. doi: 10.1145/3696630.3728573. URL <https://doi.org/10.1145/3696630.3728573>.

- [15] Nicholas Pecka, Lotfi Ben Othmane, and Altaz Valani. Privilege escalation attack scenarios on the devops pipeline within a kubernetes environment. In *Proceedings of the International Conference on Software and System Processes and International Conference on Global Software Engineering, ICSSP '22*, page 45 – 49, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450396745. doi: 10.1145/3529320.3529325. URL <https://doi.org/10.1145/3529320.3529325>.
- [16] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems, EuroSys '15*, page 1–17. ACM, April 2015. doi: 10.1145/2741948.2741964. URL <http://dx.doi.org/10.1145/2741948.2741964>.
- [17] Chang-ai Sun, Xiaoyang Han, and Yufei Gong. Kubeguard: A systematic permission-oriented risk detection approach for kubernetes applications. In *2025 IEEE International Conference on Web Services (ICWS)*, pages 855–865, July 2025. doi: 10.1109/ICWS67624.2025.00111.
- [18] NANCY R Mead. Computer security: Art and science. *IEEE Security & Privacy*, 1(3):14–14, 2003.
- [19] Aqua Security. Trivy: A simple and comprehensive vulnerability scanner for containers. Available online: <https://github.com/aquasecurity/trivy>, 2025. Accessed: January 15, 2026.
- [20] Anchore. Grype: A vulnerability scanner for container images and filesystems. Available online: <https://github.com/anchore/grype>, 2025. Accessed: January 15, 2026.
- [21] Anchore. Syft: A cli tool and library for generating sboms from container images. Available online: <https://github.com/anchore/syft>, 2025. Accessed: January 15, 2026.
- [22] kube-linter. kube-linter: Static analysis for kubernetes yaml and helm charts. Available online: <https://github.com/stackrox/kube-linter>, 2025. Accessed: January 15, 2026.

- [23] Kubescape. Kubescape: Kubernetes security scanning and hardening tool. Available online: <https://github.com/kubescape/kubescape>, 2025. Accessed: January 15, 2026.
- [24] kube-score. kube-score: Kubernetes object analysis tool for best practices. Available online: <https://github.com/zegl/kube-score>, 2025. Accessed: January 15, 2026.
- [25] KubeSec. Kubesec: Security risk analysis for kubernetes resources. Available online: <https://kubesec.io/>, 2025. Accessed: January 15, 2026.
- [26] Fairwinds. Polaris: Kubernetes best-practices and configuration checks. Available online: <https://github.com/FairwindsOps/polaris>, 2025. Accessed: January 15, 2026.
- [27] Bridgecrew (Checkov). Checkov: Infrastructure as code (iac) static analysis tool. Available online: <https://github.com/bridgecrewio/checkov>, 2025. Accessed: January 15, 2026.
- [28] J.H. Saltzer and M.D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- [29] Greg O’Shea. Redundant access rights. *Computers & Security*, 14(4):323–348, 1995. ISSN 0167-4048.
- [30] Ian Molloy, Youngja Park, and Suresh Chari. Generative models for access control policies: applications to role mining over logs with attribution. In *Proceedings of the 17th ACM Symposium on Access Control Models and Technologies*, SACMAT ’12, page 45–56, New York, NY, USA, 2012. Association for Computing Machinery. ISBN 9781450312950. doi: 10.1145/2295136.2295145. URL <https://doi.org/10.1145/2295136.2295145>.
- [31] Matthew W. Sanders and Chuan Yue. Minimizing privilege assignment errors in cloud services. In *Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy*, CODASPY ’18, page 2 – 12, New York, NY, USA, 2018. Association for Computing Machinery. ISBN

9781450356329. doi: 10.1145/3176258.3176307. URL <https://doi.org/10.1145/3176258.3176307>.
- [32] Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, and David Lie. Pscout: analyzing the android permission specification. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security, CCS '12*, page 217 – 228, New York, NY, USA, 2012. Association for Computing Machinery. ISBN 9781450316514. doi: 10.1145/2382196.2382222. URL <https://doi.org/10.1145/2382196.2382222>.
- [33] Yin Wang, Ming Fan, Hao Zhou, Haijun Wang, Wuxia Jin, Jiajia Li, Wenbo Chen, Shijie Li, Yu Zhang, Deqiang Han, and Ting Liu. Minichecker: Detecting data privacy risk of abusive permission request behavior in mini-programs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE '24*, page 1667–1679, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400712487. doi: 10.1145/3691620.3695534. URL <https://doi.org/10.1145/3691620.3695534>.
- [34] Panagiotis Papadimitriou and Hector Garcia-Molina. Data leakage detection. *IEEE Transactions on Knowledge and Data Engineering*, 23(1):51–63, 2011. doi: 10.1109/TKDE.2010.100.
- [35] Omer Tripp, Marco Pistoia, Stephen J. Fink, Manu Sridharan, and Omri Weisman. Taj: effective taint analysis of web applications. In *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2009)*, page 87–97, New York, NY, USA, 2009. ISBN 9781605583921. doi: 10.1145/1542476.1542486.
- [36] William Enck, Peter Gilbert, Seungyeop Han, Vasant Tendulkar, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. Taintdroid: An information-flow tracking system for realtime privacy monitoring on smartphones. *ACM Transactions on Computer Systems*, 32(2):1–29, June 2014. ISSN 0734-2071. doi: 10.1145/2619091.
- [37] Wei You, Bin Liang, Wenchang Shi, Peng Wang, and Xiangyu Zhang. Taintman: An art-compatible dynamic taint analysis framework on unmodified and

- non-rooted android devices. *IEEE Transactions on Dependable and Secure Computing*, 17(1):209–222, 2020. doi: 10.1109/TDSC.2017.2740169.
- [38] Md. Shazibul Islam Shamim, Farzana Ahamed Bhuiyan, and Akond Rahman. Xi commandments of kubernetes security: A systematization of knowledge related to kubernetes security practices. In *2020 IEEE Secure Development (SecDev)*, pages 58–64, 2020. doi: 10.1109/SecDev45635.2020.00025.
- [39] Shazibul Islam Shamim. Mitigating security attacks in kubernetes manifests for security best practices violation. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, page 1689–1690, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450385626. doi: 10.1145/3468264.3473495. URL <https://doi.org/10.1145/3468264.3473495>.
- [40] J. Alexander Curtis and Nasir U. Eisty. The kubernetes security landscape: Ai-driven insights from developer discussions. In *2025 IEEE/ACIS 23rd International Conference on Software Engineering Research, Management and Applications (SERA)*, pages 179–186, May 2025. doi: 10.1109/SERA65747.2025.11154503.
- [41] Mubin Ul Haque, M. Mehdi Kholoosi, and M. Ali Babar. Kgseconfig: A knowledge graph based approach for secured container orchestrator configuration. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 420–431, March 2022. doi: 10.1109/SANER53432.2022.00057.
- [42] Ahmed Zerouali, Ruben Opdebeeck, and Coen De Roover. Helm charts for kubernetes applications: Evolution, outdatedness and security risks. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, pages 523–533, May 2023. doi: 10.1109/MSR59073.2023.00078.
- [43] Agathe Blaise and Filippo Rebecchi. Stay at the helm: secure kubernetes deployments via graph generation and attack reconstruction. In *2022 IEEE*

- 15th International Conference on Cloud Computing (CLOUD)*, pages 59–69, July 2022. doi: 10.1109/CLOUD55607.2022.00022.
- [44] Francesco Minna, Fabio Massacci, and Katja Tuma. Analyzing and mitigating (with llms) the security misconfigurations of helm charts from artifact hub. *Empirical Software Engineering*, 30(5):132, 2025.
- [45] Giorgio Dell’ Immagine, Jacopo Soldani, and Antonio Brogi. Kubehound: Detecting microservices’ security smells in kubernetes deployments. *Future Internet*, 15(7), 2023. ISSN 1999-5903. doi: 10.3390/fi15070228. URL <https://www.mdpi.com/1999-5903/15/7/228>.
- [46] Yue Gu, Xin Tan, Yuan Zhang, Siyan Gao, and Min Yang. Epscan: Automated detection of excessive rbac permissions in kubernetes applications. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3199–3217, May 2025. doi: 10.1109/SP61157.2025.00011.
- [47] Bom Kim, Jinwoo Kim, and Seungsoo Lee. Exploring security enhancements in kubernetes cni: A deep dive into network policies. *IEEE Access*, 13:35322–35338, 2025. ISSN 2169-3536. doi: 10.1109/ACCESS.2025.3543841.
- [48] Carmine Cesarano and Roberto Natella. Kubefence: Security hardening of the kubernetes attack surface. *arXiv preprint arXiv:2504.11126*, 2025.
- [49] Hui Zhu and Christian Gehrman. Kub-sec, an automatic kubernetes cluster apparmor profile generation engine. In *2022 14th International Conference on COMmunication Systems & NETworkS (COMSNETS)*, pages 129–137, Jan 2022. doi: 10.1109/COMSNETS53615.2022.9668504.
- [50] Michael V. Le, Salman Ahmed, Dan Williams, and Hani Jamjoom. Securing container-based clouds with syscall-aware scheduling. In *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*, ASIA CCS ’23, page 812 – 826, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400700989. doi: 10.1145/3579856.3582835. URL <https://doi.org/10.1145/3579856.3582835>.

- [51] CyberArk. Kubiscan: Kubernetes risk assessment tool. Available online: <https://github.com/cyberark/KubiScan>, 2024. Accessed: January 15, 2026.
- [52] Appvia. Krane: Kubernetes deployment tool. Available online: <https://github.com/appvia/krane>, 2024. Accessed: January 15, 2026.
- [53] Palo Alto Networks. Kubernetes privilege escalation: Excessive permissions in popular platforms. Available online: <https://www.paloaltonetworks.com/resources/whitepapers/kubernetes-privilege-escalation-excessive-permissions-in-popular-platforms>, 2022. Accessed: January 15, 2026.
- [54] Xuansheng Wu, Padmaja Pravin Saraf, Gyeonggeon Lee, Ehsan Latif, Ninghao Liu, and Xiaoming Zhai. Unveiling scoring processes: Dissecting the differences between llms and human graders in automatic scoring. *Technology, Knowledge and Learning*, pages 1–16, 2025.
- [55] Jorge Velez et al. Evaluating large language models as raters in large-scale writing assessments. *Learning and Instruction*, 2025.
- [56] Peiyi Wang, Lei Li, et al. Large language models are not fair evaluators. *arXiv preprint arXiv:2305.17926*, 2023.
- [57] Yerin Hwang, Dongryeol Lee, Taegwan Kang, Yongil Kim, and Kyomin Jung. Can you trick the grader? adversarial persuasion of llm judges. *arXiv preprint arXiv:2508.07805*, 2025.
- [58] Yang Liu et al. G-eval: Nlg evaluation using gpt-4 with better human alignment. *arXiv preprint arXiv:2303.16634*, 2023.
- [59] Yidong Wang, Yunze Song, Tingyuan Zhu, Xuanwang Zhang, Zhuohao Yu, Hao Chen, Chiyu Song, Qiufeng Wang, Cunxiang Wang, Zhen Wu, et al. Trustjudge: Inconsistencies of llm-as-a-judge and how to alleviate them. *arXiv preprint arXiv:2509.21117*, 2025.

附录 A 单位

以下内容可放在附录之内：

- (1) 正文内过于冗长的公式推导；
- (2) 方便他人阅读所需的辅助性数学工具或表格；
- (3) 重复性数据和图表；
- (4) 论文使用的主要符号的意义和单位；
- (5) 程序说明和程序全文；
- (6) 企业应用证明；
- (7) 项目鉴定报告；
- (8) 获奖成果证书；
- (9) 设计图纸；
- (10) 程序源代码；
- (11) 论文发表；
- (12) 作者简介。

这部分内容可省略。如果省略，删掉此页。

书写格式说明：

标题“附录 A 附录内容名称”样式为字体：黑体，英文用 Times New Roman 字体，居中，加粗，字号：小三，2.41 倍行距，段前 17 磅，段后为 16.5 磅。

附录正文样式为字体宋体小四，英文用 Times New Roman 字体小四，两端对齐书写，段落首行左缩进 2 个字符。1.3 倍行距（段落中有数学表达式时，可根据表达需要设置该段的行距），段前 0.1 行，段后 0.1 行，1.3 倍行距。

示例

表 A-1: 表 A.1 国际单位制的基本单位

量的名称	单位名称	单位符号
长度	米	m
质量	千克（公斤）	kg
时间	秒	s
电流	安〔培〕	A
热力学温度	开〔尔文〕	K
发光强度	坎〔德拉〕	cd

表 A-2: 表 A.2 国家规定的非国际单位制单位

量的名称	单位名称	单位符号	换算关系和说明
时间	分	min	1 min=60 s
	[小] 时	h	1 h=60 min=3600 s
	天(日)	d	1 d=24 h=86400 s
平面角	[角] 秒	"	$1'' = (\pi/648000) \text{ rad}$
	[角] 分	'	$1' = 1^\circ/60 = (\pi/10800) \text{ rad}$
	度	°	$1^\circ = (\pi/180) \text{ rad}$
	度	°	$1^\circ = (\pi/180) \text{ rad}$
旋转速度	转每分	r/min	1 r/min=(1/60) r/s
长度	海里	n mile	1 n mile=1852 m
速度	节	kn	1 kn=1 n mile/h=(1852/3600) m/s (只用于航行)

附录 B 云原生权限提升漏洞分类与 CWE 对应关系

表 B-1: 云原生权限提升漏洞分类框架与 CWE 对应关系

一级分类	二级分类	主要 CWE	CWE 描述
配置缺陷	冗余权限	CWE-269	不当访问控制（通用型）
		CWE-266	权限分配错误
		CWE-732	关键资源权限分配错误
	网络隔离不足	CWE-653	隔离措施不足
		CWE-668	资源暴露至错误安全域
		CWE-669	资源在不同安全域间转移错误
安全逻辑缺陷	访问控制缺陷	CWE-863	授权验证错误
		CWE-284	不当访问控制
		CWE-862	缺失授权机制
	身份验证绕过	CWE-287	不当身份验证
		CWE-290	硬编码密码进行身份验证
		CWE-327	使用存在缺陷或高风险的加密算法
	输入验证不足	CWE-20	不当输入验证
		CWE-74	输出内容中特殊元素的中和处理不当
		CWE-79	Web 页面生成过程中输入的中和处理不当
	授权验证不完善	CWE-285	授权验证不当
		CWE-250	以非必要权限执行操作
		CWE-268	特权操作执行不当
敏感信息泄露	API 端口暴露	CWE-200	敏感信息泄露给未授权主体
		CWE-358	动态识别资源的操作限制不当
		CWE-276	默认权限配置错误
	日志泄露	CWE-532	敏感信息写入日志文件
		CWE-214	存储/传输前敏感信息清除不当
		CWE-497	敏感系统信息泄露至未授权控制域
	配置文件泄露	CWE-922	敏感信息存储不安全
		CWE-200	敏感信息泄露给未授权主体
代码注入	命令注入	CWE-270	权限上下文切换错误
		CWE-78	操作系统命令中特殊元素的中和处理不当
		CWE-94	代码生成控制不当
	路径遍历	CWE-601	URL 重定向至不可信站点
		CWE-22	路径名限制不当，未限定在指定目录
		CWE-36	绝对路径遍历
		CWE-74	输出内容中特殊元素的中和处理不当

附录 C CVE-2022-24768 复现步骤与策略清单

本附录给出案例一（CVE-2022-24768）的复现详细步骤与纵深防御策略代码清单，用于复核与工程复现。为降低对生产环境的风险，建议仅在隔离测试集群中执行。

C.1 漏洞复现详细步骤

步骤 1：部署漏洞版本 Argo CD (v2.3.1)

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argocd/v2.3.1/manifests/install.yaml
kubectl rollout status deployment/argocd-server -n argocd
```

步骤 2：获取初始管理员密码并登录 UI/CLI

```
kubectl -n argocd get secret argocd-initial-admin-secret -o
  jsonpath="{.data.password}" | base64 -d && echo
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

步骤 3：创建宽松的 AppProject（用于演示风险）

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: demo-project
  namespace: argocd
spec:
  destinations:
  - namespace: '*'
    server: '*'
  sourceRepos:
  - '*'
```

```
clusterResourceWhitelist:
- group: '*'
  kind: '*'
```

EOF

步骤 4: 创建低权用户 bob, 并授予其对目标 Application 的 sync+override 权限在 argocd-rbac-cm 中添加如下策略（注意缩进）:

```
policy.default: role:readonly
policy.csv: |
    p, role:syncer, applications, sync, demo-project/*,
    ↪ allow
    p, role:syncer, applications, override, demo-project/*,
    ↪ allow
    g, bob, role:syncer
```

并在 argocd-cm 中开启账户:

```
accounts.bob: apiKey, login
```

随后重启 Argo CD Server 使配置生效:

```
kubectl rollout restart deployment/argocd-server -n argocd
```

步骤 5: 创建 Application 指向恶意仓库

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: bob-app
  namespace: argocd
spec:
  project: demo-project
  source:
    repoURL:
    ↪ https://github.com/Ivanbeethoven/bad_repo
```



```

        path: .
        targetRevision: HEAD
    destination:
        server: https://kubernetes.default.svc
        namespace: default
    syncPolicy:
        automated:
            prune: true
            selfHeal: true
EOF

```

步骤 6: 模拟 bob 触发同步（或依赖自动同步）

```

argocd login localhost:8080 --username bob --password
↪ <token-or-password>
argocd app sync bob-app --force

```

步骤 7: 验证是否提权成功

```

kubectl auth can-i '*' '*' --as bob

```

若输出为 **yes**，说明 bob 获得了集群范围的高权限（该结果用于说明风险，生产环境严禁在真实集群中进行）。

C.2 策略代码清单

C.2.1 Kubernetes RBAC: 收敛 AppProject 的写权限

将 `appprojects.argoproj.io` 的创建/更新/删除限制在平台管理员组，租户仅只读（示例中组名可按实际替换）：

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
    name: argocd-appproject-edit
rules:
- apiGroups: ["argoproj.io"]

```

```

    resources: ["appprojects"]
    verbs: ["get","list","watch","create","update","patch",
↪      "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
    name: argocd-appproject-readonly
rules:
- apiGroups: ["argoproj.io"]
    resources: ["appprojects"]
    verbs: ["get","list","watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
    name: argocd-appproject-edit-platform-admins
roleRef:
    apiGroup: rbac.authorization.k8s.io
    kind: ClusterRole
    name: argocd-appproject-edit
subjects:
- kind: Group
    name: argo:platform-admin
    apiGroup: rbac.authorization.k8s.io
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
    name: argocd-appproject-readonly-tenants
roleRef:
    apiGroup: rbac.authorization.k8s.io
    kind: ClusterRole

```

```

    name: argocd-appproject-readonly
subjects:
- kind: Group
    name: argo:tenant-readonly
    apiGroup: rbac.authorization.k8s.io

```

C.2.2 Argo CD RBAC: 默认只读 + 高风险能力隔离

建议将 argocd-rbac-cm 纳入 Git 基线管理,并在修改后滚动重启 argocd-server:

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-rbac-cm
  namespace: argocd
data:
  policy.csv: |
    g, argo:platform-admin, role:admin
    g, argo:tenant-readonly, role:readonly

    p, role:admin, projects, *, *, allow
    p, role:admin, repositories, *, *, allow
    p, role:admin, clusters, *, *, allow
    p, role:admin, applications, *, *, allow
    p, role:admin, exec, *, *, allow
    p, role:admin, logs, *, *, allow

    p, role:readonly, applications, get, *, allow
    p, role:readonly, repositories, get, *, allow
    p, role:readonly, projects, get, *, allow
    p, role:readonly, clusters, get, *, allow
    p, role:readonly, logs, get, *, allow
    p, role:readonly, exec, *, *, deny

```

```
policy.default: role:readonly
```

C.2.3 准入控制：Gatekeeper（推荐）

目标：在 AppProject 创建/更新时，对 `spec.roles[].policies[]` 进行一致性校验。本策略覆盖三类约束：

- 禁止出现 `*`，`*`，`allow` 的全局通配符放权；
- 要求 `policy subject` 绑定到项目域（`proj:<project>:<role>` 前缀）；
- 禁止在 `project` 内授予 `projects/repositories/clusters/exec` 等高危资源写权限。

(1) ConstraintTemplate

```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sargocdappprojectpolicies
spec:
  crd:
    spec:
      names:
        kind:
          ↪ K8sArgoCDAppProjectPolicies
      validation:
        openAPIV3Schema:
          properties:
            enforceProjPref_
              ↪ ix:
                type:
                  ↪ bool
                  ↪ lean
      targets:
        - target: admission.k8s.gatekeeper.sh
          rego: |
            package k8sargocdappprojectpolicies
```

```
trim_space(s) = out {
    out :=
        ↪ regex.replace("^\\s+|\\s+$",
        ↪ "", s)
}
```

1) 禁止出现 "..., *, *, allow"

↪ 的全局通配符放权

```
deny[msg] {
    input.review.kind.group ==
        ↪ "argoproj.io"
    input.review.kind.kind ==
        ↪ "AppProject"
    roles := input.review.object.sp
        ↪ ec.roles
    some i
    policies := roles[i].policies
    some j
    policy := policies[j]
    regex.match(".*,\\s*\\*,\\s*\\*
        ↪ ,\\s*allow\\s*$", policy)
    msg := sprintf("AppProject
        ↪ policy contains global
        ↪ wildcard allow: %s",
        ↪ [policy])
}
```

2) 要求 subject 采用

↪ proj:<project>:<role> 前缀 (可选开关)

↪

```
deny[msg] {
```

```

input.review.kind.group ==
  ↪ "argoproj.io"
input.review.kind.kind ==
  ↪ "AppProject"
input.parameters.enforceProjPre_
  ↪ fix == true
ap := input.review.object.metad_
  ↪ ata.name

roles := input.review.object.sp_
  ↪ ec.roles
some i
policies := roles[i].policies
some j
policy := policies[j]

startswith(policy, "p,")
parts := split(policy, ",")
count(parts) >= 2
subject := trim_space(parts[1])
not startswith(subject,
  ↪ sprintf("proj:%s:", [ap]))
msg := sprintf("AppProject
  ↪ policy subject must start
  ↪ with 'proj:%s:': %s", [ap,
  ↪ policy])
}

```

3) 禁止授予

```

  ↪ projects/repositories/clusters/exec
  ↪ 的写权限
deny[msg] {

```

```

input.review.kind.group ==
  ↪ "argoproj.io"
input.review.kind.kind ==
  ↪ "AppProject"
roles := input.review.object.sp_
  ↪ ec.roles
some i
policies := roles[i].policies
some j
policy := policies[j]
regex.match("^p,.*, (projects|re_
  ↪ positories|clusters|exec), \_
  ↪ \s*(create|update|patch|del_
  ↪ ete|\\*), .*, (allow)\\s*$",
  ↪ policy)
msg := sprintf("AppProject
  ↪ policy grants write on
  ↪ global resource: %s",
  ↪ [policy])
}

```

(2) Constraint

```

apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sArgoCDAppProjectPolicies
metadata:
  name: restrict-argocd-appproject-policies
spec:
  match:
    kinds:
      - apiGroups: ["argoproj.io"]
        kinds: ["AppProject"]
  parameters:
    enforceProjPrefix: true

```

C.2.4 准入控制：Kyverno（替代方案）

若集群已使用 Kyverno，可用如下 ClusterPolicy 实现同类校验。建议先将 `validationFailureAction` 设为 `Audit`，稳定后再切换为 `Enforce`。

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-argocd-appproject-policies
spec:
  validationFailureAction: Audit
  background: true
  rules:
    - name: forbid-global-wildcard-allow
      match:
        any:
          - resources:
              kinds:
                - argoproj.io/v1alpha1/
                ↪ AppProject
      validate:
        message: "forbid global wildcard allow
        ↪ in AppProject role policies"
        foreach:
          - list: "request.object.spec.roles[].po_
          ↪ lices[]"
            deny:
              conditions:
                any:
```



```

- key: "{{ rege
  ↪ x_match('.*'
  ↪ ,\\s*\\*,\\
  ↪ s*\\*,\\s*a
  ↪ llow\\s*$',
  ↪ element) }}"
      operato
        ↪ r:
        ↪ Equ
        ↪ als
      value:
        ↪ true

- name: enforce-proj-prefix
  match:
    any:
      - resources:
          kinds:
            - argoproj.io/v1alpha1/
              ↪ AppProject
      validate:
        message: "policy subject must use
          ↪ proj:<project>:<role> prefix"
        foreach:
          - list: "request.object.spec.roles[].po
            ↪ licies[]"
            deny:
              conditions:
                all:

```

```

- key: "{{
  ↪ regex_match_
  ↪ ('^p,\\s*pr_
  ↪ oj:[^:]+:[^_
  ↪ ,]+,',
  ↪ element) }}"
      operato_
      ↪ r:
      ↪ Equ_
      ↪ als
      value:
      ↪ fal_
      ↪ se

- name: forbid-global-resource-write
  match:
    any:
      - resources:
          kinds:
            - argoproj.io/v1alpha1/_
              ↪ AppProject

  validate:
    message: "forbid write on
      ↪ projects/repositories/clusters/exec
      ↪ in AppProject policies"
    foreach:
      - list: "request.object.spec.roles[] .po_
        ↪ licies[]"
        deny:
          conditions:
            any:

```

```
- key: "{{"
  ↪ regex_match_
  ↪ ('^p,.*(pr_
  ↪ ojects|repo_
  ↪ sitories|cl_
  ↪ usters|exec_
  ↪ ),\\s*(crea_
  ↪ te|update|p_
  ↪ atch|delete_
  ↪ |\\*),.*(a_
  ↪ llow)\\s*$',
  ↪ element) }}"
      operato_
        ↪ r:
        ↪ Equ_
        ↪ als
      value:
        ↪ true
```


附录 D CVE-2025-2241 复现步骤与策略清单

本附录给出案例二（CVE-2025-2241）的复现要点与缓解策略清单，用于复核与工程落地。该漏洞属于敏感信息泄漏：vCenter 凭据出现在 Hive 的 ClusterProvision 对象可读字段中，导致“无 Secret 权限也可读到凭据”的旁路风险。请仅在隔离测试环境中操作。

D.1 漏洞复现详细步骤

前提条件

- 已部署 ACM/MCE 与 Hive，并启用 vSphere 集群供应能力（具体版本与安装方式依赖环境，以厂商公告与实验环境为准）；
- 至少存在一个“只读身份”。该身份不具备 secrets 读取权限，但具备 clusterprovisions.hive.openshift.io 的 get/list/watch 权限（该条件用于验证旁路泄露）。

步骤 1：触发一次 vSphere 集群供应通过 ACM 控制台或 YAML 创建 Hive 的 ClusterDeployment（示例仅展示关键字段，真实环境中密码应引用 Secret；本漏洞的问题在于后续对象中出现明文暴露）：

```
apiVersion: hive.openshift.io/v1
kind: ClusterDeployment
metadata:
  name: vsphere-cluster
  namespace: your-namespace
spec:
  baseDomain: example.com
  platform:
    vsphere:
      vCenter: vc.example.com
      username: administrator@vsphere.local
      password: your-password
  provisioning:
    installConfigSecretRef:
```

```

        name: your-install-config
imageSetRef:
    name: img4.14.0

```

步骤 2：用“可读 ClusterProvision 但不可读 Secret”的身份读取对象先确认权限面，再读取 ClusterProvision：

```

# 谁能读 clusterprovisions (OpenShift)
oc adm policy who-can get clusterprovision
oc adm policy who-can list clusterprovision

# 低权用户自检 (Kubernetes 通用)
kubect1 auth can-i get clusterprovision --as=<user>
↪ --all-namespaces
kubect1 auth can-i get secrets --as=<user> --all-namespaces

# 导出对象 (名称可能带随机后缀, 按实际替换)
kubect1 get clusterprovision -n your-namespace
kubect1 get clusterprovision <name> -n your-namespace -o yaml

```

步骤 3：定位敏感字段在输出中查找类似结构（位置因版本实现而异，示例以 `status.vCenter.credentials` 为代表）：

```

status:
  vCenter:
    credentials:
      username: ...
      password: ...

```

若凭据以明文形式出现，即满足漏洞复现判据。

D.2 策略代码清单

本节策略以“收敛读取面 + 缩短暴露窗口 + 审计发现与轮换止血”为主线。由于敏感信息出现在控制器写入的 `status` 字段中，准入控制更适合作为审计/检测手段，不建议强制拦截控制器更新（否则可能影响供应流程）。

D.2.1 1) 立刻收敛 ClusterProvision 的读取权限 (RBAC)

风险盘点命令 (OCP)

```
oc adm policy who-can get clusterprovision
oc adm policy who-can list clusterprovision
oc adm policy who-can watch clusterprovision
```

最小可读权限示例 (仅绑定 **Hive** 控制器服务账号) 说明: Hive 控制器的具体 ServiceAccount/namespace 需按实际部署确认。

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: hive-clusterprovision-read-min
rules:
- apiGroups: ["hive.openshift.io"]
  resources: ["clusterprovisions"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: hive-clusterprovision-read-min-binding
subjects:
- kind: ServiceAccount
  name: hive-controllers
  namespace: hive
roleRef:
  kind: ClusterRole
  name: hive-clusterprovision-read-min
  apiGroup: rbac.authorization.k8s.io
```

D.2.2 2) 对读取行为启用审计并联动告警

Kubernetes 审计策略片段 (加载方式依平台而定):

```

apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
  verbs: ["get","list","watch"]
  resources:
  - group: "hive.openshift.io"
    resources: ["clusterprovisions"]
    omitStages: ["RequestReceived"]

```

建议在 SIEM/日志平台中将 Hive 控制器主体加入白名单，仅对非白名单主体的读取行为告警。

D.2.3 3) 缩短暴露窗口：供应完成后清理 ClusterProvision

手动清理

```

oc get clusterprovision -n <ns>
oc delete clusterprovision -n <ns> <name>

```

自动化清理（示例 **CronJob**）说明：删除条件建议通过标签或状态字段精确匹配，避免误删；示例以标签 `hive.openshift.io/retain=false` 为条件。

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: cleanup-sa
  namespace: hive
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: cleanup-role
  namespace: hive
rules:

```



```

- apiGroups: ["hive.openshift.io"]
  resources: ["clusterprovisions"]
  verbs: ["list","get","delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cleanup-rb
  namespace: hive
subjects:
- kind: ServiceAccount
  name: cleanup-sa
roleRef:
  kind: Role
  name: cleanup-role
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cleanup-clusterprovision
  namespace: hive
spec:
  schedule: "0 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName:
            ↪ cleanup-sa
          containers:
            - name: cleaner

```

```
image: registry_
↳ .access.red_
↳ hat.com/ubi_
↳ 9/ubi:latest
command: ["/bin_
↳ /sh", "-c"]
args:
- |
    set -eu
```

```
oc delete
→ te
→ clu
→ ste
→ rpr
→ ovi
→ sion
→ -n
→ hive
→ -l
→ hiv
→ e.o
→ pen
→ shi
→ ft.
→ io/
→ ret
→ ain
→ =fa
→ lse
→ --i
→ gno
→ re-
→ not
→ -fo
→ und
```

restartPolicy: OnFailure

D.2.4 4) vCenter 凭据轮换与强化（止血）

建议在确认泄露后立即轮换 vCenter 凭据，并启用最小权限、MFA、来源 IP 允许列表等约束。轮换完成后更新 Hive 使用的 Secret（示例命令需按实际 Secret 名称/namespace 调整）：

```
oc -n hive create secret generic vcenter-credentials \
  --from-literal=username=<new-user> \
  --from-literal=password=<new-pass> \
  --dry-run=client -o yaml | oc apply -f -
```

D.2.5 5) “敏感字段守护”检测（Gatekeeper/Kyverno, 建议审计模式）

Gatekeeper (dryrun)：检测 ClusterProvision 中是否出现敏感键名（password/token/secret/credential）。

```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sclustprovkeysensitive
spec:
  crd:
    spec:
      names:
        kind: K8sClustProvKeySensitive
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8sclustprovkeysensitive

        sensitive_key(k) {
          lk := lower(k)
          contains(lk, "password")
        }

        sensitive_key(k) {
          lk := lower(k)
          contains(lk, "token")
        }

        sensitive_key(k) {
```

```

        lk := lower(k)
        contains(lk, "secret")
    }

    sensitive_key(k) {
        lk := lower(k)
        contains(lk, "credential")
    }

# 深度遍历 object, 收集所有键
walk(obj, path, val) {
    is_object(obj)
    some k
    v := obj[k]
    walk(v, array.concat(path, [k]),
        ↪ val)
}

walk(obj, path, val) {
    is_array(obj)
    some i
    v := obj[i]
    walk(v, array.concat(path,
        ↪ [sprintf("%v", [i])]), val)
}

walk(obj, path, obj) {
    not is_object(obj)
    not is_array(obj)
}

deny[msg] {
    input.review.kind.group ==
        ↪ "hive.openshift.io"
    input.review.kind.kind ==
        ↪ "ClusterProvision"
}

```

```

        walk(input.review.object, [], _)
        some k
        _ = input.review.object[k]
        sensitive_key(k)
        msg := sprintf("ClusterProvision
        ↪ contains sensitive key at
        ↪ top-level: %s", [k])
    }

    deny[msg] {
        input.review.kind.group ==
        ↪ "hive.openshift.io"
        input.review.kind.kind ==
        ↪ "ClusterProvision"
        walk(input.review.object, path,
        ↪ _)
        some i
        k := path[i]
        sensitive_key(k)
        msg := sprintf("ClusterProvision
        ↪ contains sensitive key on
        ↪ path %v", [path])
    }
}

---
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sClustProvKeySensitive
metadata:
  name: audit-clusterprovision-sensitive-keys
spec:
  enforcementAction: dryrun
  match:
    kinds:

```

```
- apiGroups: ["hive.openshift.io"]
  kinds: ["ClusterProvision"]
```

Kyverno (Audit): 对对象文本进行检测，以提示可能存在的敏感键名。
该策略更适用于告警与排查，不建议直接用于阻断控制器更新。

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: audit-clusterprovision-sensitive-keys
spec:
  validationFailureAction: Audit
  background: true
  rules:
    - name: detect-secret-like-keys
      match:
        any:
          - resources:
              kinds:
                - hive.openshift.io/v1/ClusterProvision
      validate:
        message: ClusterProvision
        ↪ 可能包含敏感字段 password token
        ↪ secret credential。
      deny:
        conditions:
          any:
            - key: "{{ to_string(request.object) }}"
              operator: AnyIn
              value:
                - password
                - token
```

- secret
- credential

附录 E CVE-2023-30512 复现步骤与策略清单

本附录给出案例三（CVE-2023-30512, CubeFS CSI 插件过度授权导致的集群权限提升风险）的验证性复现与策略清单。为避免提供可被滥用的细粒度利用操作，复现部分以“权限证据 + 受控权限验证”为主，重点证明 CSI ServiceAccount 被绑定了不必要的 secrets 读取能力；策略部分给出可落地的 RBAC 收敛、准入防回归与审计检测。

E.1 漏洞复现详细步骤

变量约定（按实际替换）

```

NAMESPACE=<cubeFS-namespace>
SA=cfs-csi-service-account
CLUSTERROLE=cfs-csi-cluster-role
    
```

如 SA 名称不同，请先定位：

```

kubectl get sa -A | findstr /i "csi cfs cubeFS"
kubectl get clusterrole,clusterrolebinding -A | findstr /i "csi
↪ cfs cubeFS"
    
```

步骤 1：导出 RBAC 证据（确认 secrets 权限存在）

```

kubectl get clusterrole $CLUSTERROLE -o yaml
kubectl get clusterrolebinding -o yaml | findstr /i "cfs-csi
↪ cubeFS"
    
```

重点检查 ClusterRole 规则中是否出现：

```

# resources: ["secrets"]
# verbs: ["get","list","watch"]
    
```

步骤 2：受控权限验证（不执行实际数据提取）

直接验证该 SA 是否具备跨命名空间读取 secrets 的能力

```

kubectl auth can-i get secrets
↪ --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
    
```

```
kubect1 auth can-i list secrets
```

```
↪ --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
```

```
kubect1 auth can-i watch secrets
```

```
↪ --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
```

权限提升路径（文字图，便于复盘）

CubeFS CSI 安装环境



一旦攻击者控制节点或 CSI Pod => 可滥用该 SA 的身份边界

（风险：读取高价值凭据 -> 演化为集群级权限提升）

E.2 策略代码清单

本节策略遵循“最小权限 + 防回归 + 可观测”主线：先移除 CSI 不必要的 `secrets` 读取权限；再用准入策略阻断未来再次引入过度授权；最后启用审计检测以便快速发现异常访问并触发止血流程。

E.2.1 1) RBAC 最小化：移除 ClusterRole 中的 secrets 权限

说明：CSI 需要的资源/动词与具体实现有关，建议在预生产以 PVC/PV 创建、挂载、扩容、attach/detach 等流程回归验证后再推广到生产。以下示例展示“覆盖式”更新 ClusterRole 的方式，明确不包含 `resources: ["secrets"]`。

```
cat <<'EOF' | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cfs-csi-cluster-role
  labels:
    app.kubernetes.io/name: cfs-csi
rules:
- apiGroups: [""]
  resources: ["nodes","persistentvolumes","pods"]
  verbs: ["get","list","watch"]
- apiGroups: ["storage.k8s.io"]
  resources: ["volumeattachments","csinodes"]
  verbs: ["get","list","watch"]
# 注意：不应包含 resources:["secrets"]
EOF

# 复测（期望输出 no）
kubectl auth can-i list secrets
→ --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
```

E.2.2 2) ServiceAccount 硬化（按需评估）

若 CSI 驱动不需要直接调用 Kubernetes API，可评估禁用自动挂载令牌以降低“被攻陷即可滥用身份”的风险：

```
# 将 automountServiceAccountToken 设为 false（需评估 CSI
→ 功能是否依赖 token）
kubectl -n $NAMESPACE patch serviceaccount $SA --type='merge' -p
→ '{"automountServiceAccountToken":false}'
```

E.2.3 3) 准入防回归 (OPA Gatekeeper): 禁止创建包含 secrets 读取动词的 ClusterRole

说明：该策略用于防止未来误配置再次引入过度授权。建议先以 dryrun 运行观察影响，再逐步强制。

```
cat <<'EOF' | kubectl apply -f -
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sdenysecretsrbac
spec:
  crd:
    spec:
      names:
        kind: K8sDenySecretsRBAC
      targets:
        - target: admission.k8s.gatekeeper.sh
          rego: |
            package k8sdenysecretsrbac

            has_secrets_rule(rule) {
              rule.resources[_] == "secrets"
              rule.verbs[_] == "get"
            }

            has_secrets_rule(rule) {
              rule.resources[_] == "secrets"
              rule.verbs[_] == "list"
            }

            has_secrets_rule(rule) {
              rule.resources[_] == "secrets"
              rule.verbs[_] == "watch"
            }
```

```
violation[{"msg": msg}] {
    input.review.kind.kind ==
    ↪ "ClusterRole"
    some i
    rule :=
    ↪ input.review.object.rules[i]
    has_secrets_rule(rule)
    msg := "ClusterRole must not
    ↪ grant get/list/watch on
    ↪ secrets"
}
```

EOF

```
cat <<'EOF' | kubectl apply -f -
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sDenySecretsRBAC
metadata:
    name: deny-clusterrole-secrets-read
spec:
    enforcementAction: dryrun
    match:
        kinds:
        - apiGroups: ["rbac.authorization.k8s.io"]
          kinds: ["ClusterRole"]
```

EOF

E.2.4 4) 审计与检测：对 secrets 枚举行为进行可观测化

Kubernetes 审计策略片段（加载方式依平台而定）：

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
```

```

verbs: ["get","list","watch"]
resources:
- group: ""
  resources: ["secrets"]
omitStages: ["RequestReceived"]

```

建议在 SIEM/日志平台中对来自 CSI ServiceAccount 的 secrets 枚举进行聚合告警，并结合阈值与白名单降低噪音。

E.2.5 5) 网络收敛（可选）：限制 CSI Pod 出站访问面

说明：NetworkPolicy 能降低被攻陷工作负载的横向移动能力，但在不同 CNI 下支持情况不同；同时 API Server 可能位于外部 LB，需按实际环境调整。

```

cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: cfs-csi-egress-lockdown
  namespace: $NAMESPACE
spec:
  podSelector:
    matchLabels:
      app: cfs-csi
  policyTypes: ["Egress"]
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
    podSelector:
        matchLabels:
          k8s-app: kube-dns
  ports:

```

```
- protocol: UDP  
  port: 53
```

EOF

附录 F CVE-2020-4062 复现步骤与策略清单

本附录给出案例四（CVE-2020-4062，Conjur OSS Helm Chart 旧版本缺失网络隔离导致 Postgres 在集群内可达）的复现要点与缓解策略清单。考虑到该问题涉及数据库直连带来的敏感数据风险，复现环节仅进行网络暴露面验证，用于确认 5432 端口在集群内的可达性以及网络策略的生效性，不展开数据库操作细节。

F.1 漏洞复现详细步骤

前提条件

- 集群 CNI 支持 NetworkPolicy，例如 Calico。
- Helm v3 可用。
- 受影响的 Conjur OSS Helm Chart 版本为 < 2.0.0。

步骤 1：在隔离命名空间部署旧版 Chart，并进行必要的兼容性适配旧版 Chart 可能使用过时 API，例如 apps/v1beta2。在较新 Kubernetes 版本上复现时，可在保持网络策略缺失这一特性不变的前提下，将 Deployment API 适配为 apps/v1 并补齐 selector 字段以完成安装。

步骤 2：确认 Postgres Service 与 5432 端口暴露

NAMESPACE=conjur-vuln

```
kubectl get ns $NAMESPACE
```

```
kubectl -n $NAMESPACE get svc
```

重点确认 Postgres Service（名称以实际为准）与 5432 端口存在

```
kubectl -n $NAMESPACE get svc | findstr /i "postgres 5432
```

```
↪ conjur"
```

步骤 3：确认未配置 NetworkPolicy

```
kubectl -n $NAMESPACE get networkpolicy
```

若不存在任何约束 Postgres 入站的策略，且集群默认允许 Pod 间互通，则可判定数据库对集群内工作负载开放。

步骤 4：连通性验证在不执行数据库操作的前提下，使用 TCP 连通性探测验证非 Conjur 组件工作负载是否能够与 Postgres 的 5432 端口建立连接，以此作为暴露面存在的验证依据。

F.2 策略代码清单

F.2.1 1) 缓解措施：NetworkPolicy，默认拒绝并仅放通 Conjur Server 访问 Postgres 5432

下述策略在命名空间内实现两点：其一，默认拒绝 Postgres Pod 的入站流量；其二，仅允许 Conjur Server 访问 Postgres 的 5432 端口。其中标签选择器需与实际部署对象一致，示例使用 `app=conjur-oss` 与 `app=conjur-oss-postgres`，可按实际标签调整。

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: conjur-postgres-default-deny
  namespace: conjur-vuln
spec:
  podSelector:
    matchLabels:
      app: conjur-oss-postgres
  policyTypes:
  - Ingress
  ingress: []
EOF
```

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: conjur-postgres-allow-conjur
```

```

        namespace: conjur-vuln
spec:
  podSelector:
    matchLabels:
      app: conjur-oss-postgres
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: conjur-oss
      ports:
        - protocol: TCP
          port: 5432
EOF

```

策略验证

```
kubectl -n conjur-vuln get networkpolicy
```

```

# 复测：从非 Conjur 组件工作负载到 Postgres 5432 的连通性应失败
# 从 Conjur Server 到 Postgres 5432 的连通性应成功

```

F.2.2 2) 命名空间隔离与 RBAC 收敛

- 将 Conjur 部署在独立命名空间，并通过 NetworkPolicy 限制跨命名空间访问。
- 收敛对数据库凭据相关 Secret 的读取权限，降低网络可达与凭据可读叠加带来的风险。

附录 G CVE-2022-24877 复现步骤与策略清单

本附录给出案例五（CVE-2022-24877，Flux kustomize-controller 通过恶意 kustomization.yaml 触发路径穿越，导致控制器 Pod 文件系统敏感数据暴露，并在多租户部署中可能引发权限提升）的验证性复现与缓解策略清单。为避免提供可直接滥用的利用细节，复现部分以“路径越界行为证据与判据”为主，不提供可复用的攻击脚本。

G.1 漏洞复现详细步骤

复现目标验证控制器在构建过程中是否会因配置文件中的异常路径引用而产生目录边界越界读取风险。重点关注构建阶段的路径规范化与边界约束，并以日志行为与构建产物的异常特征作为证据判据。本案例涉及的控制器为 kustomize-controller，相关配置文件为 kustomization.yaml。

环境与前提建议在隔离测试集群中进行，并使用受影响版本的 kustomize-controller。在多租户集群中，该类问题可能放大隔离破坏风险，故不建议在生产环境进行实验。修复信息：该漏洞已在 kustomize-controller v0.24.0 修复，并随 flux2 v0.29.0 集成发布。

步骤要点（受控）

1. 在隔离仓库中准备最小化示例，包含 kustomization.yaml 及少量合法资源清单，用于建立可对比的基线构建产物。
2. 在受控条件下引入异常路径引用（例如包含父目录引用或绝对路径风格的条目），触发控制器重新拉取并执行构建。
3. 观察控制器日志与事件输出，记录与路径解析、文件读取失败或异常引用相关的报错/告警。
4. 对比构建产物与基线差异，检查是否出现“非预期文件内容进入产物”的迹象。该步骤应仅使用无敏感性的测试素材进行验证。

证据判据与逻辑解释该类漏洞的关键证据在于“路径解析是否受目录边界约束”。若控制器在构建阶段对异常路径引用给出明确的拒绝与报错，并且构建产物未出现越界读取痕迹，可视为风险被抑制。若在严格受控的测试条件下观察到异常路径引用导致非预期内容进入构建产物，则可判定目录边界约束存在缺口，需立即升级组件并补齐流程治理措施。

G.2 策略代码清单（优先级方案）

本节选取对该类风险最直接且工程可落地的缓解措施：CI/CD 前置校验，并辅以多租户下的变更治理与运行时硬化。

G.2.1 1) 工作区缓解：在 CI/CD 中校验 kustomization.yaml 路径策略

在补丁窗口期或多团队协作场景下，可在 CI/CD 流水线中加入配置校验门禁。校验对象为 kustomization.yaml，要求其资源路径满足安全策略，例如：禁止绝对路径风格、禁止父目录引用、限制引用的文件类型与目录范围。下述示例为 conftest (OPA) 策略骨架，用于展示如何对 kustomization.yaml 的路径字段进行约束；可按组织规范扩展字段列表与白名单策略。

```
package kustomizepolicy
```

```
deny[msg] {
    some p
    p := input.resources[_]
    risky_path(p)
    msg := sprintf("kustomization.yaml: resources contains risky
    ↪ path: %s", [p])
}
```

```
deny[msg] {
    some i
    p := input.patches[i].path
    risky_path(p)
    msg := sprintf("kustomization.yaml: patches[%d].path contains
    ↪ risky path: %s", [i, p])
}
```

```
risky_path(p) {
    startswith(p, "/")
}
```

```
}

risky_path(p) {
    contains(p, "..")
}
```

说明：为降低误报，可进一步将 `contains(p, "..")` 改为按路径分段匹配“父目录段”，并对允许的相对路径前缀建立白名单。

G.2.2 2) 多租户变更治理与运行时硬化（纵深防御）

在多租户集群中，应严格控制谁可以向 Flux 同步源提交变更、谁可以修改 Flux 相关资源，并通过代码评审与分支保护降低恶意配置进入同步面的概率。同时可对控制器工作负载启用非特权运行、只读根文件系统、默认 `seccomp` 配置与能力集收敛，以降低异常路径下的后续风险。

附录 H CVE-2022-29171 复现步骤与策略清单

本附录给出案例六（CVE-2022-29171，Sourcegraph 在 Gitolite 与 Phabricator 联动集成中存在配置驱动的命令执行风险）的验证性复现与缓解策略清单。为避免提供可直接滥用的利用细节，复现部分以执行路径证据与回归判据为主，不给出可直接用于远程执行的命令载荷与请求模板。

H.1 漏洞复现详细步骤

复现目标验证在受影响版本中，特定集成配置项（`callsignCommand`）是否会被 `gitserver` 在处理仓库元数据获取流程时执行。复现以日志、审计与可观测证据为主，并在禁用集成后复测，确认风险被消除。

环境与前提仅在隔离测试环境开展。前置条件包括：（i）Sourcegraph 版本处于受影响范围；（ii）启用了 Gitolite code host 并与 Phabricator 元数据获取路径相关联；（iii）存在可编辑/新增 Gitolite code host 的管理权限；（iv）存在对内置 Grafana 的高权限访问，用于说明观测面可能降低内部寻址成本。

步骤要点（受控）

1. 部署受影响版本 Sourcegraph，并确认 `gitserver` 作为独立工作负载运行（记录其命名空间与标签信息）。
2. 在管理面将 Gitolite 与 Phabricator 集成配置为可用状态，保证仓库元数据获取流程可被触发。
3. 在严格受控条件下对 `callsignCommand` 进行无害探针式变更（例如仅返回固定标识、且不访问外部网络、不读取敏感文件）。
4. 触发一次与该集成相关的元数据获取/同步流程，并观察 `gitserver` 日志、任务记录或审计日志中是否出现外部命令执行的迹象。
5. 禁用 Gitolite code host 后重复触发流程，确认相同迹象不再出现。

证据判据与逻辑解释若在受影响版本中，`gitserver` 的运行日志或可观测数据中出现因集成配置而执行外部命令的明确迹象，可判定存在执行面风险。禁用相关集成后，该执行路径应被移除或被显式限制；若仍可观察到可疑执行迹象，则需进一步检查是否存在配置残留或旁路路径。

H.2 策略清单

本节仅选取 3 条对该类风险最直接且工程可落地的策略：功能禁用、网络隔离与权限分离/暴露面收敛。

H.2.1 1) 快速缓解：禁用或移除 Gitolite code host（若业务允许）

若业务不依赖 Gitolite，可移除所有 Gitolite code host 配置并保持禁用状态，从输入面直接消除 `callsignCommand` 的风险来源。

H.2.2 2) 网络隔离：限制 gitserver 的可达面（默认拒绝并按需放通）

在 Kubernetes 部署中，建议对 gitserver 应用 NetworkPolicy：仅允许 Sourcegraph 内部必要组件访问其入站端口；出站仅放通 DNS 与必要的代码托管出口。该策略用于降低内部拓扑信息被获取后的横向触达能力，并为权限误授提供纵深缓冲。

策略示例分别约束入站与出站。入站仅允许 Sourcegraph 内部调用方访问 gitserver；出站仅放通 DNS，并对代码托管出口按需单独放通。选择器与端口应按实际部署对象调整。

NAMESPACE=sourcegraph

1) 入站收敛：仅允许 Sourcegraph 内部组件访问 gitserver

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: gitserver-ingress-allow-internal
  namespace: sourcegraph
spec:
  podSelector:
    matchLabels:
      app: gitserver
```

```

policyTypes: ["Ingress"]
ingress:
- from:
  - podSelector:
      matchLabels:
        app:
          ↪ sourcegraph-frontend
  - podSelector:
      matchLabels:
        app: repo-updater
  ports:
  - protocol: TCP
    port: 3178
EOF

```

2) 出站收敛: 仅放通 DNS; 代码托管出口按需另行放通

```

cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: gitserver-egress-dns-only
  namespace: sourcegraph
spec:
  podSelector:
    matchLabels:
      app: gitserver
  policyTypes: ["Egress"]
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          ↪ name: kube-system

```

```
        podSelector:
            matchLabels:
                k8s-app: kube-dns

    ports:
        - protocol: UDP
            port: 53
        - protocol: TCP
            port: 53
EOF

# 验证：策略已创建，且 gitserver 的出站/入站符合预期
kubectl -n $NAMESPACE get networkpolicy | findstr /i "gitserver"
```

验证要点建议先在预生产灰度启用，并结合 gitserver 的错误日志与调用链路指标观察误拦截情况。对确需访问的代码托管出口，按最小集合逐项放通，并保留变更记录。

H.2.3 3) 权限分离与暴露面收敛：限制管理面与观测面的高权限

严格限制可编辑外部服务配置（尤其是 Gitolite code host 配置）的主体数量，并启用变更审批与审计。同时收敛 Grafana 的暴露面，例如仅内网访问、受控 Ingress、强认证与最小管理员集合，并将 Grafana 管理员与 Sourcegraph 站点管理员职责分离，避免观测面权限放大攻击链的信息优势。

落地要点在平台流程上，可将外部服务配置变更与观测面账号授权纳入同一类高风险变更，要求双人复核、强制留痕与定期回收。在技术控制上，可对管理入口启用强认证并限制来源网络，同时在日志平台中对关键配置项变更进行集中审计与告警。

附录 I CVE-2023-22482 复现步骤与策略清单

本附录给出案例七（CVE-2023-22482，Argo CD 在 OIDC 场景下未校验令牌受众 `aud`，导致跨受众令牌可能被接受）的验证性复现与缓解策略清单。为避免提供可直接复用的越权访问细节，复现部分以配置证据、令牌声明证据与接口返回判据为主，不展开可直接用于获取令牌与构造请求的具体命令。

I.1 漏洞复现详细步骤

复现目标验证在受影响版本中，Argo CD 是否会接受 `aud` 不匹配但签名与发行者合法的 OIDC 令牌，并据此返回受 RBAC 控制的资源响应。

环境与前提仅在隔离测试环境开展。前置条件包括：（i）部署受影响版本的 Argo CD（例如 2.5.4），并启用 OIDC 登录；（ii）OIDC 提供方能够为至少两个不同受众签发令牌，且其中一个受众并非 Argo CD；（iii）Argo CD 已配置为信任该 OIDC 提供方，并启用 `groups` 相关声明用于 RBAC 映射；（iv）具备可观察的审计与日志渠道，用于对登录校验与资源访问结果进行证据采集。

步骤要点（受控）

1. 固化基线配置：记录 Argo CD 版本与 OIDC 配置要点（`issuer`、`clientID`、所请求的 `scope`），并确认 Argo CD 能够正常使用 OIDC 完成会话建立。
2. 获取一枚由同一 OIDC 提供方签发、但 `aud` 指向其他受众的有效令牌。该令牌应满足验签可通过且包含可用于 RBAC 映射的 `groups` 声明。
3. 提取令牌声明证据：对该令牌解析其 `payload`，保留 `iss`、`aud`、`exp` 与 `groups` 等字段的记录，证明其受众并非 Argo CD。
4. 使用该令牌访问 Argo CD 的受保护接口，选择一个只读且便于判定的 API 作为验证点，记录响应结果与 Argo CD 侧日志。

证据判据与逻辑解释若在受影响版本中，Argo CD 在验签通过的前提下接受 `aud` 不匹配的令牌，并返回受 RBAC 控制的资源响应，可判定存在受众校验缺失。

I.2 策略清单

本节仅选取 3 条对该类风险最直接且工程可落地的策略：身份提供方客户端隔离、前置受众校验与 RBAC 收敛。

I.2.1 1) 身份提供方治理：独立客户端与唯一受众

为 Argo CD 配置独立 OIDC 客户端与唯一受众标识，避免与其他业务系统共享同一受众空间。同时对该客户端签发的 groups 声明进行裁剪，确保仅包含与 Argo CD 授权相关的组，并建立清晰的组命名规范，避免跨系统复用带来的权限含义漂移。

I.2.2 2) 入口补偿控制：前置强制校验 aud

在 Argo CD 前置入口（如 API 网关或反向代理）对令牌进行一致性校验，至少包含发行者、签名与受众校验。落地时应提供灰度与回滚机制，并覆盖 Web 与自动化访问路径。

I.2.3 3) 权限收敛：RBAC 最小化与项目边界约束

收敛 argocd-rbac-cm 中的角色授权，将高权限组限制在最小集合，并通过 AppProject 限定可部署的命名空间、资源类型与来源仓库。同时将关键接口的访问失败与异常登录事件纳入审计与告警，以便在令牌泄露或误用时快速发现与止血。

致 谢

致谢是作者对该论文的形成作出过贡献的组织或个人予以声明的文字记载，语言要客观、准确、简短。

字数一般不超过 500 字，最多不超过一页。论文作者可以在致谢页对下列方面致谢：

国家科学基金、资助研究工作的奖学金基金，合同单位、资助或支持的企业、组织或个人；

协助完成研究工作和提供便利条件的组织或个人；

在研究工作中提出建议和提供帮助的人；

给予转载和引用权的资料、图片、文献、研究思想和设想的所有者；

其它应感谢的组织或个人。

作者简历及在学研究成果

一、主要教育经历/工作经历（从大学起，到硕士入学止）

起止年月	学习或工作单位	备注
2019 年 9 月至 2023 年 6 月	在北京科技大学计算机科学与技术专业攻读学士学位	

二、在学期间从事的科研工作

三、在学期间所获的科研奖励

四、在学期间发表的论文

- [1] Sun C, **Han X**, Gong Y. KubeGuard: A Systematic Permission-Oriented Risk Detection Approach for Kubernetes Applications[C]//2025 IEEE International Conference on Web Services (ICWS). IEEE, 2025: 855-865.

- [2] 第二作者（导师一作）.ICWS.CCF-B.2025.

盲审说明（正式写作请删除此说明）：

盲审论文需要隐藏掉所有会影响盲审结果的论文作者及其导师的信息，以便论文评阅人能够公正的进行评阅。

当学校要求提供盲审论文时，请按如下方法制作。

1) 对于论文中下列学生和导师信息。请将学生姓名、学生学号、导师姓名，依次全部替换为[本论文作者]、[论文作者学号]、[本论文导师]。论文中上述信息均需要替换，包括作者研究成果等部分的有关信息。

2) 在研究成果中，论文作者发表的文章列表中应隐去所有作者的名字，只标明论文期刊名，级别，发表年份。

3) 盲审论文，请不要填写致谢，致谢页除标题、页眉、页码外请保持空

白。

4) 其他会影响盲审结果的信息, 请采用类似方式处理。

5) 提交的盲审论文应为正式论文, 除了上述替换后的信息外, 应为可以评阅的正式论文。

6) 论文封面, 请填写专业名称、论文题目等, 将学号和姓名项目保持空白不填。制作盲审论文时, 论文书脊、封二、题名页请删除。

替换前后将文档分别保存, 以便盲审论文与其他论文分开管理。

盲审样例 (正式写作请删除此样例):

五、在学期间发表的论文:

[1] **第一作者**. 中国矿业. 中文核心期刊.2020.

[2] **第二作者 (导师一作)**. 中国矿业. 中文核心期刊.2020.

独创性声明

本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的
研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包
含其他人已经发表或撰写过的研究成果，也不包含为获得北京科技大学或其
它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所
做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：_____ 日期：_____ 年_____ 月_____ 日

备注：提交时须有作者亲笔签名！

关于论文使用授权的说明

本人完全了解北京科技大学有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

（保密的论文在解密后应遵循此规定）

签名：_____ 导师签名：_____ 日期：_____

学位论文数据集

关键词 *	密级 *	中图分类号 *	UDC	论文资助
学位授予单位名称 *		学位授予单位代码 *	学位类别 *	学位级别 *
北京科技大学		10008		
论文题名 *		并列题名		论文语种 *
作者姓名 *			学号 *	
培养单位名称 *		培养单位代码 *	培养单位地址	邮编
北京科技大学		10008	北京市海淀区学院路 30 号	100083
学科专业 *		研究方向 *	学制 *	学位授予年 *
论文提交日期 *				
导师姓名 *			职称 *	
评阅人	答辩委员会主席 *	答辩委员会成员		
电子版论文提交格式文本 () 图像 () 视频 () 音频 () 多媒体 () 其他 () 推荐格式: application/msword; application/pdf				
电子论文出版 (发布) 者		电子论文出版 (发布) 地		权限声明
论文总页数 *				
共 33 项, 其中带 * 为必填数据, 为 22 项。				